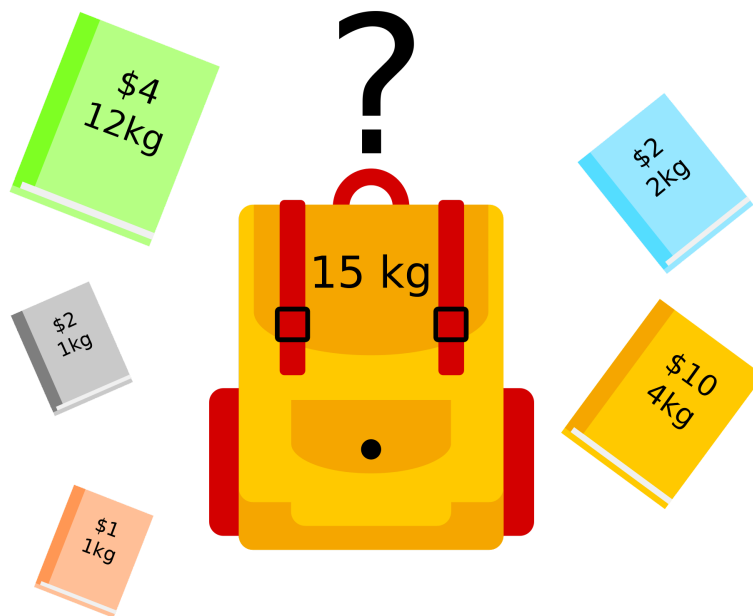


A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the Knapsack Problem

Analysis Section

Name: Aarav Bhatnagar - Centre Number: 39253 - Candidate Number: 8015

1 - The Problem



knapsackproblem.com

Table of Contents

- 1 - The Problem (pg 2)
 - A brief introduction to the problem proposed (pg 2)
 - What is the Knapsack Problem? (pg 2)
 - What are the applications of the Knapsack Problem? (pg 3)
 - Why did I choose this topic and specifically the knapsack problem for the project? (pg 3)
- 2 - Research around the Problem (pg 4)
 - A brief introduction into P vs NP (pg 4)
 - What is the difference between NP-hard and NP-complete problems? (pg 4)
 - What are the different types of knapsack problems? (pg 5)

- *Further problem research* (pg 6)
 - Quadratic Knapsack Problem (QKP) (pg 6)
 - Multidimensional (Multiconstraint) Knapsack Problem (MKP) (pg 6)
- *Further research into knapsack cryptosystems* (pg 7)
- 3 - Research into Algorithms that solve the Knapsack Problem (pg 8)
- 4 - User Identification and User Needs (pg 10)
- 5 - Primary User (pg 10)
 - Summary of Primary User Needs (pg 12)
- 6 - Prospective Users (pg 12)
 - Summary of Prospective Users Needs (pg 20)
- 7 - Analysis of Current Systems (pg 20)
 - 1 - Knapsack Algorithm Visualisation (pg 20)
 - Summary Table (pg 22)
 - 2 - Genetic Algorithm Visualization Tool (pg 22)
 - Summary Table (pg 23)
- 8 - Objectives (pg 23)
 - Acceptable Limitations (pg 25)
- 9 - Modelling (pg 26)
- 10 - Bibliography (pg 27)

A brief introduction to the problem proposed

The problem that I first proposed to my teacher, Ms Lynn, who teaches the 'Theory of Computation' and 'Fundamental of Algorithms' modules at my school, was *how can we more interactively demonstrate the relative time complexity and accuracy of exact and heuristic algorithms for intractable problems*. She suggested that for my NEA, I create an application to be used in higher education that explores and solves a 'hard' computational problem as a form of exploring these topics practically. This concept underlines the motivation and drive behind the project and will influence my design decisions along the way.

What is the Knapsack Problem?

There are several variations of the knapsack problem (also known as the backpack problem) that I will explore further in my analysis section, but in its simplest form (the 0-1 knapsack optimisation problem) may be defined as:

Given a set of 'n' items where each item 'i' has a weight and a value (w_i, v_i) and a knapsack with a maximum capacity 'W', what subset of items maximises the total value of the knapsack without exceeding its weight limit.

$$\begin{aligned}
&\text{Maximize} && \sum_{i=1}^n v_i x_i \\
&\text{subject to} && \sum_{i=1}^n w_i x_i \leq W, \\
&&& x_i \in \{0, 1\}, \quad i = 1, \dots, n
\end{aligned}$$

It is a classic combinatorial optimization problem known by many computer scientists and programmers alike. **My goal is to develop a platform which may be used in education and research that specifically looks at implementing complex algorithms that solve the knapsack problem, compares these algorithms against each other, and visualises how each algorithm works.**

What are the applications of the Knapsack Problem?

The direct applications of the knapsack Problem include (Backpack Problem, 2025):

1. A shipping company trying to maximise the package value of a cargo ship or plane
2. Hedge funds selecting investments in a portfolio that maximise potential gains while keeping the risk (VaR) below a given threshold
3. Professional sports clubs trying to build a team that meets various statistical projections without exceeding the operating budget
4. The **knapsack cryptosystem** is a public-key cryptosystem based on a special case of the knapsack problem developed by Ralph Merkle and Martin Hellman in 1978 where the solution is very time-consuming to calculate as it belongs to the complexity class of NP

Why did I choose this topic and specifically the knapsack problem for the project?

I chose the topic as I have always been interested in the infamous **P vs NP** problem and the boundary of computation. I hope that this platform provides people with a better understanding of the field of **algorithmic optimisation** and compares the techniques at use. This project should also help me learn more about the time and space complexity of different larger algorithms and the accuracy of heuristic approaches. Furtherly, I wanted to explore **genetic algorithms** that are less explicitly defined but may be fine tuned to produce powerful results.

I picked the knapsack problem because it ties in well here as it is one of **Karp's 21 NP-complete problems**, it is simple to understand and relates to lots of practical applications. It is also a lot more visual and simpler to understand compared to other NP problems like the boolean satisfiability problem, however I may explore the relationship and reducibility between other problems.

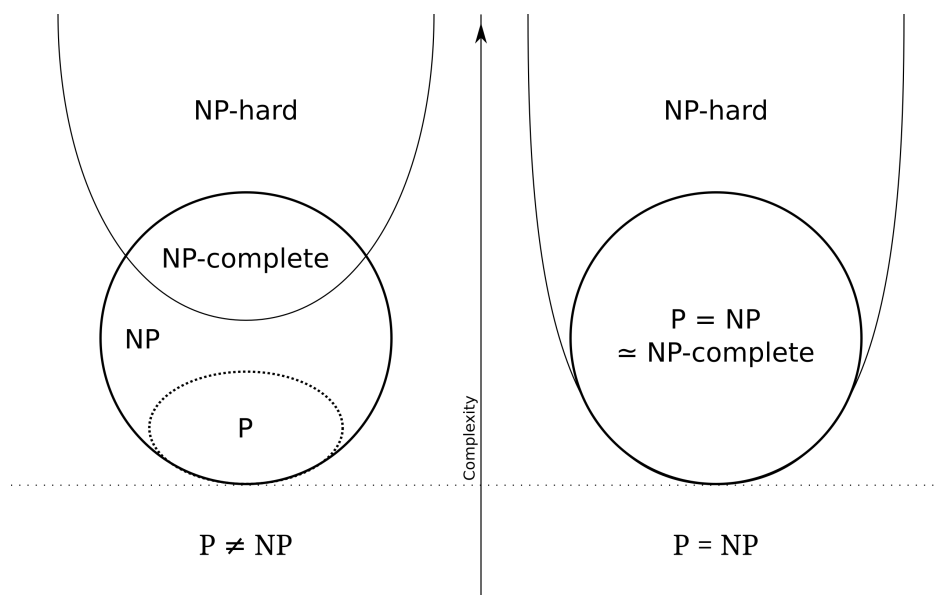
2 - Research around the Problem

A brief introduction into P vs NP

In simple terms it proposes that 'can every problem that can be checked quickly can also be solved quickly?'

The formal definition of P and NP refers to all decision problems (ones with a yes or no answer) that can be solved in polynomial time (i.e. $O(3n^3 + 2n^2 + 19)$) by a deterministic and non-deterministic Turing machine respectively. A Turing machine is an abstract mathematical model which was a precursor of a computer and is powerful enough to simulate any other computer or program. Just like deterministic and non-deterministic algorithms, a deterministic Turing machine is one that runs predictably, for every step based on the same input however a non-deterministic Turing machine is a theoretical concept that can explore all possible solutions at once and instantly pick the correct one. This helps us further explain what P and NP problems are; P are ones that we can solve efficiently using real-world algorithms such as searching and sorting whereas NP are those where we can check a solution efficiently such as the travelling salesman problem (TSP), graph colouring and the *knapsack problem*. If it was proven that $P = NP$, meaning all problems in NP would be in P, it would have vast implications in the field of mathematics, cryptography and algorithm research enforcing why it is one of the Millennium Prize Problems each of which carries a 1 million dollar prize for the first correct solution if P vs NP can be proven either way.

What is the difference between NP-hard and NP-complete problems?



Problems can be further grouped together in terms of NP-hard and NP-complete classes as shown in the diagram above. NP-complete is the set of all problems for which all other NP problems can be reduced to, and are therefore the hardest problems in NP. In 1971, the Cook-Levin theorem was introduced that showed that the 3SAT boolean satisfiability problem is NP-complete and that any other problem in NP can be reduced to this problem in polynomial time by a deterministic Turing machine and first introduced the idea of NP-completeness. An NP-Hard problem is at least as hard as an NP-complete problem, but does not have to be in NP meaning that they don't have to be decision

problems. Therefore it may also take a long time to verify a solution to one of these problems as well as finding a solution. Different variations of the knapsack problem are in different complexity classes. For example, the decision version of the 0/1 knapsack problem is NP-complete and the optimisation version of the problem is NP-Hard for the above reason.

What are the different types of knapsack problems?

There are many further more specific versions and generalisations of the 0/1 knapsack problem as it is one of the most popular problems in this area (List of knapsack problems, Wikipedia)

Variation	Description
Bounded Knapsack	For each item i , there is an upper bound u_i (which may be a positive integer or infinity) that determines the maximum number of that item can be selected
Unbounded Knapsack/Integer Knapsack	There is no upper bound on the number of times an item can be selected
Fractional Knapsack	A variation of the knapsack problem where it is possible to take fractions/parts of an item
Multiple Choice Knapsack Problem	The set of items is divided into different classes, and exactly one item must be chosen from each class of items
Subset Sum Problem	Another yet closely related NP-complete problem, defined as 'Given a set of integers S and a target sum T , is there a subset of S that exactly sums to T '
Multiple Knapsack Problem	If we have multiple knapsacks n with each having a weight limit W_n , how do we maximise the total value across all knapsacks
Quadratic Knapsack Problem	The objective function becomes quadratic as we now are given an additional $n \times n$ matrix P that represents the additional value of the knapsack added if we choose 2 items $[i, j]$ (P_{ij})
Multidimensional/Multi-constraint Knapsack Problem	There is another constraint such as total volume V that can't be exceeded and each item has a volume property v_i . Additionally, this constraint could be the dimensions of each item (x,y,z)

Further problem research

I wanted to further explore the Quadratic Knapsack Problem (QKP) and the Multidimensional Knapsack Problem (MKP), due to the form data collected below. This should add further depth to my project and complexity allowing me to adapt my existing algorithms for different purposes, allowing the user to test them under new conditions.

In order to implement modularity effectively, I have decided to use *object orientated programming* to classify algorithms related to the problems and adapt them easily.

Quadratic Knapsack Problem (QKP)

The quadratic knapsack problem makes the objective function quadratic as seen by the double sum above. It does this by introducing a nxn symmetrical matrix 'P' (for profit) which is added to the value when two items are included together. Each entry in the matrix (i,j) represents when the pair of i and j items are included together. The matrix will be an additional input in the program.

Formalisation:

$$\begin{aligned} \text{Maximize} \quad & \sum_{i=1}^n v_i x_i + \sum_{i=1}^n \sum_{j=i}^n p_{ij} x_i x_j \\ \text{subject to} \quad & \sum_{i=1}^n w_i x_i \leq W, \\ & x_i \in \{0, 1\}, \quad i = 1, \dots, n \end{aligned}$$

Multidimensional (Multiconstraint) Knapsack Problem (MKP)

Instead of having only weight and value of properties of an item, this problem adds another property to increase the difficulty of the problem. This is commonly 'Volume' which acts independent to weight and the knapsack subsequently has another Volume capacity. This specific form of the m-dimensional knapsack problem is shown below with 'U' representing volume. (The multidimensional knapsack problem is commonly confused with the multiple choice knapsack problem where there are multiple knapsacks with different capacities)

Formalisation:

$$\begin{aligned}
&\text{Maximize} && \sum_{i=1}^n v_i x_i \\
&\text{subject to} && \sum_{i=1}^n w_i x_i \leq W, \\
&&& \sum_{i=1}^n u_i x_i \leq U, \\
&&& x_i \in \{0, 1\}, \quad i = 1, \dots, n
\end{aligned}$$

For both variations of the problem, the brute force and exact solutions quickly become infeasible as the number of items increases. This will therefore be an interesting way to see how the meta-heuristic approaches handle with the added complexity of the problem.

Further research into knapsack cryptosystems

To gain a better understanding of how the knapsack problem was used in public-key cryptography and the implications for the wider-world of this project, I did some research around this topic area. It was created as one of the first systems and relied on the time-consuming nature of the knapsack problem, particularly in this case closer to the subset sum problem, as an exponential time algorithm with an input size of greater than 100 would be infeasible for a computer to brute force. (Knapsack Encryption Algorithm in Cryptography, 2025)

1. To start with, the private key is generated as a **super-increasing** sequence meaning each element is greater than the sum of the previous elements. This is a special case and can be solved by an algorithm in $O(n)$ time as we can simply loop through the sequence in reverse and if the sum is greater than or equal to the element, we include that element in our choice and decrease the sum by this amount
2. This private key is then transformed/mapped into a public key by applying a mathematical function to each element in the sequence. Commonly this is 2 integers (n, m) contained as an additional layer in the private key and $E_{\text{new element}} = n * E \pmod{m}$. The public key is published and used to encrypt messages, the private key is not shared and used to decrypt messages
3. For encryption, the plaintext is transformed into binary and then divided into sub-sections each of the length of the public key sequence. Each binary digit in a section is multiplied with the corresponding public key value and then summed together with the other digits in the section, to form the cipher text
4. When the device receives the transmitted cipher text, it is decrypted as we apply the multiplicative inverse of the $n \pmod{m}$ that can be calculated through (n, m) , and convert this back to binary using the technique stated in step 1, to calculate what elements of the private key sequence is needed to sum to this value. This can be then converted from binary to plaintext and be read by the user

Helpful and worked examples of this method can be found in greater detail here, (Knapsack Encryption Algorithm in Cryptography, 2025). However, eventually this system was proven susceptible to an attack in polynomial time, published in 1984 by Adi Shamir.

3 - Research into Algorithms that solve the Knapsack Problem

Here is the most popular and interesting algorithms used to solve this program however it is not an exhaustive list. The first 6 listed produce an exact solution, however the last 5 are different metaheuristic approaches resulting in an approximation, but should execute faster for large values of n .

Algorithm	Description	Time Complexity
Brute Force	Loop through every possible combination/subset of elements that can be chosen and store the set that sums to the greatest value while remaining under the weight limit	Exponential $O(2^n)$
Dynamic Programming	Due to the overlapping sub-problems of the knapsack problem and optimal structure, the most common approach to solving the knapsack problem is through dynamic programming, where either a memoization (top down) or tabulation (top-up) approach can be adopted to limit the amount of times a recursive function is called with the same parameters. I may explore Fully Polynomial-Time Approximation Scheme (FPTAS) which scales down the DP table to make an approximation within $(1 - \epsilon)$ of the optimal value	Pseudo-polynomial time $O(nW)$
Greedy Approach	Greedy algorithms are a class of algorithms that make locally optimal choices at each step with the hope of finding a global optimum solution. This algorithm works ideally for the fractional knapsack problem to produce an exact solution by sorting items by their value/weight ratio and then adding as much of each item in order	Quasilinear $O(n \log n)$
Backtracking	Uses a state space tree and a DFS approach yet optimises the brute force approach by stopping to explore a branch as soon as we reach a point that becomes infeasible	Exponential

Algorithm	Description	Time Complexity
Branch and Bound	Advances the backtracking algorithm to use bounds to prune branches that cannot produce a solution better than the best existing solution	Exponential
Meet in the Middle	Splits the problem into 2 smaller sub-problems and combines them in such a way to produce the final solution.	Exponential
Genetic Algorithms	An approximation algorithm that mimics the process of natural selection and genetics to effectively direct the initially random search to an optimal solution in a large solution space. There are multiple generations that pass on genes from the 'fittest' parents to the next generation.	-
Tabu Search	Similar structure to hill-climbing algorithms to explore solution space (starting with a valid solution and flipping the inclusion of items), but able to escape local maxima and find global maxima by accepting non-improving moves	-
Ant Colony Optimisation	A probabilistic technique for solving the knapsack problem used to find good paths through graphs simulating the behaviour of ants as they leave pheromone trails that accumulates over time for other ants to follow when moving from a nest to a food source	-
Simulated Annealing	Derived from the heating and cooling to remove defects and achieve a stable crystalline structure, it works by changing the 'temperature' of the algorithm to control the randomness at which solutions are adjusted, which decreases over time to find and refine an optimal solution	-
Particle Swarm Optimisation	Derived from flocks of birds or schools of fishes searching for food, it can provide a search based upon collaborative trial and error. Each particle has an associated fitness value and moves through the solution space with a specific velocity, keeping track of a particle best solution and the populations/global best solution. It may be used to tackle more complex versions of the knapsack problem such as the multidimensional knapsack problem	-

4 - User Identification and User Needs

In order to make my project suitable for both students and teachers, I wanted to explore the areas in which different types of users would interact with my project.

Teachers

As previously mentioned in the background information, Ms Lynn - a teacher at the Royal Grammar School Newcastle - is the primary user for this project. She will be most likely using this application in **class demonstrations** during lessons on algorithm design and the classifications of computational problems.

As some of the **algorithmic concepts extend beyond the A-level specification**, using the project as **a comparison to show the time complexity and accuracy of heuristic verses exact algorithms will be better appreciated and understood by the students** over the specific implementations and functionality.

Teachers should be able to preload data into the applications, perhaps from a csv file that can also be shared with students in a worksheet, to set specific tasks for them to complete. Furthermore, teachers should be able to collate specific information about the algorithms from descriptions displayed on the application to then develop and present to the students.

Students

A-Level students studying computer science at my school are as identified earlier, the prospective users in this project. They may use this application in **practical lessons** while learning related topics, and is why it should be easily accessible through a website or shared app.

They must be able to **adjust key parameters** of related algorithms in order to see the impacts on performance and therefore **revealing key insights into how each algorithm works**. Additionally, they may want to input their own data for custom test cases or download specific related information on the result of a test case for future learning.

The ultimate goal of this project is to inspire students to learn more about the boundary of computation and push them to explore this topic further. This is why I want to make the **user interface friendly and easy to use**, while using **data graphs holding important information** to further captivate the interest of the students.

I have bullet pointed the specific user needs after doing my primary user interview and analysing the prospective user's form

5 - Primary User

In order to make my product well suited for my audience, I decided to first conduct an interview with my primary user (Ms Lynn).

In your experience, have you ever come across the knapsack problem?

I have come across it, but haven't used it in teaching.

When students learn about complex algorithms, time and space complexity, linear programming and heuristics what do they find easy and hard?

Generally, identifying constant and linear time complexity is straightforward, but identifying when algorithms have higher complexity can be more challenging. Considering space complexity also tends to be more challenging as it is more abstract - we aren't used to considering memory usage, as modern computers generally have large amounts of available memory. With heuristics, this can seem quite abstract as well, since most problems that we will encounter at A Level are generally tractable, so there can be a difficulty with a lack of experience with problems which require heuristic solutions.

Are there any specific variations of the knapsack problem you would like to see?

As I haven't used it in teaching before, there isn't a specific version that I would want to see, but it would be nice to have multiple versions available to allow comparison, if possible.

What of the following algorithms do you think would be the most informative and interesting to students?

Brute force, dynamic programming, branch and bound, and ant colony optimisation would be my first choices.

Do you think my product should show how each algorithm works step by step, or a more holistic and comparative view between algorithms?

For this, I think a more holistic and comparative view would be fine, but as none of the algorithms are specifically taught, some explanation of how they work would be nice.

What visual aspects of the app do you think will be helpful to students to visualise the individual instructions and execution order of algorithms?

(Such as a dynamic programming table, decision tree, or genetic populations)

I think this would depend on the algorithm - I'd be happy to discuss this more with you once you have some designs to see which seem the most useful.

What is the best way to illustrate the performance of an algorithm?

(Such as graphs, fitness evaluation etc.)

Having a graphical representation is always useful, as this makes it easier to compare.

Have you seen any tools online that help students learn about the knapsack problem and this topic area in general?

I think I have at some point, but it was long enough ago that I wouldn't have specific links/examples now.

Should users be able to adjust algorithm parameters (cross over rate, number of ants) and change/generate input data?

Ideally I think this would be a good feature to be able to see how the size of the data set and other parameters impact algorithms with the various complexities.

Do you think visually learning algorithms through an interactive tool is more effective than reading through an academic paper?

Yes - although this is probably more the case for novice learners, I would argue the two used together would be more effective for even experienced students.

Is there any other suggestions or requests to be included within this product?

Not currently.

Summary of Primary User Needs

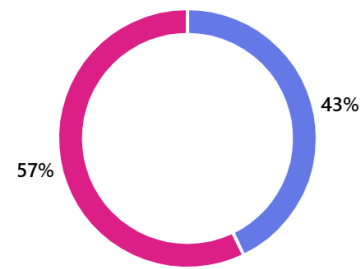
- I must include the following algorithms: Brute force, dynamic programming, branch and bound, and ant colony optimisation
 - Multiple different versions of the problem would be preferred but not necessary
 - Many graphical representations needed to make algorithms easier to compare
 - Be able to change individual parameters of each algorithm
 - Explanation of each algorithm as none are explicitly taught on the A-level/GCSE specification
-

6 - Prospective Users

In order to gain more data to inform my decisions when modelling my solution to this problem I created an online form and sent it out to prospective users such as other students that take A-level computing, further maths or go to 'comp-soc'. I gained a total of 14 responses and here is the analysis of my results:

1. The knapsack problem in its simplest form is stated as 'Given a set of items where each item has a weight and a value and a knapsack with a maximum capacity what subset of items maximises the total value of the knapsack without exceeding its capacity'. Have you ever seen this problem before? [More details](#)

● Yes 6
● No 8



This shows that the majority of my prospective users may not have heard of the knapsack problem before, therefore a simple explanation in the start menu might be needed.

2. What do you find difficult when learning about complex algorithms?

Responses

Memorising what the algorithm does and features about it such as the time complexity

Visualising the steps they take

Understanding how they are implemented in a computer through code

Teachers will often explain it in a way that makes sense to them, but may not make sense to the students

understanding it from a wider perspective. instead of what each step of the algorithm does, i struggle to understand how the steps fit together in order to complete the task.

Actually understanding the fundamentals of how it works

Thinking where to implement vaguer ones in everyday life - even though its aspect is incredible

Data structures, how can you define the data/variables in computer programming to represent the features and principles of the problem. And flowchart design to represent the process of solving a problem.

Where to start when making them

fully understanding its purpose

What even is a complex algorithm?????

Visualising instructions

Quantifying the time complexity

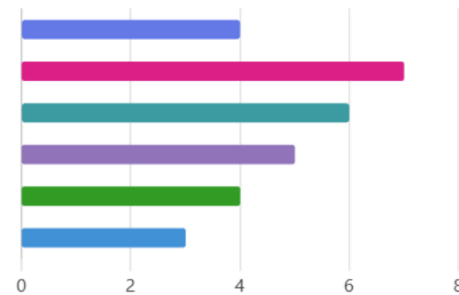
I sometimes find them hard to follow, especially when hand tracing them in a trace-table, or heuristic approaches where It can be hard to tell how accurate the solution is or the detailed steps it is following.

In this question, I got a wide variety of responses, showing how everyone learns and understands 'complex algorithms' in a different way. Some include, visualising the steps they take which could be implemented by a way of showing pseudocode running or having different graphs to track progress. Additionally, trace-tables were mentioned which could be used in the dynamic programming solution to store recursive calls.

3. Which exact algorithms are you most interested by and would like to see implemented and learn about?

[More details](#)

Brute Force	4
Dynamic Programming	7
Backtracking	6
Branch and Bound	5
Meet in the Middle	4
Other	3

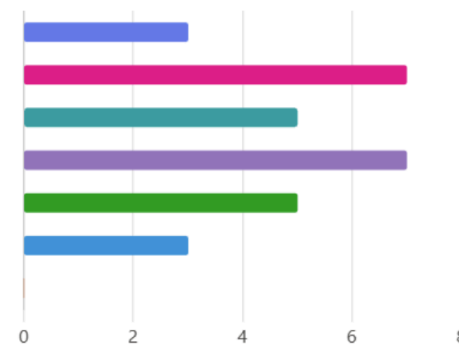


This graph shows the highest voted exact algorithms picked, with the most popular being the dynamic programming approach. I think it will be necessary to implement a brute force approach as well, just to show the huge number of combinations that need to be checked and how long this algorithm can take to execute with large values of items (n). Meet in the middle is a small extension of this used when the input size is small but not small enough to use direct brute force. Branch and Bound is a very useful technique as pruning branches can save a lot of time immediately, however the time complexity will still be seen as exponential.

4. Which heuristic algorithm approaches are you most interested by and would like to see implemented and learn about?

[More details](#)

Greedy Approximation	3
Genetic Algorithms	7
Tabu Search	5
Ant Colony Optimisation	7
Simulated Annealing	5
Particle Swarm Optimisation	3
Other	0



I want to implement as many meta-heuristic approaches as possible and compare them as I and many other people find them very interesting. Genetic algorithms may be further subdivided and have different parameters such as a population size, selection type and mutation type. An effective algorithm depends on how well these variables are tuned, and I want to give the user this flexibility. Another top choice was ant colony optimisation which is more commonly utilised when working with the travelling salesman problem (another very famous NP-hard problem), however it can be applied to the knapsack problem in a similar way. Simulated annealing and Tabu search both had 5 votes and both are also used frequently when approximating NP problems, and use very similar approaches to approximate solutions it would be interesting to compare them. I may also combine aspects of multiple algorithms when I feel more comfortable with the logic.

5. What visualisations would you like to see and find helpful to better understand how your chosen algorithm(s) works?

Responses

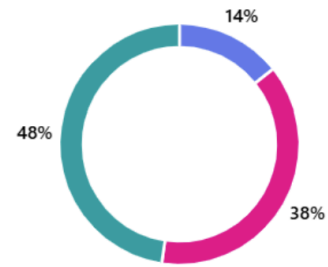
Graphs showing how effective the program is compared to other solutions. Trace tables showing the program working and allowing variables to be tracked.
Clearly showing the different generations and steps of each algorithm
The choices a computer makes in order/a tree of decisions
A play by play of the steps taken with an available explanation as to why that step was taken
a set of digital scales where the total weight of the items is shown. the items are represented by blocks which split or combine to find a better combination. the weight display should change with each iteration.
Flowcharts
Visually acceptable diagrams / animations with clear application of where algorithm is being used
Use animation to show how the solutions are improved iteratively
Not sure
graph
pictures
Decision tree
Decision tree
A graph of fitness/max global value against generation

A common response was a decision tree which could be implemented as part of backtracking and branch and bound algorithms, to show the different possible choices that could be made, i.e. the combinations of items that can be included. Another suggestion was different graphs to illustrate perhaps the maximum value of a valid solution achieved over time. Diagrams and animations could also be used to visually demonstrate what is happening for example a response suggested a set of digital scales to weigh the items, to make it as clear as possible what each algorithm is doing.

6. How would you like to generate input data for this problem?

[More details](#)

● Input Data Manually	3
● Import from CSV file (Excel)	8
● Randomly with parameters such as size of data set and spread of values/weights	10

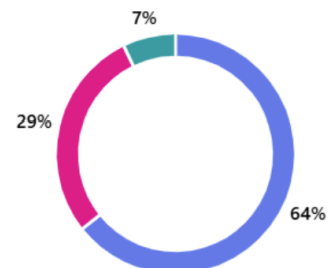


The most common choice here was to use a randomly generated set of data values, and have input parameters such as number of items, spread (variance) of values and weights, and even bounds for the bounded knapsack problem. Another way if the user was using my application in a more real world scenario, it may be helpful for data to be imported from a csv file which is manipulated as spreadsheets and tables in Excel, however I would have to outline the format of the data in a somewhat rigid way.

7. What level of control would you like over the algorithms parameters?

[More details](#)

● Full control	9
● Limited control (multiple guided options)	4
● No control	1

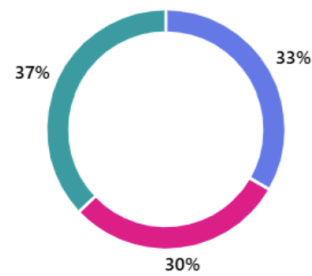


This data suggests that user wants full control over the parameters of the algorithms, allowing them to learn and observe the impacts of changing such variables. However, a brief description may be needed perhaps when you hover over the name of the variable to help less experienced programmers or algorithm specialists to appreciate what each algorithm does.

8. What would you like to see measured between algorithms?

[More details](#)

● Execution time	9
● Memory usage	8
● Accuracy	10

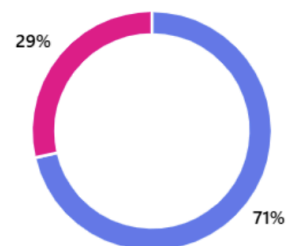


Due to the evenness of the results, I will try and measure as much general and specific data on each algorithm. Time complexity can be measured either through the theoretical analysis of the algorithm but also by timing the execution runtime using 'time' libraries in python. For space complexity I could measure stack, heap and global/static memory usage. Another interesting statistic is looking at how many times a candidate solution is evaluated.

9. Would you like to save and export data/graphs/images?

[More details](#)

● Yes	10
● No	4

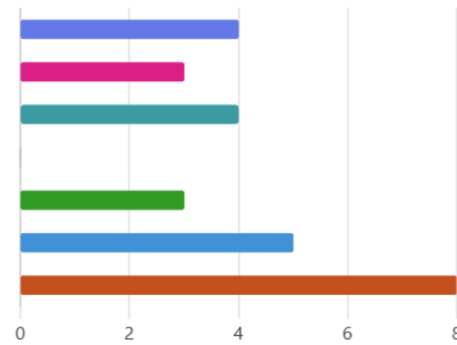


This may be useful if the user want to analyse results and save data for later, such as graphs produce in genetic algorithms to determine how well the model is growing. This is another feature which increases the ease of use and may allow for my application to be referenced included in academic papers if needed.

10. Are there any specific additional variations of the (0/1) knapsack problem you would like to see?

[More details](#)

Bounded Knapsack	4
Unbounded Knapsack	3
Multiple Choice Knapsack	4
Subset Sum	0
Multiple Knapsack	3
Quadratic Knapsack	5
Multidimensional Knapsack	8

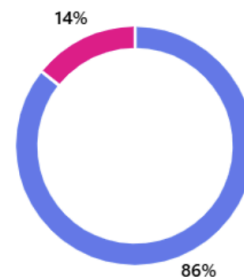


I will start with definitely include the 0/1 knapsack problem and then maybe branch out to more complex variants that my prospective users want to see. However each algorithm themselves might have to be modified in order to work correctly for these variations, meaning I will have to make my programs flexible, but adds significant depth when comparing algorithms.

11. Would you like a graphic representation of physical models of the knapsack and its items?

[More details](#)

Yes	12
No	2



This may be used as another visualisation when perhaps creating items, and linking the formal logic to real world applications. Perhaps the knapsack could be a shipping crate, and items could be several boxes with different colours, to make it more clear what the problem is and how the algorithm is working.

12. Is there any additional suggestions you have?

A brief explanation of each algorithm in the selection menu

Comparing the heuristic algorithms output to the true solution.

Include a detailed explanation on how each algorithm works and its advantages and disadvantages.

Summary of Prospective Users Needs

- Description of problem(s)
 - Graphs/Tables should track the progress of the algorithm over time
 - A wide variety and selection of both exact and heuristic algorithms should be implemented and available to the user
 - A decision tree should be included for one of the related algorithms
 - The user should have a range of ways to input data and specifically able to quickly generate bounded random data
 - Full control over algorithm parameters
 - Data should be able to be easily exported/copied from website
-

7 - Analysis of Current Systems

There is no current system online who has targeted this problem in as much detail as I plan to do so due to the uniqueness of this project (There are however many academic/research papers in this area). However, I found a couple simulations that showed and explained the dynamic programming approach with a table and simple animation systems to demonstrate how this algorithm work. For example, the following website:

1 - Knapsack Algorithm Visualisation

(<https://monicagranbois.com/knapsack-algorithm-visualization/>)



Knapsack Algorithm Visualization

Step 2: Build Table

Press the "Step" button to fill the table.

	0	1	2	3	4	5
	0	0	0	0	0	0
item 1 (v: 4 w: 2)	0	0	4	4	4	4
item 2 (v: 3 w: 1)	0	0	0	0	0	0
item 3 (v: 5 w: 3)	0	0	0	0	0	0

Reset

Step

```
1 // w: item weight (2)
2 // value: item value (4)
3 // c: current capacity (5)
4 // i: index (1)
5 if (w <= c) { // 2 <= 5
6     T[i][c] = Math.max(T[i-1][c], (value + T[i-1][c-w]))
7     /*
8     T[1][5] = Math.max(T[0][5], (4 + T[0][3]))
9     T[1][5] = Math.max(0, 4)
10    T[1][5] = 4
11    */
12 }else {
13     T[i][c] = T[i-1][c]
14 }
```

This phase builds the dynamic table. In this visualization the process is done in a step by step process. Pressing the "Step" button will calculate the next cell's value. That value will be the maximum value for the current knapsack capacity. The highlighted code is the logic used to calculate the value.

This website includes a brief explanation of what the 0/1 knapsack problem is and then a small simulation where you can hold up to 10 items and a maximum weight limit of 10. These restrictions are used to make sure that the table is small enough to fit on the webpage. Each item can be customised as data is inputted manually including the name of the item, value and weight. The setup is overall simple but intuitive. I will explain the concept of dynamic programming in section 2, however the basic idea is that the table index row i , column j represents the maximum value that can be obtained with items 1, 2, 3 ... i , in a knapsack that can hold j weight units. You can step through the algorithm and arrows are used to illustrate the processes. Once the algorithm has finished execution it displays the set of items to be selected. It also links to resources that further explain the algorithm.

Summary Table

Advantages	Disadvantages
Easy to use - clear layout, making it easy to input data and step through the algorithm	Data is constraint to smaller values
Includes a small explanation on how the algorithm works and links to other resources	Only includes one algorithm implemented
Includes abstracted and dynamic pseudocode where parts of it are highlighted as you step through the program	Includes full algorithm in javascript as commented code

Another more advanced system explores genetic algorithms and looks at the knapsack problem as well as the travelling salesman problem, image generation and the N Queens puzzle.

2 - Genetic Algorithm Visualization Tool

(<https://www.geneticalgorithms.online/knapsack>)

The screenshot shows the 'Knapsack problem' interface of the Genetic Algorithm Visualization Tool. The interface is divided into several sections:

- Header:** A red bar at the top with a menu icon, the text 'Knapsack problem', and a settings icon.
- Left Sidebar:** Contains navigation links: 'Home', 'What are genetic algorithms?', 'Travelling salesman problem', 'Image approximation', 'Knapsack problem' (highlighted), and 'N Queens puzzle'.
- Top Controls:** Includes a 'Number of items' slider set to 15, a 'Current generation no: 0' display, and a 'Maximum knapsack capacity is 20kg' label.
- Main Area:**
 - Inside the knapsack:** A list of 10 items, each with an icon, a value (indicated by a dollar sign), and a weight (indicated by a scale icon). The items are: (1, \$2, 3), (2, \$1, 4), (3, \$5, 5), (4, \$4, 5), (5, \$3, 1), (6, \$3, 1), (7, \$1, 3), (8, \$2, 5), (9, \$5, 4), and (10, \$5, 4).
 - Fitness chart:** A line graph showing 'Fitness' on the y-axis (ranging from -1 to 4) and 'Generation' on the x-axis (ranging from 0 to 4). The chart is currently empty.
 - What are we solving?:** A text box explaining the knapsack problem: 'Imagine that you want to go on a hiking trip. You need to take dozens of items with you, but your backpack can only fit so many. What items should you pack in the backpack for the trip and which ones should you rather leave at home? If you know the weights (or size) of the items and assign how important they are to you (specify their value), the genetic algorithm can help you with the dilemma! Without the genetic algorithm, to find the best possible solution, we would have to'.
- Parameters settings:** A sidebar on the right with various controls:
 - Generations limit:** A slider set to 100.
 - Selection type:** A dropdown menu set to 'Elite selection'.
 - Crossover type:** A dropdown menu set to 'Single-point crossover'.
 - Mutation type:** A dropdown menu set to 'Flip-bit'.
 - Mutation probability (%):** A slider set to 25.00.

Just like the previous system, it explains the 0/1 knapsack problem through an analogy to introduce the user to the problem. To input data, the user can control the number of items (up to 20) which are randomly generated and shown in a list on the left with an icon, value and weight. Labelled 'inside the knapsack' shows the current choice/subset of items selected, whereas 'outside the knapsack' shows the items that are not selected. The knapsack capacity is fixed at 20kg. There is a current generation number as well as a graph that updates in real-time when the visualisation is started. This graph system also allows the user to download plots as a png and has several zoom tools. The main

functionality of this website is in the parameters setting menu, where the user can choose the generation limit, selection type, crossover type, mutation type, and mutation probability with drop-down menus and sliders and see how these changes reflect on the performance of the algorithm. Additionally each parameter has a short explanation on what it does as you hover over the option.

Summary Table

Advantages	Disadvantages
Parameter flexibility allows the user to have almost full control over the operation of the algorithm	Must reset simulation before adjusting parameters
Fitness chart effectively shows progress over time	Only includes one algorithm implemented
Easy and quick generation of input data with simple visual appeal	No manual control over input data

8 - Objectives

Through dialogue with users of the system and the provided analysis above (specifically the primary user and prospective user requirements), I have created the following objectives to outline the specific targets of my product stemming from the initial design brief and problem description. I will also reference how I arrived at each objective in the table below.

Number	Objective Name	Explanation	Reference
1	Exact Algorithms	I want to implement the following exact algorithms: 1.1 - Brute Force - to act as a baseline 1.2 - Dynamic Programming - the most common exact algorithm approach 1.3 - Branch and Bound - should reduce the search space significantly, allowing for larger values of n	Primary user specifically detailed these algorithms
2	Heuristic Algorithms	I want to implement the following heuristic algorithms: 2.1 - Randomised Search 2.2 - Greedy Approximation 2.3 - Genetic Algorithm	I wanted to include as many algorithms as possible as supported by the

Number	Objective Name	Explanation	Reference
		• 2.4 - Ant Colony Optimisation • 2.5 - Simulated Annealing • 2.6 - Tabu Search • 2.7 - Particle Swarm Optimisation	prospective users form and requirements
3	Exception handling	I want to ensure that any input errors are caught properly and displayed to the user: • 3.1 (Manual Input Error)- Errors for manually inputting data • 3.2 (CSV Input Error) - Errors for importing csv in incorrect format	Primary user mentioned audience may have limited experience with inputting data to this problem
4	Ease of use	The application should have a simple explanation embedded within the interface to explain the 4.1 (Problem Description) - the knapsack problem and 4.2 (Algorithm Description) - each algorithm	Primary user mentioned audience may have limited experience with this specific problem and related algorithms
5	Control over parameters	For each algorithm, the user should be able to control variables, for example the initial temperature in simulated annealing. 5.1 (Slider Visual) - The user should see the change in the design interface, and 5.2 (Algorithm Parameter) - the user should see the effect of the change in the displayed output	Form suggested that prospective users want 'full control' over the parameters of the algorithm
6	Inputting Data	I want to give the user the option of either 6.1 (Generate Random) - randomly generating data within constraints so that the product can be easily used by anyone. 6.2 (Input Manually) - Inputting the data manually into the program. 6.3 (Import CSV) - Importing data from a CSV file for more customisability.	In the form, people preferred different methods of inputting data so I wanted to offer a selection that would work in many circumstances

Number	Objective Name	Explanation	Reference
7	Design Interface	7.1 - I want to have a dynamic html interface that scales actively for different screen sizes and is both aesthetically pleasing and functional	My product may be used by different devices, i.e. from a laptop to a phone
8	Data Graphs	For certain algorithms such as brute force recursion, I want to include 8.1 (Decision Tree) - a rooted binary tree to show the processes that are going on, and in 8.2 (Plotted Graphs) - other algorithms like genetic algorithms I want to show some kind of fitness/generation graph. 8.3 (DP Tabulation Table)- For DP tabulation, I want to include a DP table as this is covered by most other knapsack solvers online. I need to do more research into each algorithm before I can state all relevant data I will include in the end-product	Primary user stated the benefits of graphical visualisation
9	Comparative Data	9.1 - In order to make comparisons between algorithms, I want the program to output key information including the subset of items chosen, their total value and execution time	The comparative aspect of this project is very important and is emphasised both in the form and by the primary user

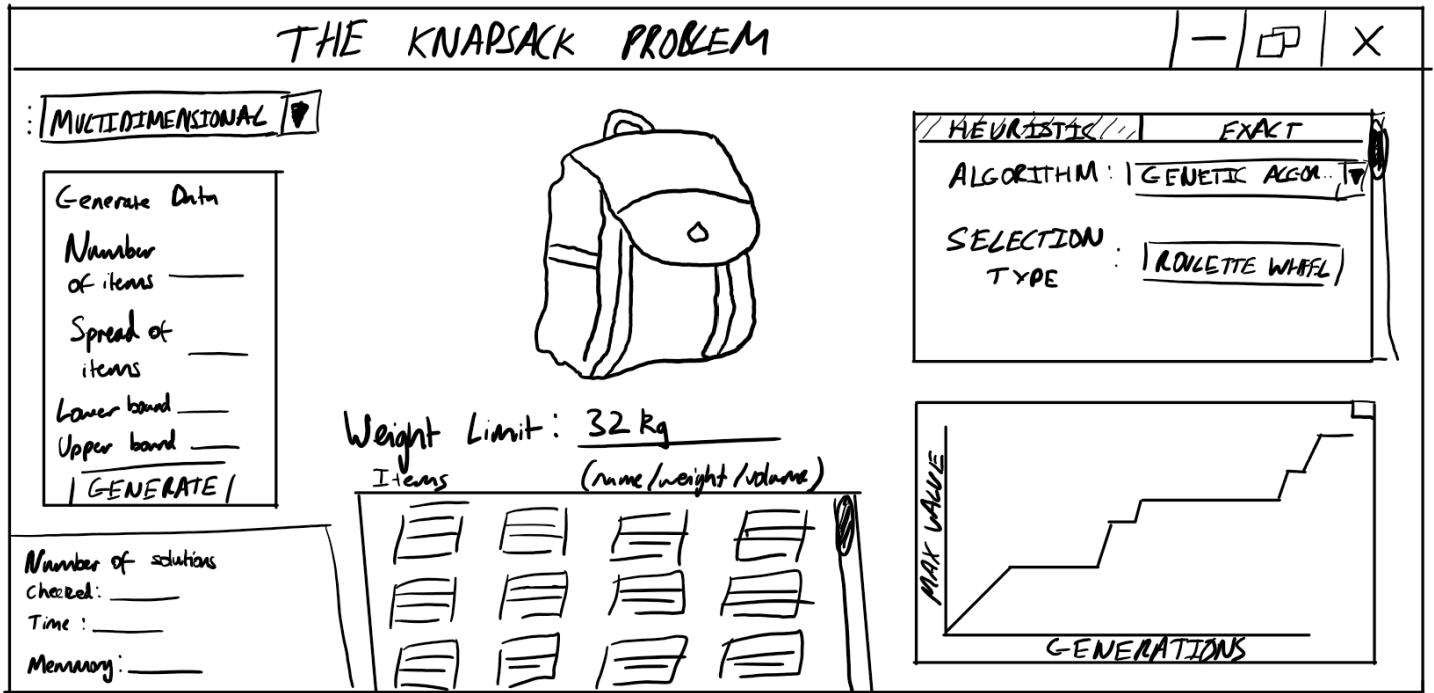
Acceptable Limitations

- I have decided to specifically orientate my product around the 0/1 knapsack optimisation problem due to the timescales of this project, and the number of individual algorithms I want to implement. Additionally, this version of the knapsack problem is the most well known and I think has the most direct applications.
- I have decided to create the visualisation elements around the performance and functionality of each algorithm rather than the problem itself. I think this will allow the user to run algorithms on much larger sets of data, and be more suitable to my audience.
- I have only selected a total of 12 algorithms to develop, however this is not an exhaustive list. It would be simply impossible to implement all algorithms for this problem as new hybrid algorithms

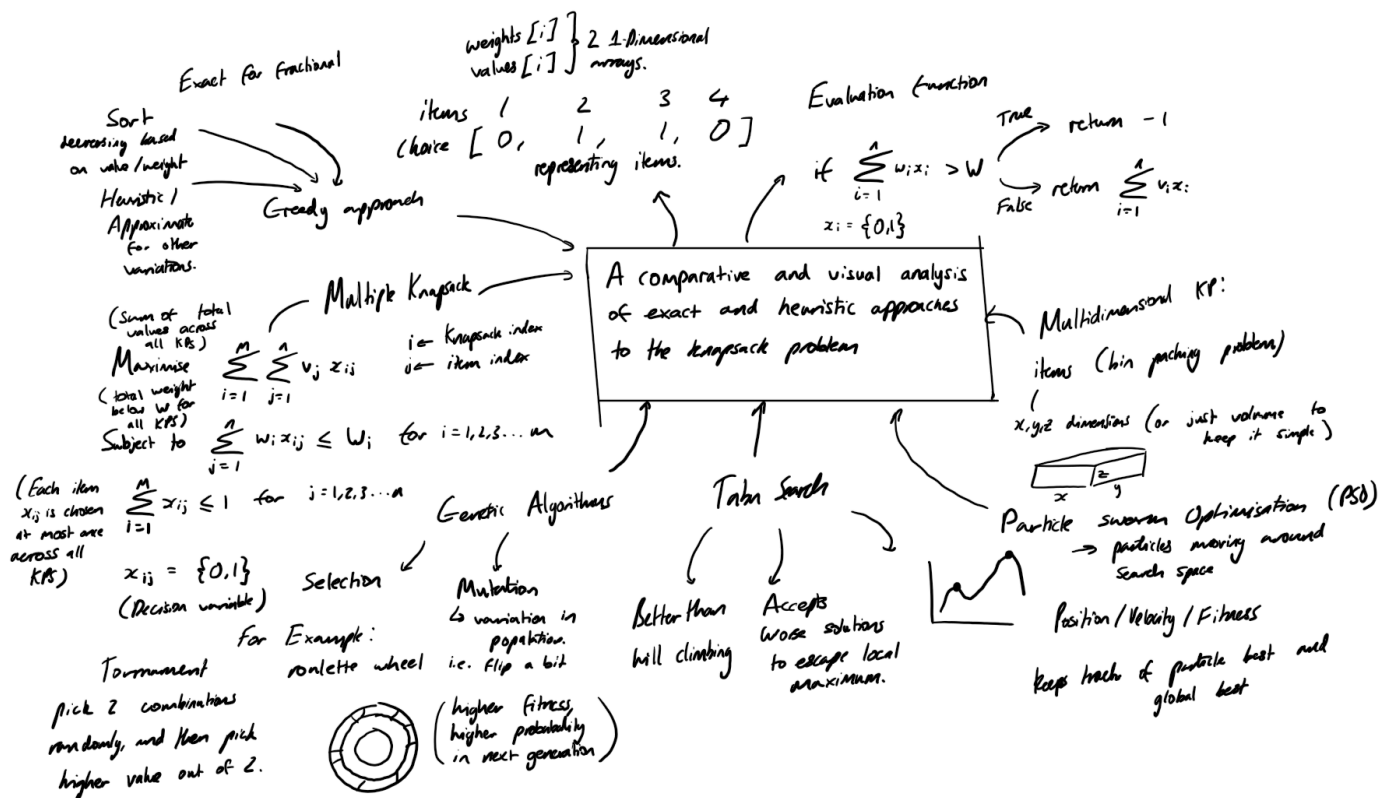
are constantly being developed further and under the time constraints of this project. I have chosen the ones that I think have the most potential to develop further.

9 - Modelling

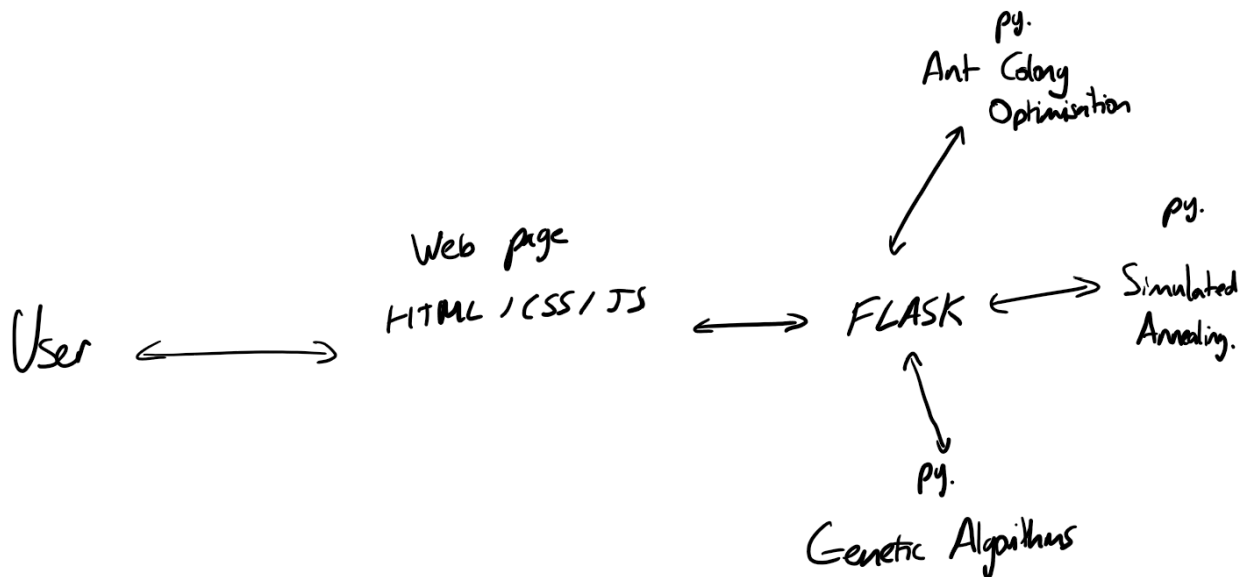
Here, I will show an abstracted view of what my product may look like, in order to plan forward into my design stages. This is a simple blocked-out layout of what the interface could look like:



Here is a mindmap of some of the elements I might include in the product.



To build my product I wanted to use python to create the algorithms (backend) and perhaps flask as a framework to connect it to the visual elements of the website as shown in the initial flow diagram below:



I chose flask as it is flexible, light weight, offers simple front end and I can connect with several python libraries like numpy and matplotlib. However, may choose to make develop this idea into an app after researching into the GUI technologies available.

10 - Bibliography

- 'Backpack Problem'. [Brilliant.org](https://brilliant.org/wiki/backpack-problem/). From <https://brilliant.org/wiki/backpack-problem/>
- 'Knapsack Encryption Algorithm in Cryptography'. [geeksforgeeks](https://www.geeksforgeeks.org/computer-networks/knapsack-encryption-algorithm-in-cryptography). From <https://www.geeksforgeeks.org/computer-networks/knapsack-encryption-algorithm-in-cryptography>
- 'List of knapsack problems'. [Wikipedia.org](https://en.wikipedia.org/wiki/List_of_knapsack_problems). From https://en.wikipedia.org/wiki/List_of_knapsack_problems

A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the Knapsack Problem

Documented Design

Name: Aarav Bhatnagar - Centre Number: 39253 - Candidate Number: 8015



Table of Contents

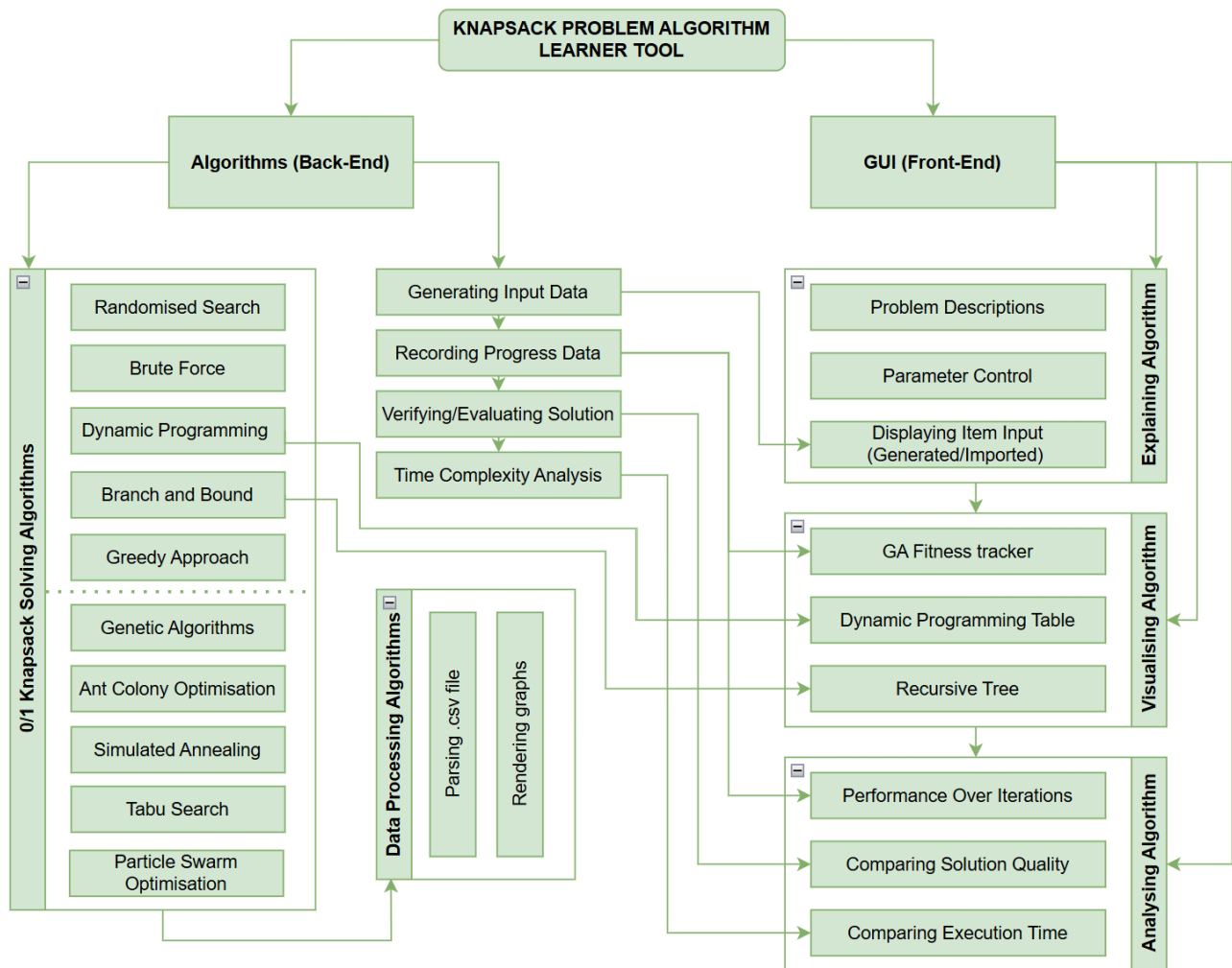
- [1 - Introduction](#) (pg 29)
- [2 - Higher Level Flowchart](#) (pg 30)
- [3 - Algorithms \(Top Down Design\)](#) (pg 30)
 - [General Structure](#) (pg 30)
 - [UML Object Diagram](#) (pg 31)
 - [Evaluation/Objective Function](#) (pg 32)
 - [Brute Force](#) (pg 32)
 - [Iterative](#) (pg 33)
 - [Recursive](#) (pg 33)
 - [Dynamic Programming](#) (pg 33)
 - [Memoization \(Top down\)](#) (pg 34)
 - [Tabulation \(Bottom up\)](#) (pg 35)
 - [Greedy Approximation](#) (pg 36)
 - [Branch and Bound](#) (pg 36)

- [Genetic Algorithms \(GA\)](#) (pg 39)
- [Ant Colony Optimisation \(ACO\)](#) (pg 42)
- [Tabu Search](#) (pg 44)
- [Simulated Annealing](#) (pg 46)
- [Particle Swarm Optimisation \(PSO\)](#) (pg 48)
- [Specialised Data Structures](#) (pg 51)
- [Input Process Storage Output Chart - IPSO Chart](#) (pg 53)
- [4 - GUI \(Top Down Design\)](#) (pg 54)
 - [Generating input data](#) (pg 54)
 - [Approach 1](#) (pg 55)
 - [Approach 2](#) (pg 55)
 - [Working with Flask](#) (pg 55)
 - [Rendering Graphs](#) (pg 56)
 - [Design Interface](#) (pg 56)
- [5 - Conclusion](#) (pg 58)
- [6 - Bibliography](#) (pg 58)

1 - Introduction

In this section, I aim to further research the algorithms that I am using, and further delve into the logic behind them. This will allow me to apply them to the knapsack problem. Additionally, I want to further plan the GUI and ways in which I can help the user visualise the algorithms.

2 - Higher Level Flowchart



This flowchart describes a potential overview of the individual components of my project. For example the subroutines and features grouped by sections. It shows a good balance between the algorithms, and ways of visualising these processes. It also shows the relationship between both sides of the project.

3 - Algorithms (Top Down Design)

The 2 fundamental parts of my NEA are the algorithms and the GUI. The GUI (front end) is what the user interacts with however the algorithms (back end) is the logic that solves the problem.

General Structure

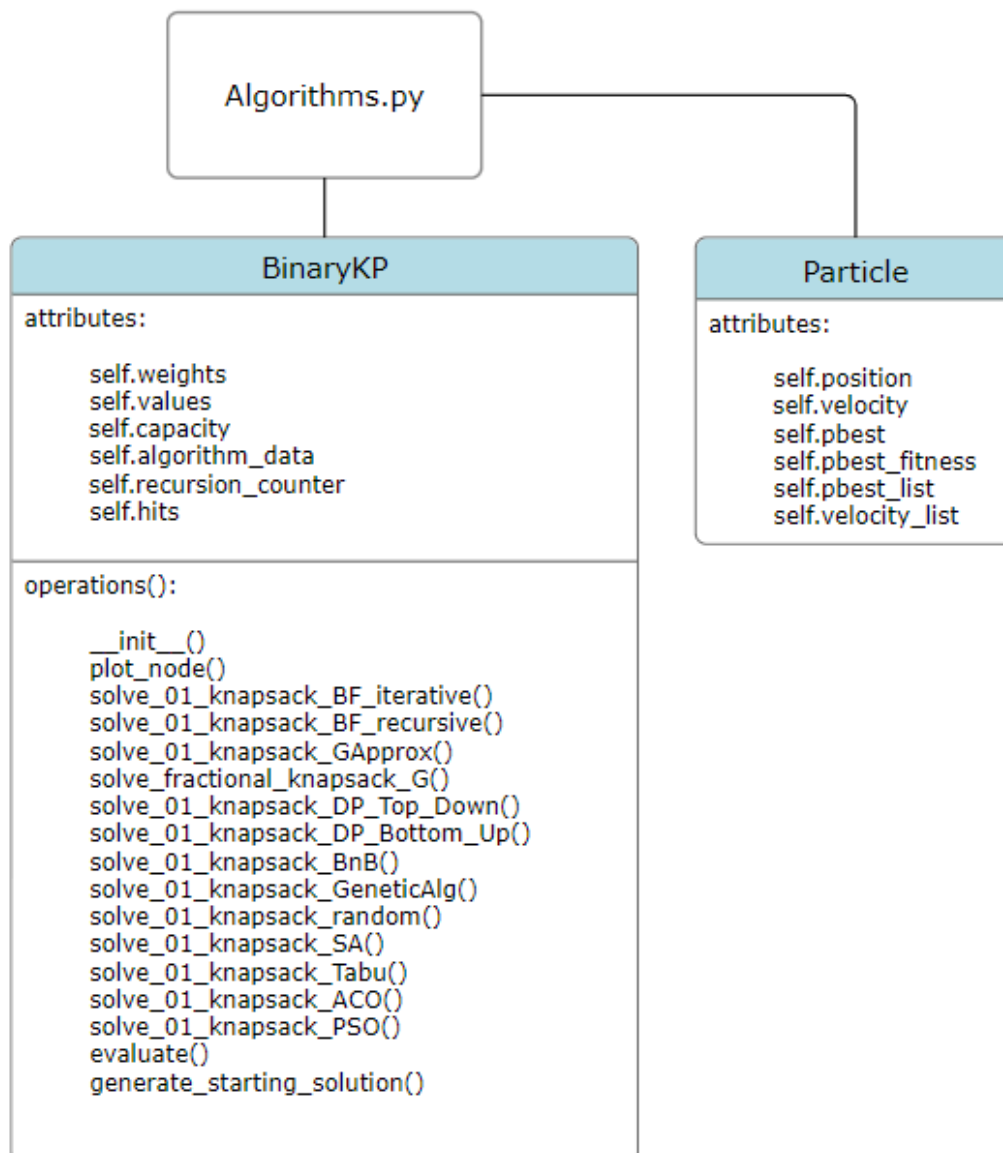
Each algorithm should have a standardised input/output set to ensure consistency and encapsulate the problem:

- The 0-1 knapsack problem should have the following parameters (weights, values, capacity)
- Also it should return (best_combination, max_value)

- I have chosen to write the algorithms in python as this is the language I am most comfortable with and is often used with algorithm design
- The solution to the 0-1 knapsack problem may be represented as a binary string where the ith character in the string represents whether (1) or not (0) the item is included in the subset of the solution. This therefore represents the optimisation version of the 0-1 knapsack problem (not the decision version and is therefore NP hard).

UML Object Diagram

Below is my class diagram for my project. It is often used during object orientated programming in order to display how different classes, methods and attributes link together. BinaryKP has all the named methods included in my technical solution of which most of them are either an exact or heuristic method for solving the problem. Particle is used in PSO as a data structure/object in order to collect related data terms together of a single particle. These classes, attributes and methods will be explained much further in the technical solution sections.



Evaluation/Objective Function

The evaluation/maximisation function remains constant for the 0-1 knapsack problem so the following pseudocode is shown below. I will commonly use this function in the algorithms pseudocode below. This stems from the formulised definition of the knapsack problem stated in the analysis section. Here is a visual explanation with the pseudocode.

Given n items indexed 0 to $n-1$
A binary string, i.e. $\overset{\text{index } 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ \dots \ n-2 \ n-1}{01001110101\dots 010}$
can represent a candidate solution through this item code.
 $\therefore$ $\text{binary_string}[i] = 0 \Rightarrow \text{item } i \text{ included in knapsack}$
 $\quad \quad \quad 1 \Rightarrow \text{item } i \text{ excluded in knapsack}$

Evaluating a binary string:

binary string $\overline{01101\dots}$
 $\quad \quad \quad \uparrow$
 $\quad \quad \quad i$

$\text{total_value} += \text{values}[i] \text{ if } \text{binary_string}[i] == '1'$
 $\text{total_weight} += \text{weights}[i] \text{ if } \text{binary_string}[i] == '1'$

RETURN total value if total weight $\leq$ capacity else 0

```
def Evaluate(binary_string):  
    if sum([WEIGHTS[x] for x in range(len(WEIGHTS)) if binary_string[x] == "1"]) >  
    CAPACITY: return 0  
    return sum([VALUES[x] for x in range(len(VALUES)) if binary_string[x] == "1"])
```

Brute Force

- There are 2 different brute force approaches - iterative and recursive
- The idea of the brute force method remains the same - to simply iterate over all possible permutations of knapsack solutions, and store the highest valid solution, i.e. the sum of the values is highest and the sum of the weights is under the capacity. There are a total of 2^n permutations as for n items it can either be included (1) or excluded (0) in the final solution - 2 choices for n items. Therefore a time complexity of $O(2^n)$ can be reached if the time needed to check a solution is $O(1)$

- The iterative approach usually simply counts up from 0 to 2^n in binary. The recursive approach divides the problem into smaller sub-problems (including and excluding the next item)

Iterative

```
maximum_value = 0
best_solution = ''
for x in range(2^n):
    solution_value = Evaluate(bin(x))
    if solution_value > maximum_value:
        maximum_value = solution_value
        best_solution = str(bin(x))
```

This specific code has time complexity $O(n2^n)$ because the evaluate function has linear time complexity. However this can be reduced to $O(2^n)$, when counting in binary using gray codes which I will explore when implementing.

Recursive

(Builds solution from item by item recursively, branching out to all subsets. n represents the number of items that have already been chosen/selected. The pseudocode below works from the first item to the last as I think this is more intuitive, however can also be called in reverse)

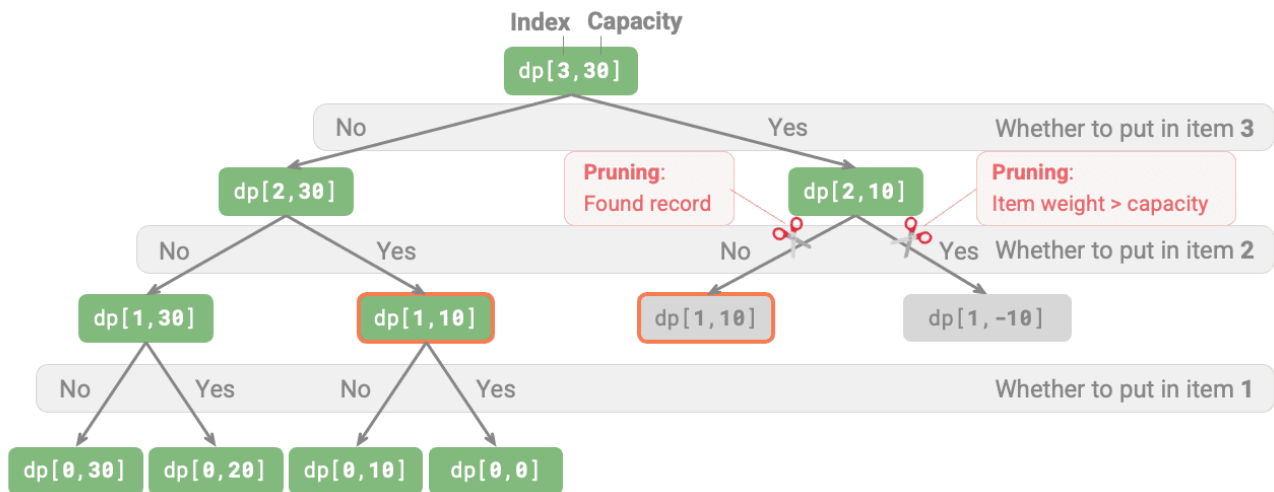
$$k(W, n) = \begin{cases} 0, & \text{if } W \leq 0 \text{ or } n \geq N, \\ k(W, n+1), & \text{if } w_n > W, \\ \max \left(\underbrace{v_n + k(W - w_n, n+1)}_{\text{Include item}}, \underbrace{k(W, n+1)}_{\text{Exclude item}} \right), & \text{otherwise.} \end{cases}$$

```
def solve01KP(weights, values, capacity, n=0):
    if n >= len(weights) or capacity <= 0:
        return 0 //base-case
    elif weights[n] > capacity:
        return solve01KP(weights, values, capacity, n+1)
    else:
        including_item = solve01KP(weights, values, capacity - weights[n], n+1)
        including_item += values[n]
        excluding_item = solve01KP(weights, values, capacity, n+1)
        return max(including_item, excluding_item)
```

Dynammmic Programming

(This is the most popular approach and is what is commonly taught when introducing this problem)

- This approach stems from the recursive method used above. It builds on smaller overlapping subproblems and storing their solutions to avoid redundant computations
- There are 2 ways to implement this. Memoization stores each solution in a cache, commonly using a dictionary, thereby we can check this cache before our recursive call. Tabulation starts by solving the base case and storing the result in a table. It then will build up the table from smaller subproblems until it has solved the required problem



www.hello-algo.com

```
def solve01KP(weights, values, capacity, n=0, memo={}):
    if n == len(weights) or capacity == 0:
        return 0 //base-case
    elif (capacity, n) in memo:
        return memo[(capacity, n)]
    elif weights[n] > capacity:
        result = solve01KP(weights, values, capacity, n+1)
    else:
        including_item = solve01KP(weights, values, capacity - weights[n], n+1)
        including_item += values[n]
        excluding_item = solve01KP(weights, values, capacity, n+1)
        result = max(including_item, excluding_item)

    memo[(capacity, n)] = result
    return result
```

Tabulation (Bottom up)

							
Weight wgt	Value val	i \ c	0	1	2	3	4
wgt[i-1]	val[i-1]	0	0	0	0	0	0
1	5	1	0	5	5	5	5
2	11	2	0	5	11	16	16
3	15	3	0	5	11	16	20



State transition:

$$dp[i, c] = \max(dp[i - 1, c], dp[i - 1, c - wgt[i-1]] + val[i-1])$$

Step 13

www.hello-algo.com

A bit more complicated than memoization but should produce even further optimised results. The following video explained the structure of the algorithm '0/1 Knapsack problem | Dynamic Programming'

Here is the standard formulation followed by the pseudocode:

$dp[i][c]$ = The maximum value using the first i items with capacity c

$$dp[0][c] = 0, \quad \forall 0 \leq c \leq W$$

$$dp[i][0] = 0, \quad \forall 0 \leq i \leq n$$

$$dp[i][c] = \begin{cases} dp[i-1][c], & \text{if } weight[i] > c \\ \max(dp[i-1][c], value[i] + dp[i-1][c - weight[i]]), & \text{if } weight[i] \leq c \end{cases}$$

```
n = length(WEIGHTS)
// Create a 2D array to store maximum values
dp = array of size (n+1) x (CAPACITY+1) initialized to 0
// Build the dp array
for i in range(1, n):
    for w in range(CAPACITY):
        if WEIGHTS[i-1] <= w: // If the weight of the current item is less than or equal
            to w, then we can choose this item
            dp[i][w] = max(dp[i-1][w], VALUES[i-1] + dp[i-1][w - WEIGHTS[i-1]])
        else:
            dp[i][w] = dp[i-1][w] // Item cannot be included

result = dp[n][CAPACITY] // Maximum value that can be achieved
```

Greedy Approximation

This is perhaps the simplest way to 'solve' the knapsack problem, and as shown and proved, will always find the exact value for the fractional knapsack problem due to its nature. However only gives an approximate result for the 0-1 knapsack problem.

The principle of the algorithm is sorting the list of items so that highest value to weight ratio/density is at the start of the list, and then selecting the first 'k' items from this list. For the fractional knapsack problem, the algorithm slightly differs in the fact we will just take a partial amount of the last item governed by the remaining weight capacity.

Pseudocode (The sorting function assumes the language has a 'Sort()' function which may be merge sort or quicksort so is ignored here)

```
Items = [(weight, value, value/weight)...] //Array of items with their weight, and
value/weight density (the property that sorts the list)
Sort_Descending(Items) //This will return a list with the highest value/weight ratio
first

Answer = []
Current_Capacity = 0

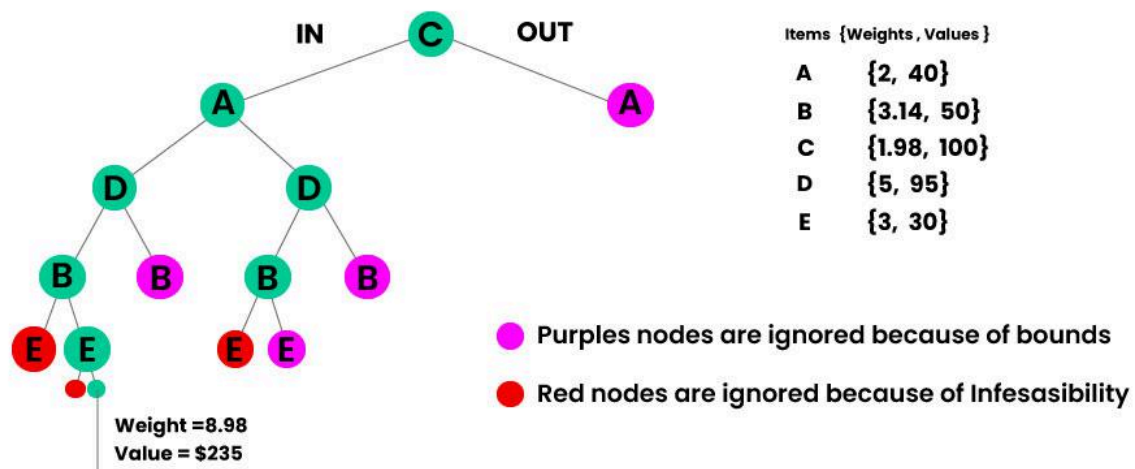
//Select items
for item in Items:
    if item[weight] + Current_Capacity > CAPACITY:
        return Answer
    else:
        Items.append(item)
        Current_Capacity += item[weight]
```

The main 'flaw' of this algorithm is that it excludes the possibility that removing items in the short term may result to a higher total output due to the discrete nature of the problem. An example is items (value, weight) = [(5, 4),(3, 3),(3, 3)] capacity = 6. The greedy approach would choose only the first item as it has the highest value/weight density. But by choosing the latter 2 elements to can utilise more of the capacity and achieve a higher total value leading to a better solution.

Branch and Bound

This is another efficient exact solution method that will be interesting when compared to the dynamic programming approach. It also works based upon the recursive approach but prunes branches based upon lower and upper bounds to reduce the search space, avoiding searching under existing 'bad' choices. Branch and bound also utilises the priority queue/heap specialised data structure in order to optimise calls. If no bounds are available the algorithm degenerates into an exhaustive search.

The search follows a rooted tree pattern that may be linked to the visualisation aspect of this project. Each 'row' i of the tree describes whether the item i is included or excluded in the knapsack, as can be seen in the diagram taken '0-1 Knapsack Problem using Branch and Bound'.



Knapsack using Branch and Bound



An upper bound sets a limit on how high a value can be. A lower bound is the least one can hope for. For example, the shortest distance between 2 points on a map is never shorter (lower bounded by) the straight linear distance (as the crow flies) between the 2 points. In a similar way, we can apply this to the 0-1 knapsack problem using the greedy approach described above. We can use the greedy approximate solution as a upper bound, and the greedy exact solution for the fractional knapsack problem as a lowerbound (however this may not be an actual solution). We will explore branches with the most promising bounds first.

General Principles ('Computer Science Distilled'):

- Divide the problem into subproblems
- Find lower and upper bounds for each subproblem
- Compare subproblem bounds of all branches
- Return to step 1 with the most promising subproblem

Branch and Bound pseudocode:

I am assuming that `lowerbound()` and `upperbound()` are predetermined functions that take in (weights, values, capacity) and are connected to the greedy 0-1 and greedy fractional approaches as stated above (floored as must be an integer). The algorithm stems from a BFS/graph search algorithm.

Node structure for pseudocode = (ub, lb, current_weight, current_value, index)

```

n = len(WEIGHTS)
max_value = 0

ub_root = upperbound(WEIGHTS, VALUES, CAPACITY)
lb_root = lowerbound(WEIGHTS, VALUES, CAPACITY)

queue = priorityqueue([(ub_root, lb_root, 0, 0, 0)]) //initialises priority queue
where priority is ub.
//Priority is set in descending order i.e. max-heap

while queue:
    ub, lb, current_weight, current_value, index = queue.pop()

    #Pruning
    if index >= n or current_weight > CAPACITY or ub <= max_value:
        continue //stop further exploring this subproblem

    #Update current best solution
    max_value = max(lb, max_value)

    #Explore branches including next indexed item
    new_weight = current_weight + WEIGHTS[i]
    if new_weight <= CAPACITY:
        new_value = current_value + VALUES[i]
        remaining_capacity = CAPACITY - new_weight

        ub_incl = new_value + upper(WEIGHTS[i+1:], VALUES[i+1:], remaining_capacity)
        lb_incl = new_value + lower(WEIGHTS[i+1:], VALUES[i+1:], remaining_capacity)

        if ub_incl > max_value:
            queue.push((ub_incl, lb_incl, new_weight, new_value, i+1))

    #Explore branches excluding next indexed item
    ub_excl = current_value + upper(WEIGHTS[i+1:], VALUES[i+1:], CAPACITY -
current_weight)
    lb_excl = current_value + lower(WEIGHTS[i+1:], VALUES[i+1:], CAPACITY -
current_weight)

    if ub_excl > max_value:
        queue.push((ub_excl, lb_excl, current_weight, current_value, i+1))

print(max_value)

```

Genetic Algorithms (GA)

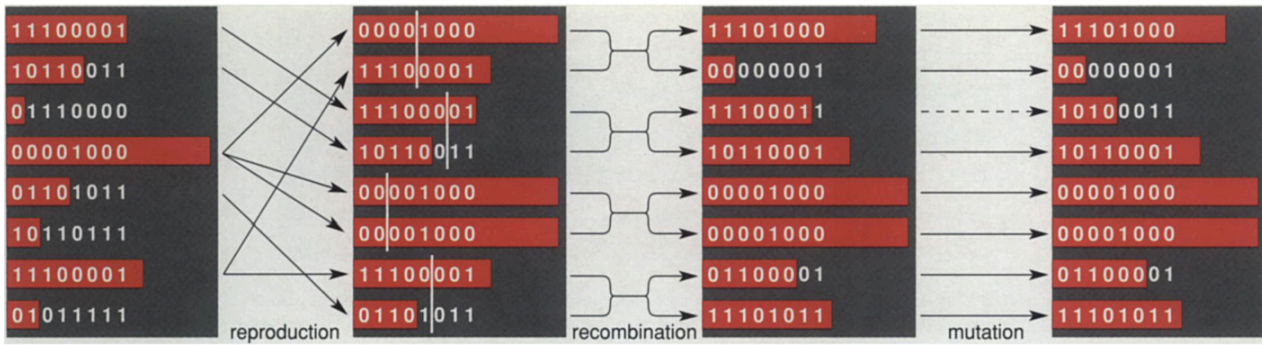


Figure 1. Genetic algorithms employ the mechanisms of evolution to solve optimization problems. Each candidate solution is encoded in a string of binary digits and assigned a “fitness,” which is represented here by the length of the red bar superimposed on the string. A population of strings evolves through selective reproduction, recombination and mutation.

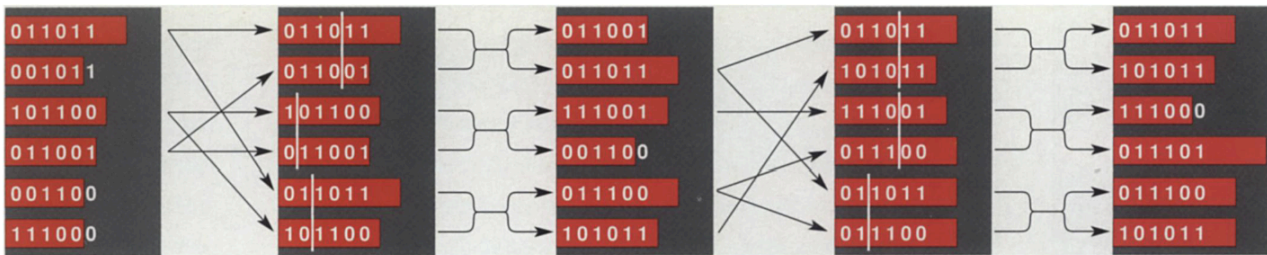


Figure 3. Optimum knapsack packing emerges after a few iterations of a genetic algorithm. Each selection of objects is represented by a binary string; for example, 011011 designates the choices of *b*, *c*, *e* and *f*. Total population fitness rises from 49 to 63, and an optimum solution appears.

- From reading the 'The Science of Computing: Genetic Algorithms' by Peter J. Denning it introduced me to the fundamental principles of using genetic algorithms to solve an optimisation problem. It also documented how this type of algorithm arose, its suitability, and common ways to implement reproduction, crossovers and mutations on populations.
- Generations: Over time a new set of 'n' (population size) solutions (valid or invalid) are generated with the average and highest fitness increasing over time until a near optimal solution is identified
- Fitness function: Used to evaluate the strength of the solution. It will return the sum of the values if the sum of the weights is less than the capacity. Else it will return 0 (as it acts as an invalid solution). The fitness function will therefore be discontinuous as adding one extra item to an optimal solution may evaluate it as 0.
- Mutation: To introduce variance into the population there is a small probability that a random 'bit' of a solution will change from generation to generation
- Crossover: Part of one solution is switched with part of another to create a new solution. The rate and position at which this happens is also random
- Elitism: The best 'x' solutions are preserved from generation to generation.
- Selection: Determining which solutions will contribute to the next generation
 - Tournament selection: There are many ways to implement this. A simple way goes as follows:

```

//while len(Population) < Population_Size
//(N1 and N2 are some constants <= to the Population_Size and N2 evenly
divides Population_Size)
Tournament_Size = N1
Individuals_Selected = N2

Tournament = GetRandomTournament(Tournament_Size) //Randomly choose a subset
of items from the population (each with equal probability)
Fitnesses = Evaluate(Tournament) //Calculate the fitness of each solution in
the tournament subset

return BestSolutions(Tournament, Fitnesses, Individuals_Selected) //This
returns the solutions from the population with the best 'k' fitnesses

```

- Roulette-wheel selection: A more complex selection process that ensures that the probability of selecting an individual is directly proportional to the fitness. It mimics a roulette wheel in the fact that the fitness determines the length of the wheel the solution occupies:

```

//To generate 'Roulette wheel' we must normalise probabilities so they sum to
1
Fitnesses = Evaluate(Tournament)
Total_Fitnesses = sum(Fitnesses)

Probabilities = [fitness / Total_Fitnesses for fitness in Fitnesses]

//Calculate cumulative probabilities so that the solution can be chosen
appropriately
Cumulative_Probabilities = []
Cumulative_Sum = 0
for probability in Probabilities:
    Cumulative_Sum += probability
    Cumulative_Probabilities.append(Cumulative_Sum)

Selected = [] //Solutions that will contribute to next generation
while len(Population) < Population_Size
    Random_Variable = random() //returns a random floating point number between
0 and 1
    for x in range(len(Cumulative_Probabilities)):
        if r <= Cumulative_Probabilities[x]:
            Selected.append(Population[x])

return Selected

```

```

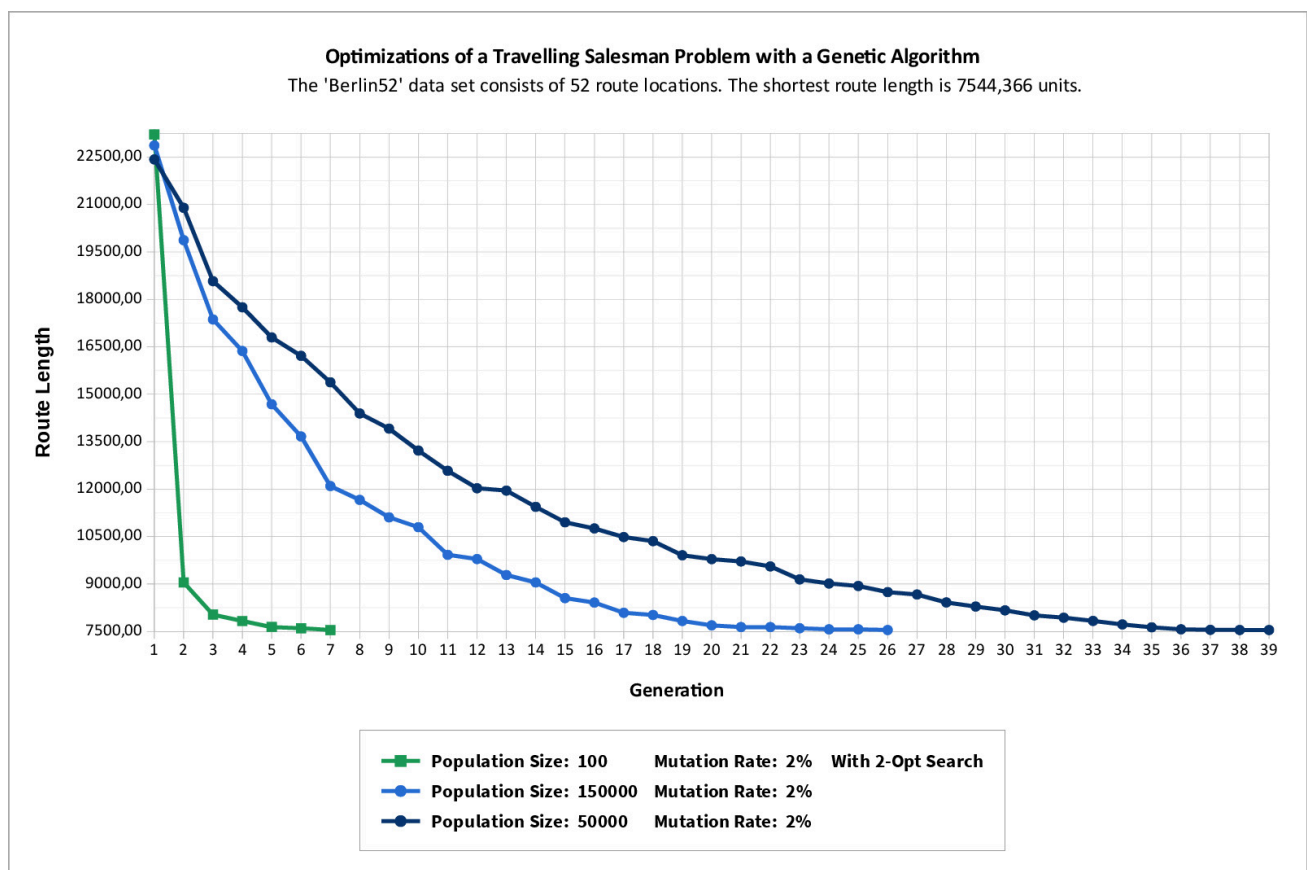
population = GenerateRandomPopulation()
for x in range(generations):
    #Selection
    if selection_type == 'Tournament':
        new_population = TournamentSelection()
    else:
        new_population = RouletteWheelSelection()
    #Crossover
    for y in range(len(pop_size)-1):
        if random() > cross_over_prob:
            new_population[y], new_population[y+1] = CrossOver(new_population[y],
new_population[y+1], random_int())

    #Mutate
    for z in range(len(pop_size)):
        for bit in range(n):
            if random() > mutation_rate:
                mutate(new_population[z], bit)

    Remeber_best_solution()

```

I found an interesting graph on genetic algorithms being applied to TSP, showing how different parameters effected the performance/route length over generations



Ant Colony Optimisation (ACO)



Ant colony optimisation is another heuristic method which typically is used on problems which can be reduced to finding good paths through graphs such as TSP. Here I will look at the theory behind it, why it works and some pseudocode. I researched using this paper 'Ant Colony Optimization Algorithm For The 0-1 Knapsack Problem'.

Overview description of the algorithm

ACO is inspired by the behaviour of ants when searching for food. Each ant lays down a pheromone trail (a type of hormone) when exploring a particular path. Over time, good/frequently chosen paths will accumulate pheromone, attracting more ants. Additionally, the choices a 'path' takes is influenced by other heuristic information (attractiveness). Evaporation of pheromone occurs, so that older suboptimal paths lose influence over time. Across many iterations, a near optimal solution should arise.

Detailed Steps

(I will follow similar notation layouts as stated in the paper above)

Each ant 'k' starts with an empty set 'S' which demonstrates a solution for the problem. They will build a final solution by selecting items over time until the set 'N' representing the available items left under the knapsacks remaining capacity is not empty. The quality of pheromone secreted to the selected items depends on the quality (value) of the final solution constituted by ant 'k'. Attractiveness is used to avoid a rapid conversion to a locally optimum solution, further assisted by Evaporation techniques. Phermenone is encoded globally so each ant observes other's behaviours.

The formula presented below shows the probability of an ant selecting that item. Alpha and beta are parameters that control how much pheromone and attractiveness respectively affect selection probability. The divisor is used to normalise the values for probability distribution. If the item is not in set N, i.e. the weight of that item exceeds the remaining knapsack capacity, the probability of selecting that items is 0 as it can't be packed.

$$p_j = \begin{cases} \frac{\tau_j^\alpha \mu_j^\beta}{\sum_{j \in N_i} \tau_j^\alpha \mu_j^\beta}, & \text{for } j \in N_i \\ 0, & \text{for } j \notin N_i \end{cases}$$

This formula states how the volume/strength of pheromone secreted relates to the quality of the solution and relative to the existing best solution found.

$$\Delta\tau = f(Q) = \frac{1}{1 + \frac{z_{best} - z}{z_{best}}}$$

This just shows how (new) pheromone = pheromone + change in pheromone

$$\tau = \tau + \Delta\tau$$

Below is shown how the Evaporation constant 'ρ' linearly affects the strength of pheromone over time at each iteration

$$\tau = \rho\tau, \quad \rho \in (0, 1]$$

This demonstrates how the 'attractiveness' of an item is calculated and acts as an additional heuristic. This is specifically chosen due to the testing as stated in the paper above (AKA2)

$$\mu_j = \frac{z_j}{w_j^2}$$

Pseudocode

```

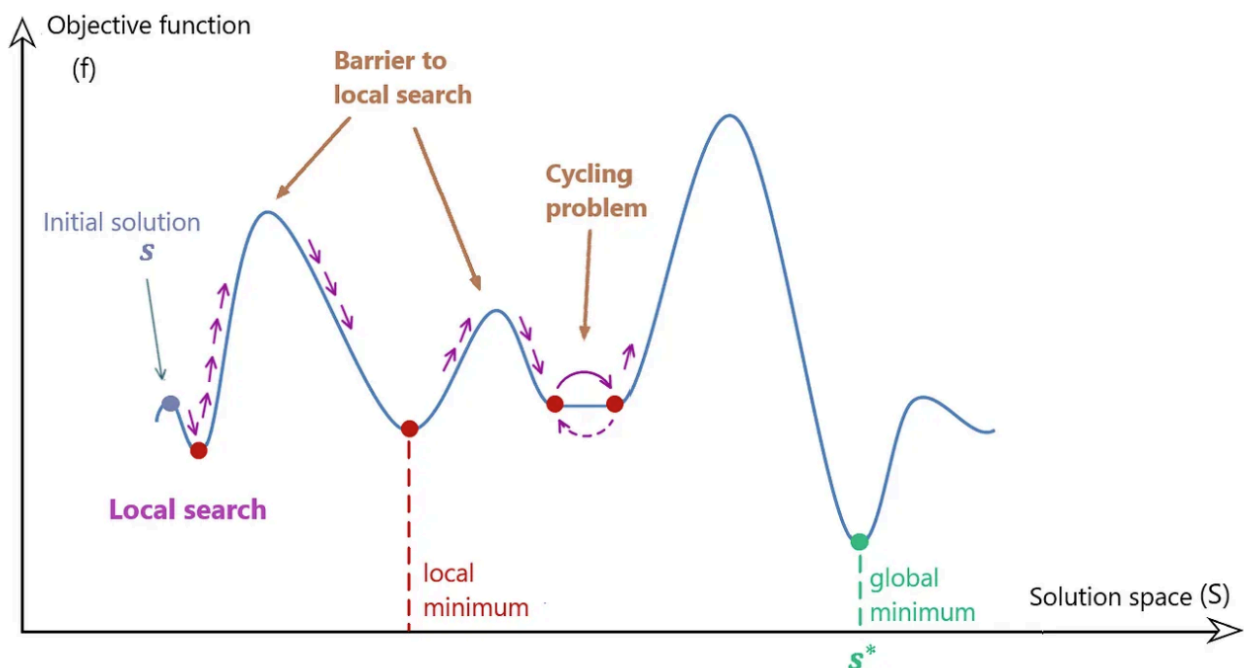
attractiveness = Calculate_Attractiveness(WEIGHTS, VALUES) //Calculates
attractiveness level before running algorithm as remain fixed
for i in range(Iteration_Count): //Loops for a fixed number of iterations
    for ant in Ants: //Loops over each ant
        //Ant generated some non-zero solution
        while Current_Capacity < CAPACITY:
            item = SelectItem(Available_Items, pheromones, attractiveness) //Selects new
item with probability as shown in formula
            Answer[item] = 1 //Include item in solution
            Current_Capacity += item[weight]
            Update_Value()
            Update_Available_Items(Available_Items)
        pheromones = Evapourate_Pheromones(Evaporation_rate) //Decrease each pheromone
level by a fixed rate
        pheromones = Update_Pheromones() //Pheromones of items that are included in ant's
solution are updated relative to the overall solution strength
        Remember_Best_Solution()
        Remember_Best_Solution()

return Best_Solution

```

A lot of the additional programming on top of this pseudocode will be general book keeping within the functions called.

Tabu Search



As discussed earlier in the analysis section, the Tabu Search algorithm is an advanced optimisation technique developed to overcome local optimality that traps simpler algorithms. Similar to other

metaheuristic approaches, the algorithm stems from a local search, where the algorithm starts with a feasible candidate solution and checks its immediate neighbours to find an improved solution.

The key distinguishing characteristic of Tabu search is it may accept worsening moves if no improving move is available, thereby avoiding strict local optima. Additionally, it keeps track of previously visited points in the search space, to avoid re-visited and redundant computations. It does this using prohibitions 'tabu' stored in a specialised data structure. Simulated Annealing is technically a form of Tabu Search, that I will further explore and compare the 2 approaches. Tabu Search is also suitable and adaptable for other variations of the knapsack problem.

Reading the following research paper 'Solving the 0-1 Multidimensional Knapsack Problem using Tabu Search and Visualization' from the National University of Singapore aided my understanding.

Tabu-Search Pseudocode

```
n = len(WEIGHTS)
solution = Generate_Random_Solution(n) // returns a random binary string of 0 and 1s
representing a new valid random starting solution. Alternatively, this could be
generated quickly by the greedy approach to further optimise the algorithm
value = Evaluate(solution)
best_solution = solution
best_value = value

tabu_list = []
tabu_list_maxsize = k

while iteration <= max_iterations:
    neighbours = Get_Neighbours(solution) //returns a list of all neighbouring solutions
    i.e. Flipped bits
    solution = Find_Best_Neighbour(neighbours, tabu_list) //evaluates solutions and
    select the best non-tabu solution
    value = Evaluate(solution)

    if value > best_value:
        best_solution = solution
        best_value = value

    tabu_list.append(solution)

    if len(tabu_list) > tabu_list_maxsize:
        tabu_list.pop() //Remove oldest elements

print(best_value, best_solution)
```

Simulated Annealing



Simulated Annealing (SA) is like ACO - inspired by real life processes. Annealing in metallurgy is when a material is first heated up to a high temperature and then cooled down slowly over time to achieve a stable crystalline structure. The algorithm takes inspiration from this with the 'heat' variable corresponding to the degree of randomness in the search process which will decrease over time to refine the solution.

Key characteristics ('what is simulated annealing')

- Begins with a random initial solution 'S' and initial temperature 'T'
- At each step a new solution 'S' is generated by making a small change, i.e. flipping a bit, to S
- The new solution S' is evaluated using the objective function and if S' has a better solution it is accepted
- If S' is a worse solution it may still be accepted (to avoid local optima) based upon the acceptance probability as shown below:

$$P(\text{accept}) = e^{-\frac{\Delta E}{T}}$$

- This means that the probability is equal to the negative exponential of the difference in solution value over the temperature T variable. This means if the difference in solution is low (only slightly worse) and the temperature is relatively high there is still a chance that this will still be accepted.
- After each step, the temperature is decreased based upon a pre set cooling schedule, which may therefore be set by the user. This could be linear, exponential logarithmic or some other decreasing function. The pseudocode below represents a standard linear schedule where after each iteration the temperature is decreased by a fixed amount.
- The algorithm will stop when the temperature gets low enough that new permutations are not improving the value of the solution or reaches a minimum temperature

The pseudocode for SA will follow the same structure as ACO as they are both metaheuristic approaches that solve the knapsack problem in different ways.

Pseudocode

```
n = len(WEIGHTS)
solution = Generate_Random_Solution(n) // returns a random binary string of 0 and 1s
representing a new valid random starting solution. Alternatively, this could be
generated quickly by the greedy approach to further optimise the algorithm
value = Evaluate(solution)
best_solution = solution
best_value = value
T = Tmax // initialise temperature to a maximum

new_solution = ''

while T > Tmin and iteration <= max_iterations:
    //Flip a random bit (like mutation in GA) to get a neighbouring solution
    while is_valid(new_solution) == True:
        random_bit = random.randint(0, n)
        new_solution = Flip_Bit(solution, random_bit)
    new_value = Evaluate(new_solution)

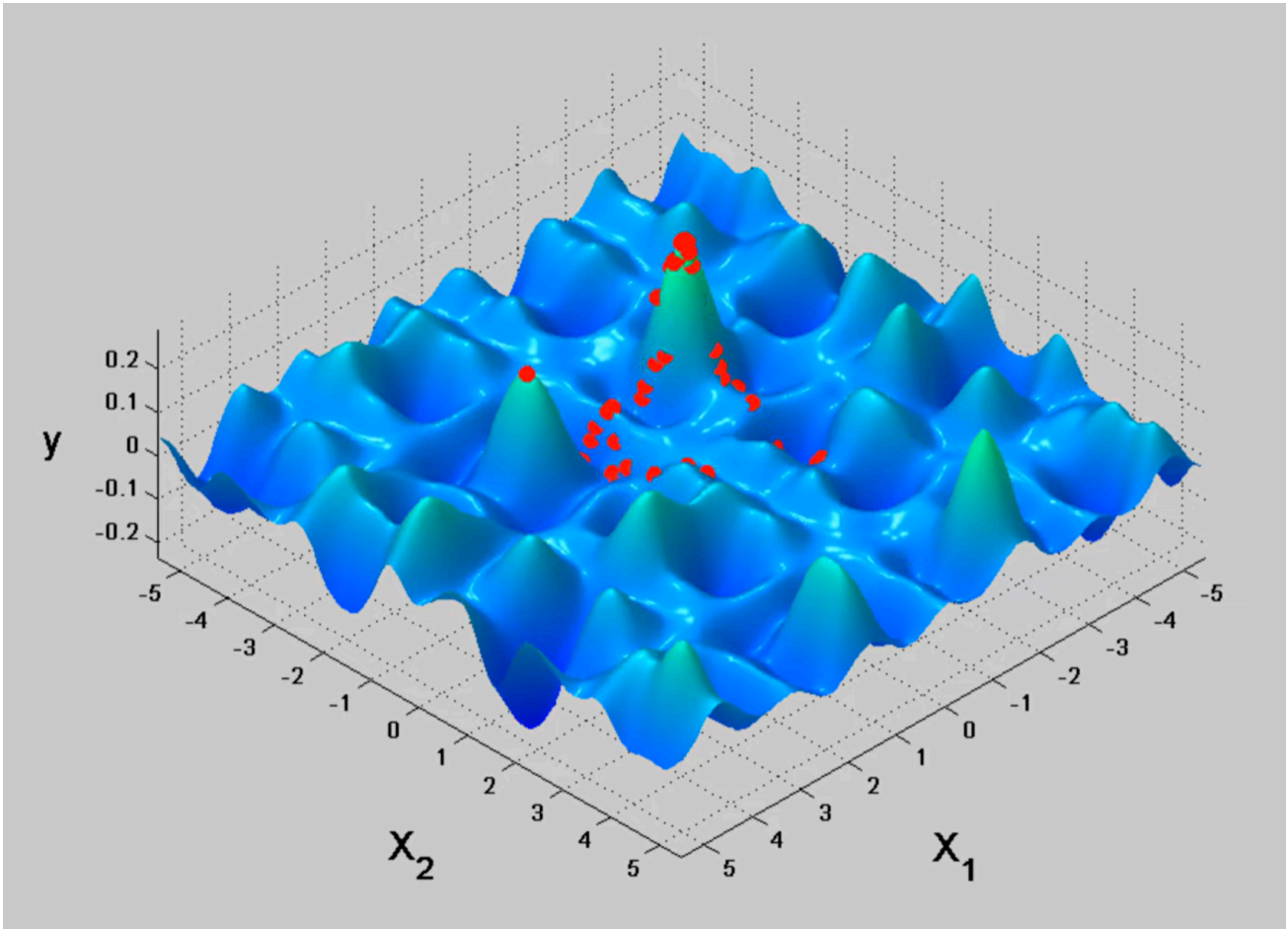
    if new_value > best_value:
        best_solution = new_solution
        best_value = new_value

    if new_value > value:
        solution = new_solution
        value = new_value
    else:
        if random.random() < e**((new_value - best_value) / T)
            solution = new_solution
            value = new_value

    T -= TLinearDecreasingRate

return best_solution
```

Particle Swarm Optimisation (PSO)



Particle swarm optimisation is inspired by natural life processes (Alife), in particular the social behaviour of birds flocking or fish schooling. It is designed to find the optimal solution in a high dimensional space by iteratively improving the candidate solution, as we have seen previously for example in GA and so can be effectively applied to the knapsack problem. I think this system will be most interesting to compare against ACO, and is the last approach I will explore.

The following article on swarm intelligence introduced me the key themes and ideas behind applying PSO 'PSO Tutorial' and this paper explored a modified BPSO for versions of knapsack problems from IIT 'A Modified Binary Particle Swarm Optimization for Knapsack Problems'.

Key Characteristics:

- A population (swarm) of candidate solutions is called particles, where each particle has position, velocity and fitness value over the search space
- Each particle is influenced by its own best and the global best known solution (pbest and gbest)
- PSO is usually designed for continuous problems where solutions are real numbers that can vary smoothly, however the knapsack problem is discrete and combinatorial
- Efficient for global searches over large spaces
- Does not require calculations of the functions gradients
- Few parameters to adjust

In depth analysis:

Specifically, I will be analysing binary particle swarm optimisation (BPSO) as the search space in this problem can be defined as $S = \{0, 1\}^D$ where D represents the number of items. (The effect of this is the equation for updating the position is re-defined, however techniques remain the same as PSO). To handle the constraints of the problem, I will implement a penalty factor/repair function as seen below and determine which is optimal (there is no penalty on a repaired solution).

BPSO mathematical formulation

N - number of particles in the swarm (swarm population)
 D - problem dimensions (the number of items in the knapsack)
 t - iteration index

Each particle i has a position and velocity vector
 where $i \in \{1, 2, \dots, N\}$

(position vector) $X_i^t = (x_{i1}^t, x_{i2}^t, \dots, x_{iD}^t)^T$ (like 'binary-string')
 $x_{id}^t \in \{0, 1\}$

(velocity vector) $V_i^t = (v_{i1}^t, v_{i2}^t, \dots, v_{iD}^t)^T$ $v_{id}^t \in \mathbb{R}^+$
 $v_{id}^t \in [-V_{max}, V_{max}]$

Each particle also keeps track of a personal best and global best solution.

(pbest) $P_i^t = (p_{i1}^t, p_{i2}^t, \dots, p_{iD}^t)^T$ $p_{id}^t \in \{0, 1\}$

(gbest) $G^t = (g_1^t, g_2^t, \dots, g_D^t)$ $g_d^t \in \{0, 1\}$ $G^t = P_{gd}^t$

Fitness function with penalty:
 (independent of time t) $\alpha \rightarrow$ penalty coefficient

$$f(X_i) = \begin{cases} \sum_{d=1}^D w_d x_{id} & \text{if } \sum_{d=1}^D w_d x_{id} \leq W \\ \sum_{d=1}^D w_d x_{id} - \alpha \left(\sum_{d=1}^D w_d x_{id} - W \right) & \text{otherwise.} \end{cases}$$

Velocity Update Equation (for each dimension 'd' of particle 'i')

$$\text{(new velocity)} \quad v_{id}^{t+1} = \omega \cdot v_{id}^t + c_1 \cdot r_{1id}^t \cdot [p_{id}^t - x_{id}^t] + c_2 \cdot r_{2id}^t \cdot [g_d^t - x_{id}^t]$$

ω = inertia $\omega \in \mathbb{R}^+$

c_1, c_2 = cognitive and social learning factors respectively $c_1, c_2 \in \mathbb{R}^+$

$r_{1id}^t, r_{2id}^t \sim U(0,1)$ (uniformly distributed in range 0 to 1)

Position update in PSO:

$$x_{id}^{t+1} = x_{id}^t + v_{id}^{t+1}$$

However in BPSO, velocity becomes probability that a bit (position) takes 0 or 1

$$x_{id}^{t+1} = \begin{cases} 1, & \text{if } U(0,1) < \text{Sigm}(v_{id}^{t+1}) \\ 0, & \text{otherwise} \end{cases}$$

where $\text{Sigm}(v_{id}^{t+1})$ represents a sigmoid limiting transformation function having an S-shape

$$\text{Sigm}(v_{id}^{t+1}) = \frac{1}{1 + e^{(-v_{id}^{t+1})}}$$



Greedy repair operator:

As this is a heuristic approach, many solutions generated may be infeasible.

Let $R(\cdot)$ modify X_i into a feasible solution such that $X_i' = R(X_i)$

A possible function of $R(\cdot)$ could stem from greedy approach or random selection

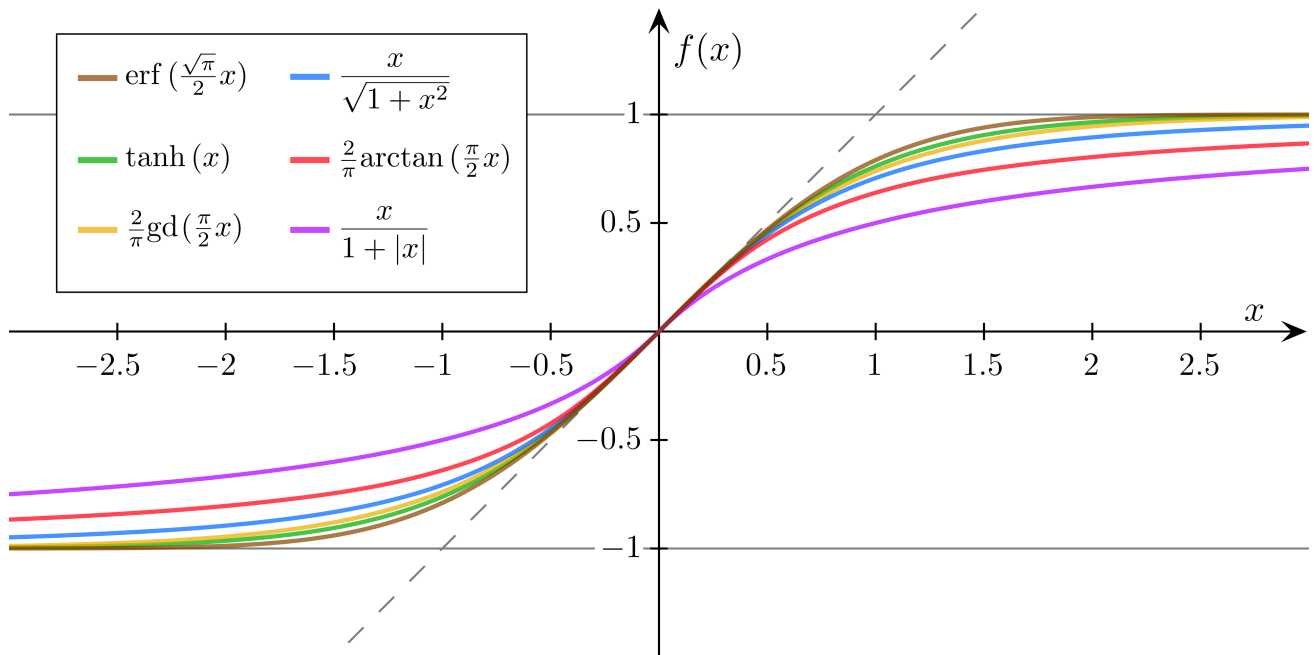
BPSO Pseudocode:

```
swarm = list<Particles>() //initialize all particles with random solutions
gbest = 0
while iteration <= max_iterations:
    for particle in swarm:
        new_fitness = particle.Calculate_Fitness_Value():
        particle.pbest = max(particle.pbest, new_fitness)
        gbest = max(particle.pbest, gbest)

    for particle in swarm:
        particle.Update_Velocity()
        particle.Update_Position()
```

Below is the various types of sigmoid functions (a type of 'S' shape mapping used in probabilistic analysis). This is used in the velocity update equation in order to transform the real-valued particle

velocity into a probability for updating binary positions. It allows continuous dynamics of PSO to operate effectively in binary search spaces, maintaining exploration and guidance towards global optima.



Example Repair Pseudocode taken from 'A New Hybrid Particle Swarm Optimization and Greedy for 0-1 Knapsack Problem' which compares penalty factor against a new repair algorithm

```

def repair(solution):
    ordering = GetGreedyOrdering() //returns list of ordering
    #ADD phase
    gap = CAPACITY - total_weight
    i = 0
    new_solution = ''

    while gap > 0 and i < N:
        index = greedy_ordering[i]
        if solution[index] == '0' and gap >= WEIGHTS[index]:
            new_solution[index] = '1'
            gap -= WEIGHTS[index]
            total_weight += WEIGHTS[index]
        i += 1

    #DROP phase
    over = total_weight - CAPACITY
    i = 0
    while over > 0 and i < N:
        index = greedy_ordering[N-1-i]
        if solution[index] == '1':
            new_solution[index] = '0'
            over -= WEIGHTS[index]
        i += 1

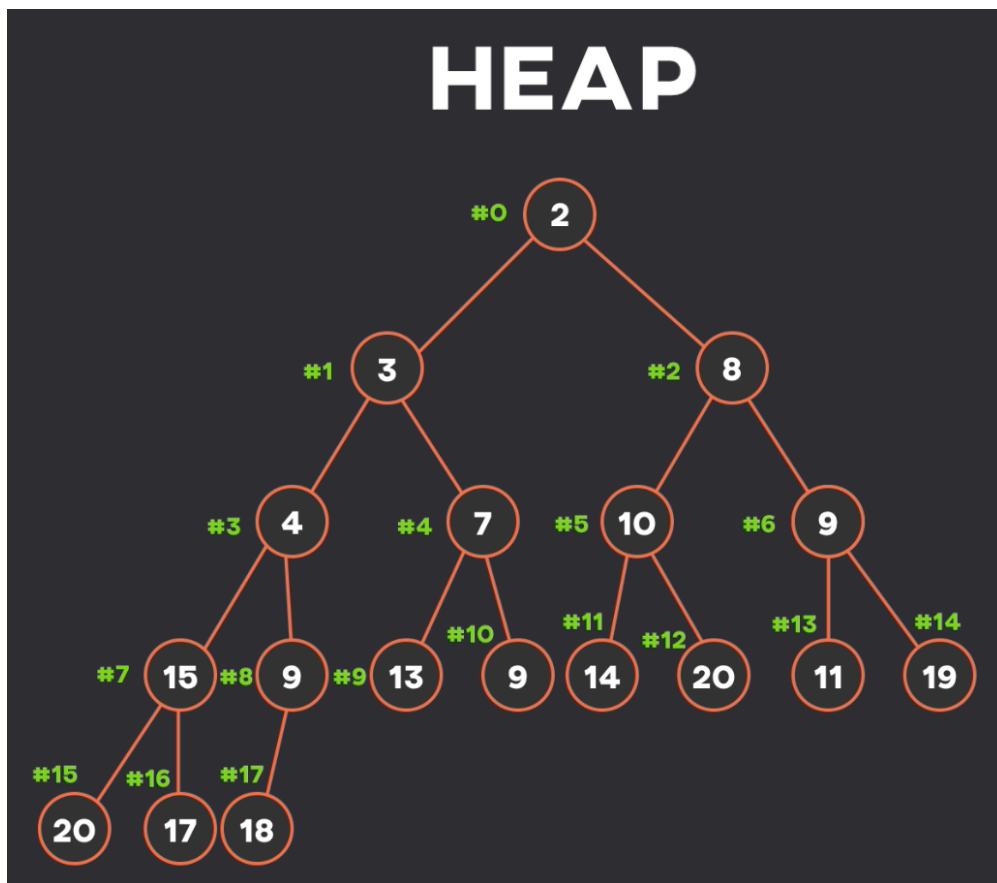
    return new_solution

```

Specialised Data Structures

I decided to create a table and brief description of each data structure I used, as this will help me during implementation and illustrates the complexity within parts of my code.

Data Structure	Algorithm Used By	Explanation
Heap/Priority Queue	• Branch and Bound - Getting the next node with the highest upper bound • Genetic Algorithm (Elitism) - Efficiently finding the biggest n item indexes in a population	'Heaps in 3 minutes" video by Michael Sambol clearly outlines what and how a heap works. A heap is a nearly complete binary tree which satisfies the heap property (max-heap: every vertex $i \leq$ value of parent vertex and a min-heap: every vertex $i \geq$ value of parent vertex). They are particularly useful when you need to repeatedly access the smallest or largest elements in a dynamic dataset
Numpy Array	DP Tabulation - Cropping data into a labelled table to be displayed on output (This could further be adapted for storing the actual dp table)	Used for handling large, multi-dimensional arrays and matrices providing efficient storage and faster processing for numerical and scientific computations. They are mutable but static (have a fixed size)
Dictionary (defaultdict)	Dictionaries are used throughout the code to group data together	A dictionary is a collection of key-value pairs. In python, dictionaries are implemented as hash tables internally. A default dict adds to the functionality by automatically assigning a default value to keys that do not exist, which can be helpful when adding lots of new items to the dictionary



Above shows a picture of a min-heap data structure (every vertex $i \geq$ value of parent vertex)

Input Process Storage Output Chart - IPSO Chart

IPSO charts are used as a form of algorithm presentation, and helps explain how data is transformed into a flow of information by a program by detailing how subroutines interact with one another.

Algorithm	Input	Process	Output	Storage
Generating random data	Number of items, range, correlation strength, capacity load (how tight is the weight capacity)	Randomly generates data within the parameters given	Weights, values, capacity	Displayed within web app (contained within the html)
Parsing CSV file	CSV file with weight, values and capacity columns	Reads the CSV file using pandas checking for any errors, before converting to python lists that can then be	Weights, values, capacity	Displayed within web app

Algorithm	Input	Process	Output	Storage
		passed into algorithms		
Exact Algorithms	Weights, values, capacity	Solves the problem through some specified algorithm which will exhaust all possible solutions in some way	Binary string representing which items are included in the 'best' subset and maximum total value of items packed in knapsack	Algorithm only stores temporary relevant data
Heuristic Algorithms	Weights, values, capacity	Approximates the problem either through improving a group of test solutions or by using some property of each item	Binary string representing which items are included in chosen subset and total value of items packed in knapsack	Algorithm only stores temporary relevant data

4 - GUI (Top Down Design)

This section describes the user interface that the user will interact with to control the parameters of each algorithm. I want to keep my design simple and minimalistic yet visual.

Generating input data

In order to fully demonstrate and compare the logic, advantages and disadvantages of the algorithms created in the section above, the input data must be sufficiently varied and have controllable parameters. The user also must be easily inspect the generated data or input their own data for research or practical applications.

Usually in real life examples and most data sets I have seen for the knapsack problem, there will be a general positive correlation between the value and the weight of an item, making the opportunity cost (maximisation) component more complicated. There is no explicit function that determines how 'high quality' the data is because of its random nature, so is left to the user in order to adjust parameters to change certain characteristics including size, spread, correlation, mean, number of outliers, perhaps even distributions of data shown as graphs - depending on the flexibility needed.

In order to introduce controllable randomness, I would either have to filter generated data or generate new random data based on previous random values. The main difference is the first approach is

easier to program and has simpler logic, but for large data sets may generate a lot of redundant data leading to inefficient computations. I will outline the basic pseudocode for each approach below.

Approach 1

```
while len(WEIGHTS) < N:
    weight = random.randint(1, max_weight)
    value = random.randint(1, max_value)
    if abs(weight - value) > max_difference:
        continue // skip this iteration if the difference between the 2 values is too
extreme
    WEIGHTS[x] = weight
    VALUES[x] = value
```

Approach 2

```
for x in range(N):
    weight = random.randint(1, max_weight)
    value = min(max_value, max(1, random.randint(int(weight*(1-
correlation_strength)), int(weight*(1+correlation_strength))))
    WEIGHTS[x] = weight
    VALUES[x] = value
```

I will therefore use approach 2 as it will execute faster.

Working with Flask

In my analysis section, I originally stated I wanted to use Flask in order to make a website. Here I will further research into this technology, and try to get used to the syntax of the library, and installing it with VS code - my choice of IDE.

Flask is a light weight framework also known as a 'micro-framework' because it provides the essential tools for building a website without unnecessary complexity. This is why it is often chosen over Django etc.

Features:

- Flask uses decorators to map URLs to python functions simplifying the process of defining application routes
- Jinja2 (the library that flask is based on) allows for dynamic HTML generation, making it easy to create web pages with variables and loops
- The framework is highly flexible standing out to both beginners and expert programmers

Rendering Graphs

In order to show data from my algorithms, I wanted to use matplotlib which is commonly used with python to create static and interactive visualisations. I have used this before and will allow the user to view data relevant to the algorithms data and performance.

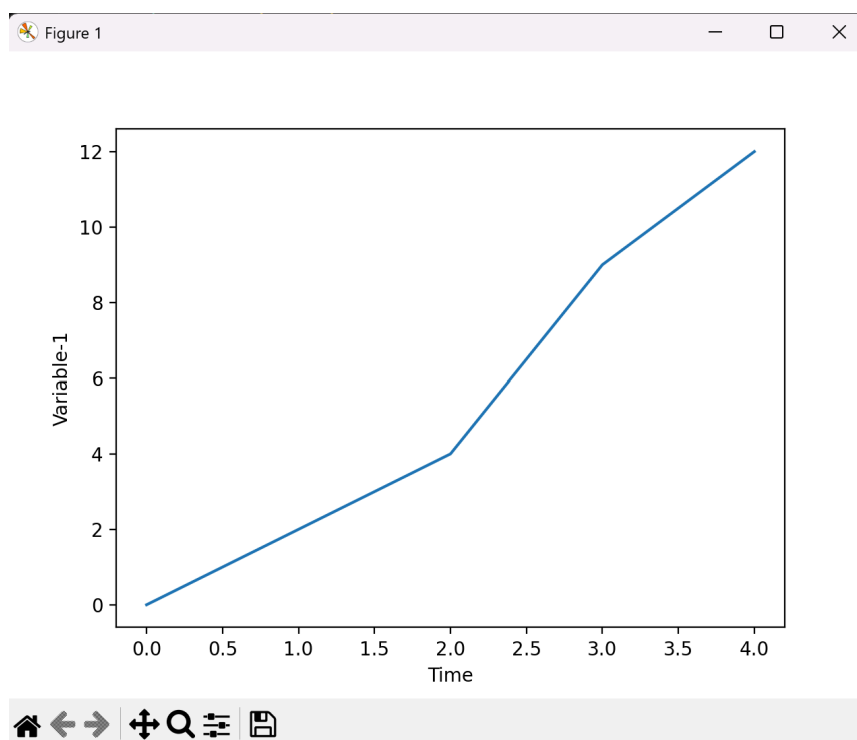
Below shows a simple python code to plot a graph

```
import matplotlib.pyplot as plt

//Example scatter data
x = [0, 1, 2, 3, 4]
y = [0, 2, 4, 9, 12]

plt.plot(x, y) # Create a line plot
plt.xlabel("Time") # Label the x-axis
plt.ylabel("Variable-1") # Label the y-axis
plt.show() # Display the plot
```

When running this code it produces the following result in a window/applet:



I will then need to render this as an image and embed this using flask within the website. This process will allow me to insert static graphs/visualisation elements onto the page.

Design Interface

Below is an annotated sketch of how I want the home page and submission page to look. I have included all key details such as viewing items, descriptions and graphs, as well as trying to make it look aesthetically pleasing. I will try and design my actual html and css around these images.

index.html

Title text

Title image

Problem definition

Paragraph explaining algorithm

parameters.

Large scroll box for items

Submit.html

Name of algorithm selected

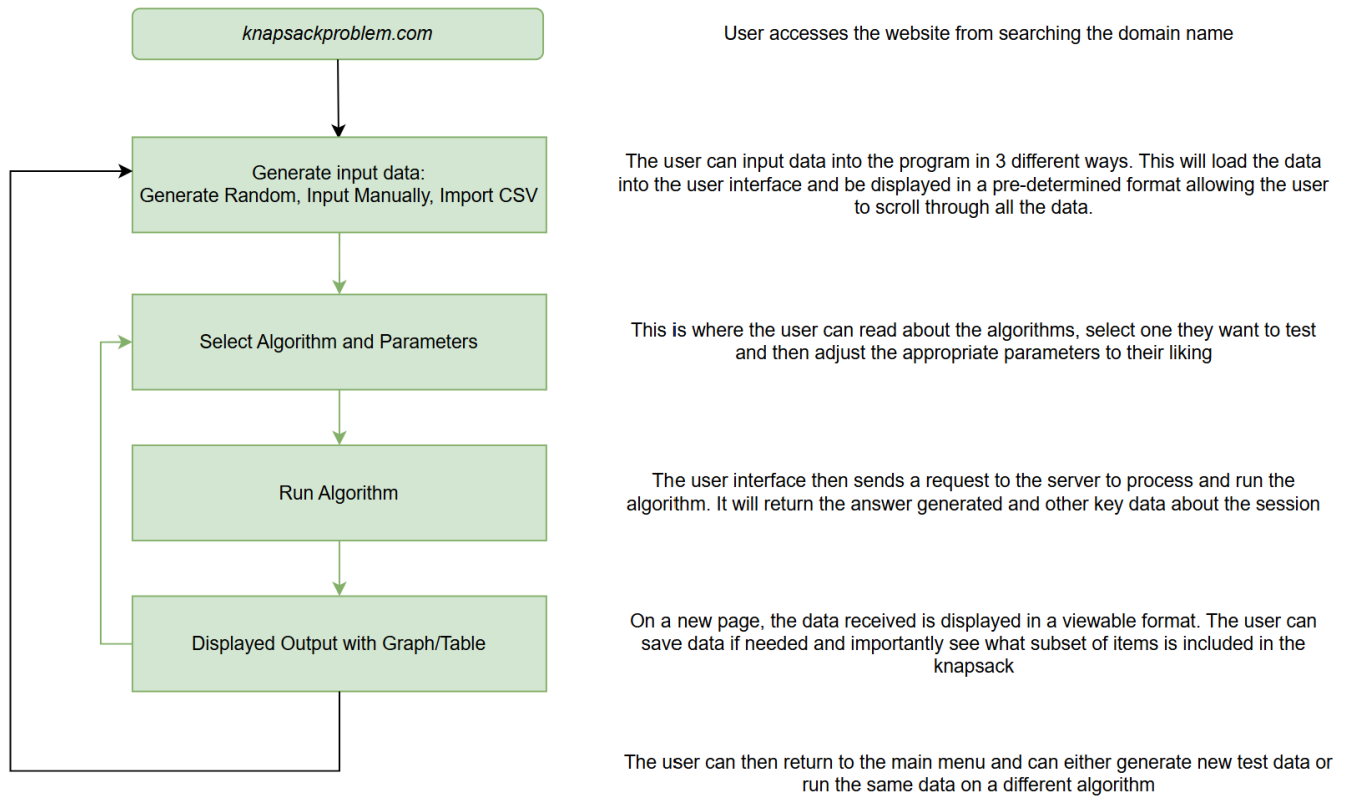
Comparative data

Subset selected

i^{th} digit corresponds to i^{th} item

Save graph option

This shows what the user's workflow may look like when visiting the website, and outlines how they should use the website to compare different algorithms.



5 - Conclusion

I have now completed all aspects and prerequisites needed and now can begin implementing and designing a full technical solution.

My plan is to develop the algorithms and back end first as this is where the significant complexity is. I will then create a front end interface and connect both using Flask. I will document parts of the development process through the testing section.

6 - Bibliography

- '0/1 Knapsack problem | Dynamic Programming'. William Fiset. From <https://www.youtube.com/watch?v=cJ21moQpofY>
- '0-1 Knapsack Problem using Branch and Bound'. geeksforgeeks. From <https://www.geeksforgeeks.org/dsa/0-1-knapsack-using-branch-and-bound/>
- 'Computer Science Distilled'. Book authored by Wladston Ferreira Filho.
- 'The Science of Computing: Genetic Algorithms' Academic Paper by Peter J. Denning. From <https://www.jstor.org/stable/29774553>
- 'Ant Colony Optimization Algorithm For The 0-1 Knapsack Problem' Academic Paper by Krzysztof Schiff. From https://repozytorium.biblos.pk.edu.pl/redu/resources/30706/file/suwFiles/SchiffK_AntColony.pdf

- 'Solving the 0-1 Multidimensional Knapsack Problem using Tabu Search and Visualization' from Trung Tuan Dung and other authors. From https://www.comp.nus.edu.sg/~stevenha/database/research_paper/tuandung_etal_2005.pdf
- 'what is simulated annealing'. geeksforgeeks. From <https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/>
- 'PSO Tutorial' Xiaohui Hu 2006. From <http://www.swarmintelligence.org/tutorials.php>
- 'A Modified Binary Particle Swarm Optimization for Knapsack Problems' Jagdish Chand Bansal and Kusum Deep. From https://people.sau.int/~jcbansal/uploads/11-Modified_BPSO_for_Knapsack_Problem.pdf
- 'A New Hybrid Particle Swarm Optimization and Greedy for 0-1 Knapsack Problem' Indonesian Journal of Electrical Engineering and Computer Science. From https://www.researchgate.net/publication/301353661_A_New_Hybrid_Particle_Swarm_Optimization_and_Greedy_for_0-1_Knapsack_Problem
- 'Heaps in 3 minutes'. Michael Sambol. From <https://www.youtube.com/watch?v=0wPlzMU-k00>

A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the Knapsack Problem

Technical Solution

Name: Aarav Bhatnagar - Centre Number: 39253 - Candidate Number: 8015



Table of Contents

- [1 - Introduction](#) (pg 61)
- [2 - Algorithms.py](#) (pg 62)
- [3 - index.py](#) (pg 96)
- [4 - base.html](#) (pg 102)
- [5 - index.html](#) (pg 102)
- [6 - submit.html](#) (pg 119)
- [7 - main.css](#) (pg 124)
- [8 - requirements.txt](#) (pg 133)
- [9 - vercel.json](#) (pg 134)
- [10 - Bibliography \(Links and Documentation\)](#) (pg 135)

1 - Introduction

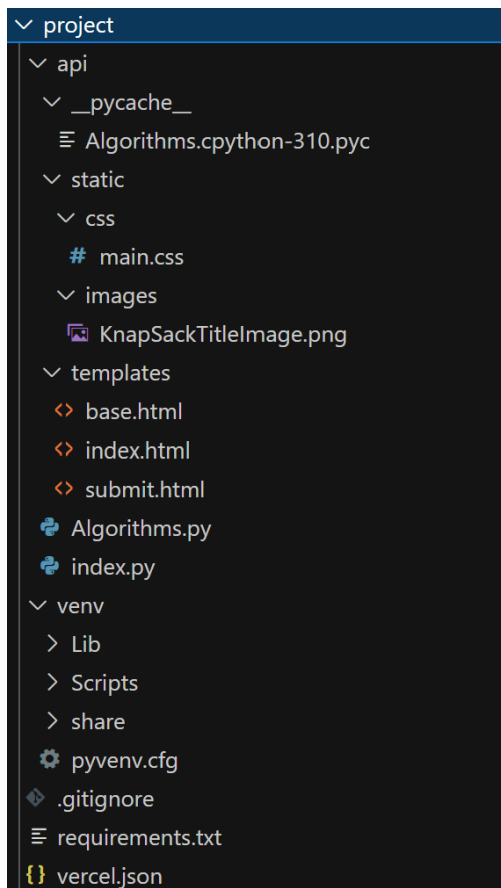
In this section, I will document and annotate my final program constituting to my end product. My final solution is currently deployed live using vercel as a website under this link knapsackproblem.com and the alternative domain knapsack-problem-rose.vercel.app, which can be compared alongside this code and can be used to actively test and demonstrate my algorithms alongside my front end design. **I have commented my code in significant detail, as well as having a general summary over more complicated and difficult to implement sections.**

QR code for Website



Project File Directory

Here is the file structure for my project to put the following code descriptions in context. The *venv* describes a virtual environment created using the pip command `python -m venv venv`, which is used to install local packages, which is very helpful when keeping track of requirements that need to be installed when deploying as you will see in **8 - requirements.txt**.



2 - Algorithms.py

This script houses the main complexity of the product - including all the logic for the algorithms and their own visualisation elements that will then be rendered and inserted into the webpage. **As all the algorithms are mostly just implementations of the pseudocode in the documented design, I will not re-explain the fundamental ideas of each algorithm, but rather how I implemented it**

Underneath is the start of the code which includes importing the libraries/extensions used, and creating class definitions with their `--init--()` functions. As mentioned in the documented design UML Object diagram, I only have 2 classes. The *Particle* class is created here, but used in the particle swarm optimisation method in order to treat each particle as its own entity with its own attributes.

There is one main class *BinaryKP* for the knapsack problem with most methods representing algorithms or parts of algorithms that solve this problem. These methods share the same general parameters for example (weights, values, capacity) along with their specific parameters i.e. number of ants for ACO and then output (max_value, binary_string) as well as a figure/table for the visualisation element.

```

#Here are the imported libraries and functions that I will be using
import time
import random
import heapq
import math
from matplotlib.figure import Figure
import numpy as np
from matplotlib.lines import Line2D
from collections import defaultdict

class Particle:
    def __init__(self, n):
        self.position = [random.randint(0, 1) for x in range(n)] #represents a
possible solution
        self.velocity = [random.random() for x in range(n)] #the velocity of a
particle which is initially randomised
        self.pbest = self.position[:] #the personal best solution seen
        self.pbest_fitness = 0 #the value of the personal best solution
        #Used for the graph/visualisation
        self.pbest_list = []
        self.velocity_list = []

class BinaryKP:
    def __init__(self, weights, values, capacity, algorithm_data):
        #Constant terms between each algorithm
        self.weights = weights
        self.values = values
        self.capacity = capacity
        self.algorithm_data = algorithm_data #This achieves 5.2 Algorithm Parameter
objective
        #algorithm_data is the dictionary from the input form and index.py
        #Used for easy access in the recursive algorithm
        self.recursion_counter = 0
        self.hits = 0

```

Below shows the brute force iterative function. I optimised the generic naive iterative approach that was described in the documented design as I realised that in order to get to a $O(1)$ time to check a solution, we could simply count up in binary but only flip one bit at a time using Gray codes as explained in the 'What Is Gray Code?' video linked below. This can be achieved quite easily using binary arithmetic and the gray code formula $gray = mask \wedge (mask \gg 1)$. The downside to the iterative approach on large data sets is it can only count up to a small value of n bits (2^n solutions), meaning that after the algorithm has reached its max iterations, the total value is very small. This function details the **1.1 Brute Force** objective.

```

def solve_01_knapsack_BF_iterative(self):
    #graph max_length
    max_length = 10000
    #Loop through all subsets/combinations of items and return one with the
highest total value that doesn't exceed self.capacity
    #A perhaps interesting way of doing this is counting upwards in binary
    #In order to count up efficiently, we must use gray codes so that between
each iteration we only flip one bit, thereby the O(n) evaluate function is not called
n times.

    n = len(self.weights)
    max_value = 0
    best_mask = 0 # to store the Gray code subset

    total_weight = 0
    total_value = 0

    best_values = []
    current_values = []
    iterations_list = []

    iterations = min((2**n), self.algorithm_data['max_iterations_iterative'] + 1)
    # Calculate step size to reduce list to <= max_length
    step = max(1, (iterations - 1) // max_length)

    for mask in range(1, iterations):#max loops from 1 to (2**n) - 1
        gray = mask ^ (mask >> 1) # compute Gray code i.e. 1110

        # Previous Gray code
        prev_gray = (mask - 1) ^ ((mask - 1) >> 1) #sub (mask - 1) into formula
i.e. 1010

        # Bit that changed
        diff = gray ^ prev_gray #XOR because 1110 1010 gives 0100
        i = n - ((diff).bit_length()) # index of changed bit (bit length returns
the number of bits to store number there 00010 returns 2)

        if gray & diff != 0: # item added prev_gray = 001, gray = 011 therefore
diff = 010: gray & diff = 011 & 010 = 010 which is not zero => item is added
            total_weight += self.weights[i]
            total_value += self.values[i]
        else: # item removed
            total_weight -= self.weights[i]
            total_value -= self.values[i]

        if total_weight <= self.capacity and total_value > max_value:
            max_value = total_value
            best_mask = gray # store best subset mask

```

```

        #Due to the significant amount of iterations, I decided to only record
data for the graph after a fixed number of steps
        if mask % step == 0:
            best_values.append(max_value)
            current_values.append(total_value if total_weight <= self.capacity
else 0)

            iterations_list.append(mask)

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

axis.plot(iterations_list, best_values, marker='o', label='Best Value')
axis.plot(iterations_list, current_values, label='Current Value', alpha=0.5,
color='grey')

axis.set_title('Brute Force (iterative) - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')
axis.legend()

# Format mask as binary string (n bits, left to right = item 0..n-1)
return max_value, bin(best_mask)[2:].zfill(n), fig, end_time

```

Here is the other brute force approach that uses recursion to solve the problem while also generating a decision tree to be displayed to the user. The *plot_node()* function takes in the axis subplot and marks the individual node in the decision tree at the given (x, y) cartesian coordinates. This function is then called at the beginning of every recursive *solve()* call, where *solve()* represents the inner contained function. After the recursion has stopped executing, using the stored prefix dictionary and comparing this against the solution returned, the selected path taken of maximum value can be highlighted. This function completes the **1.1** Brute Force objective and completes the **8.1** Decision Tree objective.

```

#Recursively called plotting methods
def plot_node(self, axis, x, y, lastnode=False):
    node_size = 10
    axis.scatter(x, y, s=node_size, color='black', label='node', zorder=2)

    if lastnode:
        #If it is a last node, the branch will end and display a diamond rather
        #than a circle
        axis.scatter(x, y-0.25, s=2, color='black', marker='d', alpha=0.5,
        label='hidden nodes')

def solve_01_knapsack_BF_recursive(self):
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)
    axis.axis('off')
    axis.set_title('Brute Force (recursive) - 0/1 Knapsack Problem')

    #Number of layers shown on the decision tree
    max_tree_layer = 7

    items = len(self.weights)

    #used for the decision tree when selecting the correct object to be
    #highlighted
    path_edges = defaultdict(dict)
    #inner recursive function
    def solve(capacity=0, n=0, best_combination='', plot_pos_x=0, plot_pos_y=0,
    dx=20):
        if n == 0:
            capacity = self.capacity

            self.recursion_counter += 1
            if n == max_tree_layer and items > max_tree_layer and plot_pos_y != 0:
                self.plot_node(axis, plot_pos_x, plot_pos_y, lastnode=True)
            elif n <= max_tree_layer:
                #Call the plot node function which adds to the figure the marked plot
                #at a given x, y coord
                self.plot_node(axis, plot_pos_x, plot_pos_y)

            if self.recursion_counter >=
self.algorithm_data['max_iterations_recursive']:
                return (0, '')

        if n == len(self.weights) or capacity == 0:
            #Adds '0' to the end of the string in order to ensure it is the
            #specified length

```

```

        return (0, best_combination.ljust(len(self.weights), '0'))
    else:
        including_item = 0
        if self.weights[n] <= capacity:
            #including item
            xi, yi = plot_pos_x - dx, plot_pos_y - 1
            if n < max_tree_layer:
                #Plots a line between previous node and new node position
                line = axis.plot([plot_pos_x, xi], [plot_pos_y, yi],
color="green", label="included", zorder=1)
                path_edges[n][best_combination + '1'] = line
                including_item, best_combination1 = solve(capacity-
self.weights[n], n+1, best_combination + '1', xi, yi, dx / 2)
                including_item += self.values[n]

            #excluding item
            xe, ye = plot_pos_x + dx, plot_pos_y - 1
            if n < max_tree_layer:
                #Plots a line between previous node and new node position
                line = axis.plot([plot_pos_x, xe], [plot_pos_y, ye], color="red",
label="excluded", zorder=1)
                path_edges[n][best_combination + '0'] = line
                excluding_item, best_combination2 = solve(capacity, n+1,
best_combination + '0', xe, ye, dx/2)

            if including_item > excluding_item:
                return (including_item, best_combination1)
            else:
                return (excluding_item, best_combination2)

value, solution = solve()

end_time = time.perf_counter()

#highlight selected path
for depth in range(min(max_tree_layer, items)):
    prefix = solution[:depth + 1]
    line = path_edges[depth].get(prefix)
    if line:
        #Coloured blue and labelled select
        line[0].set_color('blue')
        line[0].set_label('selected')

#In the table I will show the number of recursive calls
display_text = "Number of recursive calls: " + str(self.recursion_counter) +
(" (limit exceeded)" if self.recursion_counter >=

```

```

self.algorithm_data['max_iterations_recursive'] else '')
    axis.text(
        0.5, 0, display_text,
        transform=axis.transAxes,
        fontsize=12,
        fontweight="bold",
        va="center",
        ha="center",
    )

    #The decision tree will be incomplete if the number of recursive calls
exceeds the recursion limit
    #This is because not all branches will be explored

    lines, labels = axis.get_legend_handles_labels()
    # Merge them, removing duplicates by label
    by_label = dict(zip(labels, lines))

    axis.legend(by_label.values(), by_label.keys(), loc='upper right')

    return value, solution, fig, end_time

```

The *solve_01_knapsack_GApprox()* and *solve_fractional_knapsack_G()* are both very similar algorithms and are also both used by the branch and bound algorithm to generate lower and upper bounds respectively. They both start by creating a 'greedy ordering' which is just the items ordered in descending order (*sorted(list, reverse=True)*) of value/weight ratio. The difference between the algorithms are how these items are selected, as fractional will include the maximum possible fraction of the last item. *GApprox* relates to the **2.2** Greedy Approximation objective while *fractional_knapsack_G* relates to the **1.3** Branch and Bound objective.

The graph generated by the greedy approximation highlights the included items in green and excluded items in red. It plots the value/weight ratio against the items in the order in which they were selected showing a straight line with a negative gradient as expected.

```

def solve_01_knapsack_GApprox(self, weights=[], values=[], capacity=[],
get_total_weight=False, generate_graph=False):
    #To get a general approximation (this would be exact for the fractional
knapsack problem), we can generate a list of value / weight
    if len(weights) == 0:
        weights = self.weights
        values = self.values
        capacity = self.capacity

    n = len(weights)
    items = [None] * n
    for x in range(n):
        weight = weights[x]
        value = values[x]
        items[x] = ((value/weight, weight, value, x))
    items = sorted(items, reverse=True)
    #Sorts in descending order
    total_weight = 0
    max_value = 0
    best_combination = ['0'] * n
    final_item_index = 0
    #includes item until the current item weight + total weight > capacity
    for x in range(n):
        if items[x][1] + total_weight > capacity:
            final_item_index = x
            break
        total_weight += items[x][1]
        max_value += items[x][2]
        best_combination[items[x][3]] = '1'

    if generate_graph:
        #Only called when not used as bound or starting solution
        #Graph generated is ordered by the items selected not the actual item
index
        end_time = time.perf_counter()
        fig = Figure(dpi=250)
        axis = fig.add_subplot(1, 1, 1)

        item_list = list(range(n))
        item_list_1 = item_list[:final_item_index]
        item_list_2 = item_list[final_item_index:]

        axis.scatter(item_list_1, [items[x][0] for x in range(final_item_index)],
color='green', label='included')
        axis.scatter(item_list_2, [items[x][0] for x in range(final_item_index,
n)], color='red', label='excluded')

```

```

        axis.set_title('Greedy Approximation - 0/1 Knapsack Problem')
        axis.set_xlabel('Items Sorted')
        axis.set_ylabel('Value/Weight Ratio')
        axis.legend()
        #greedy graph will have
        return max_value, "".join(best_combination), fig, end_time
    else:
        if get_total_weight:
            return max_value, "".join(best_combination), total_weight
        else:
            return max_value, "".join(best_combination)

def solve_fractional_knapsack_G(self, weights, values, capacity):
    n = len(weights)
    items = []
    #Same as in solve_01_knapsack_GApprox()
    for x in range(n):
        weight = weights[x]
        value = values[x]
        items.append((value/weight, weight, value))
    items = sorted(items, reverse=True)
    total_weight = 0
    max_value = 0
    for x in range(n):
        if items[x][1] + total_weight > capacity:
            max_value += (items[x][2] * ((capacity - total_weight)/items[x][1]))
            #Includes the maximum fractional amount of this last item
            break
        total_weight += items[x][1]
        max_value += items[x][2]

    return max_value

```

These are as explored earlier 2 dynamic programming methods which have been implemented below. The top down approach stems from the brute force recursive approach however I manually cache results in the *memo* dictionary. Therefore before each call, it checks if it has been called before, and after each call it stores the result in the memo. The graph then just plots the memo size and the number of hits (when the result is found pre computed in the memo) against the iteration count.

The bottom up solution is simply an implementation of the pseudocode in the documented design. It also re-traces the complete table in order to identify the binary string/actual subset. It does this by starting in the right hand corner of the table and traces back by identifying when an item has been included or excluded. After calculating the result for the DP table, the *dp* array is efficiently sliced using numpy arrays. These contribute to the **1.2** Dynamic Programming objective.

```

def solve_01_knapsack_DP_Top_Down(self):
    memo = {}
    memo_size = []
    hits_list = []
    #follows the same recursive idea as in solve_01_knapsack_BF_recursive()
    def solve(capacity=0, n=0, best_combination=''):
        self.recursion_counter += 1
        memo_size.append(len(memo))
        hits_list.append(self.hits)
        if n == 0:
            capacity = self.capacity

        if n == len(self.weights):
            return (0, '')
        elif capacity == 0:
            return (0, '0' * (len(self.weights) - n - 1))
        elif (capacity, n) in memo:
            self.hits += 1
            return memo[(capacity, n)]
        else:
            including_item = 0
            if self.weights[n] <= capacity:
                including_item, best_combination1 = solve(capacity-
self.weights[n], n+1, best_combination)
                including_item += self.values[n]

            excluding_item, best_combination2 = solve(capacity, n+1,
best_combination)

            if including_item > excluding_item:
                result = (including_item, '1' + best_combination1)
            else:
                result = (excluding_item, '0' + best_combination2)

            memo[(capacity, n)] = result
            return result

    value, solution = solve()

    end_time = time.perf_counter()
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)

    recursions_list = list(range(self.recursion_counter))
    axis.plot(recursions_list, memo_size, label='Memo Size')
    axis.set_title('DP (Memoization) - 0/1 Knapsack Problem')

```

```

axis.set_xlabel('Recursive calls')
axis.set_ylabel('Count')
axis.plot(recursions_list, hits_list, label='hits', color="green")

axis.legend(loc='upper left')

#Displays the number of total hits achieved
axis.text(
    0.75, 0.10, 'Total hits: ' + str(self.hits),
    transform=axis.transAxes,
    fontsize=12,
    fontweight="bold",
    va="center",
    ha="center",
)

return value, solution, fig, end_time

def solve_01_knapsack_DP_Bottom_Up(self):
    #Bottom up (Tabulation)
    n = len(self.weights)
    #Rows are items and columns are self.capacity
    #dp[i][c] = best value using items 0...row-1 with self.capacity x
    #therefore columns is capacity and rows is number of items
    dp = [[0] * (self.capacity + 1) for _ in range(n+1)]
    # Fill table bottom-up
    for i in range(1, len(dp)):#i-items
        for c in range(self.capacity + 1):#c-self.capacity
            if self.weights[i-1] <= c:
                include_item = dp[i-1][c-self.weights[i-1]] + self.values[i-
1]#row above shifted along by the weight of the new item
                exclude_item = dp[i-1][c] #row above
                dp[i][c] = max(include_item, exclude_item)
            else:
                dp[i][c] = dp[i-1][c] #Carry over from row above
    max_value = dp[n][self.capacity]

    #Reconstruct best_combination
    i = n
    c = self.capacity
    best_combination = ['0'] * n
    while i != 0 and c != 0:
        if dp[i][c] != dp[i-1][c]:#Different value, therefore item was picked
            best_combination[i-1] = '1'
            c -= self.weights[i-1]
            i -= 1

```

```

        else:
            i -= 1

    end_time = time.perf_counter()

    #Crops data into a labelled table to be displayed on output
    m_data = np.array(dp)
    items_list = np.array(list(range(n + 1)))
    m_data = np.column_stack((items_list, m_data))

    capacity_list = np.array([0] + list(range(self.capacity + 1)))
    m_data = np.vstack((capacity_list, m_data))

    data = m_data[:,250, :250]

    return max_value, ''.join(best_combination), data, end_time

```

The branch and bound code also closely follows the pseudocode and theory explained in the document design. A small implementation detail that is important is that `heapq` defaults to a minheap, however we can simply negate all `ub` and simulate a maxheap as required. Branches are pruned whenever the following condition is satisfied *current_weight > self.capacity or ub <= max_value* and the lowerbound *lb* is used as a potential solution while the upper bound is used as a limit (as may not represent an actual solution). This algorithm satisfies the **1.3** Branch and Bound objective.

```

def solve_01_knapsack_BnB(self):
    n = len(self.weights)
    max_value = 0
    best_solution = '0' * n
    priority_queue = []
    priority_queue_length_list = []
    branches_pruned = 0
    branches_pruned_list = []
    # Compute initial bounds
    ub_root = math.floor(self.solve_fractional_knapsack_G(self.weights,
self.values, self.capacity))
    lb_root, lb_root_solution = self.solve_01_knapsack_GApprox()

    # Priority queue stores (-ub, lb, weight, value, solution, index,
lb_solution)
    # Negate ub to simulate max-heap (heapq defaults to minheap). Therefore those
with a maximum ub are explored first.
    # Algorithm should trap ub and lb in order to find optimal solution
    heapq.heappush(priority_queue, (-ub_root, lb_root, 0, 0, '', 0,
lb_root_solution))
    iterations = 0
    while priority_queue:
        iterations += 1
        priority_queue_length_list.append(len(priority_queue))
        branches_pruned_list.append(branches_pruned)
        #pop the next node from the queue
        neg_ub, lb, current_weight, current_value, current_solution, i,
lb_solution = heapq.heappop(priority_queue)
        ub = -neg_ub

        #End of branch
        if i >= n:
            continue

        #Prune branches
        if current_weight > self.capacity or ub <= max_value:
            branches_pruned += 1
            continue

        #set max_value
        if lb > max_value:
            max_value = lb
            best_solution = current_solution + lb_solution

        #including item
        new_weight = current_weight + self.weights[i]

```

```

        if new_weight <= self.capacity:
            new_value = current_value + self.values[i]
            remaining_capacity = self.capacity - new_weight
            #only lb is an actual solution
            ub_incl = new_value +
math.floor(self.solve_fractional_knapsack_G(self.weights[i+1:], self.values[i+1:],
remaining_capacity))
            lb_incl, lb_incl_solution =
self.solve_01_knapsack_GApprox(self.weights[i+1:], self.values[i+1:],
remaining_capacity)
            lb_incl += new_value
            #only add to the queue to be explored if the ub is greater than the
max value

            if ub_incl > max_value:
                heapq.heappush(priority_queue, (-ub_incl, lb_incl, new_weight,
new_value, current_solution + '1', i + 1, lb_incl_solution))

            #excluding item
            ub_excl = current_value +
math.floor(self.solve_fractional_knapsack_G(self.weights[i+1:], self.values[i+1:],
self.capacity - current_weight))
            lb_excl, lb_excl_solution =
self.solve_01_knapsack_GApprox(self.weights[i+1:], self.values[i+1:], self.capacity -
current_weight)
            lb_excl += current_value
            if ub_excl > max_value:
                heapq.heappush(priority_queue, (-ub_excl, lb_excl, current_weight,
current_value, current_solution + '0', i + 1, lb_excl_solution))

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

iterations_list = list(range(iterations))
axis.plot(iterations_list, priority_queue_length_list, label='Priority Queue
Length')
axis.set_title('Branch and Bound - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Length')

axis2 = axis.twinx()
axis2.set_ylabel('Branches Pruned')
axis2.plot(iterations_list, branches_pruned_list, label='Branches Pruned',
color="green")

# Optional: Add legends for both axes

```

```

lines1, labels1 = axis.get_legend_handles_labels()
lines2, labels2 = axis2.get_legend_handles_labels()
axis.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

return max_value, best_solution, fig, end_time

```

This function has multiple different stages each described in the documented design section under Genetic Algorithm. This includes first generating a random initial population before looping over many iterations and selecting the best solutions (through roulette wheel or tournament selection). Then the population is slightly changed in order to introduce variance between generations. This includes crossing over parts of the population, mutating solutions randomly and directly preserving the very best solutions. From programming the genetic algorithm, I have learned to appreciate the similarities between natural selection and the embedded processes including reproduction, mutation and elitism. The *heapq.nlargest* uses the maxheap specialised data structure in order to quickly select the n largest indexes in a list (equivalent to *sorted(iterable, key=key, reverse=True)[:n]*). This completes my **2.3** Genetic Algorithm objective.

```

def solve_01_knapsack_GeneticAlg(self):
    #variables
    n = len(self.weights)

    isTournamentSelection = True if self.algorithm_data['selection_type_GA'] ==
    "Tournament selection" else False
    tournament_size = self.algorithm_data['tournament_size_GA']

    GENERATIONS = self.algorithm_data['max_iterations_GA']
    population_size = self.algorithm_data['population_size_GA']
    crossover_rate = self.algorithm_data['cross_over_rate']
    mutation_rate = self.algorithm_data['mutation_rate']
    elitism_rate = self.algorithm_data['elitism_rate']

    elitism_number = int(elitism_rate * population_size)
    global_best_combination = '0' * n
    global_max_value = 0

    population = [''] * population_size
    fitnesses = [0] * population_size
    #Graph lists
    average_fitnesses = [0] * GENERATIONS
    best_fitnesses = [0] * GENERATIONS
    lowest_fitnesses = [0] * GENERATIONS
    #Generate random population to begin
    for x in range(population_size):
        value, binary_string, weight =
self.generate_starting_solution(is_greedy=False)
        population[x] = binary_string
        fitnesses[x] = value

    for i in range(GENERATIONS):
        new_population = [''] * population_size
        if isTournamentSelection:
            #Tournament Selection (else roulette wheel selection)
            for x in range(population_size):
                binary_string = '0' * n
                max_value = 0
                for y in range(tournament_size):
                    index = random.randint(0, population_size - 1)
                    if fitnesses[index] > max_value:
                        binary_string = population[index]
                        max_value = fitnesses[index]
                new_population[x] = binary_string
        else:
            total_fitness = sum(fitnesses)

```

```

        for x in range(population_size):
            random_variable = random.random() * total_fitness
            cumulative_sum = 0
            solution_index = 0
            for p in range(population_size):
                cumulative_sum += fitnesses[p]
                if random_variable <= cumulative_sum:
                    solution_index = p
                    break
            new_population[x] = population[solution_index]

#Cross over certain pairs of the population
for x in range(0, population_size-1, 2):
    cross_over_random_variable = random.random()
    if cross_over_random_variable > crossover_rate: continue
    crossover_point = random.randint(0, n-1)

    new_population[x], new_population[x+1] = new_population[x]
[:crossover_point] + new_population[x+1][crossover_point:], new_population[x+1]
[:crossover_point] + new_population[x][crossover_point:]

#Mutate to introduce variance
for x in range(population_size):
    for mutation_bit in range(n):
        mutate_random_variable = random.random()
        if mutate_random_variable > mutation_rate: continue
        new_population[x] = (
            new_population[x][:mutation_bit] +
            ('1' if new_population[x][mutation_bit] == '0' else '0') +
            new_population[x][mutation_bit+1:]
        )

#Elitism
#Uses heap data structure to efficiently find the n largest item indexes
indexes = heapq.nlargest(elitism_number, range(len(fitnesses)),
key=fitnesses.__getitem__)
for x in range(elitism_number):
    swapping_index = random.randint(0, population_size-1)
    new_population[swapping_index] = population[indexes[x]]

average_fitnesses[i] = sum(fitnesses) / population_size
best_fitnesses[i] = max(fitnesses)
if best_fitnesses[i] > global_max_value:
    global_max_value = best_fitnesses[i]
    global_best_combination =
population[fitnesses.index(global_max_value)]

```

```

        lowest_fitnesses[i] = min(x for x in fitnesses if x > 0)

    population = new_population
    fitnesses = list(map(self.evaluate, population))

    end_time = time.perf_counter()
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)

    generations = list(range(GENERATIONS))

    #Plot average, best and minimum fitness all on same graph
    axis.plot(generations, average_fitnesses, marker='o', label='Average
Fitness')
    axis.plot(generations, best_fitnesses, marker='o', label='Highest Fitness')
    axis.plot(generations, lowest_fitnesses, marker='o', label='Minimum Fitness')

    axis.set_title('Genetic Algorithm - 0/1 Knapsack Problem')
    axis.set_xlabel('Iterations')
    axis.set_ylabel('Fitness')
    axis.legend()

    return global_max_value, global_best_combination, fig, end_time

```

Below is the most naive approach of simply generating random solutions and then using the evaluation function to explore the search space. This process will repeat for a given *RUNTIME*, however this approach quickly fails with datasets with a low capacity load (only a few items can be included) or with a very large number of items. This function underscores the **2.1** Randomised Search objective.

```

def solve_01_knapsack_random(self):
    n = len(self.weights)
    RUNTIME = self.algorithm_data['time_limit']
    time_interval = min(0.001, RUNTIME / 100)
    start_time = last_time = time.time()
    best_combination = '0' * n
    max_value = 0

    best_values = []
    current_values = []
    times_list = []
    counter = 0
    while time.time() < start_time + RUNTIME:
        #Generates a random binary string of fixed length n
        binary_string = bin(random.randint(0, (2**n)-1))[2:].zfill(n)

        value = self.evaluate(binary_string)
        if value > max_value:
            best_combination = binary_string
            max_value = value

        if time.time() - last_time >= time_interval:
            last_time = time.time()
            best_values.append(max_value)
            current_values.append(value)
            times_list.append(last_time - start_time)

    end_time = time.perf_counter()
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)

    axis.plot(times_list, best_values, marker='o', label='Best Value')
    axis.plot(times_list, current_values, label='Current Value', alpha=0.5,
color='red')

    axis.set_title('Randomised Search - 0/1 Knapsack Problem')
    axis.set_xlabel('Time')
    axis.set_ylabel('Value')
    axis.legend()

    return max_value, best_combination, fig, end_time

```

SA (Simulated Annealing) iterates over a random solution, however can escape local optima due to the degree of randomness when selecting a worse solution. This is given by the very important $acceptance_probability = \mathit{math.exp}(-\mathit{delta} / \mathit{temperature})$ line which describes the probability by which

a worse solution will be accepted. The graph shows the standard current value and best value as seen with the brute force iterative approach however also includes the temperature curve to demonstrate the cooling schedule. This function successfully achieves the **2.5** Simulated Annealing objective.

```

def solve_01_knapsack_SA(self):
    n = len(self.weights)
    value, solution, weight =
self.generate_starting_solution(self.algorithm_data['is_greedy_SA'])

    max_value = value
    best_solution = solution

    initial_temperature = self.algorithm_data['initial_temperature']
    minimum_temperature = self.algorithm_data['minimum_temperature']
    alpha = self.algorithm_data['alpha']

    iteration = 0
    max_iterations = self.algorithm_data['max_iterations_SA']

    #worse = 0
    #accepted = 0

    range_of_data = max(self.values) - min(self.values)

    temperature = initial_temperature

    temperature_list = []
    best_values = []
    current_values = []

    while temperature >= minimum_temperature and iteration < max_iterations:
        temperature_list.append(temperature)
        Found = False
        while Found == False:
            random_bit = random.randint(0, n-1)
            if solution[random_bit] == '1':
                new_solution = solution[:random_bit] + ('0') +
solution[random_bit + 1:]
                new_value = value - self.values[random_bit]
                new_weight = weight - self.weights[random_bit]
                Found = True
            else:
                new_solution = solution[:random_bit] + ('1') +
solution[random_bit + 1:]
                new_value = value + self.values[random_bit]
                new_weight = weight + self.weights[random_bit]

            if new_weight <= self.capacity:
                Found = True

```

```

        if new_value >= value:
            value = new_value
            weight = new_weight
            solution = new_solution

        if new_value > max_value:
            max_value = new_value
            best_solution = new_solution
    else:
        #Probability formula
        #Delta will normalise the range as input data may vary in terms of
        'Range of values:' variable
        delta = (value - new_value) / range_of_data #relative positive
        'worse' difference
        acceptance_probability = math.exp(-delta / temperature)
        #print("Difference: " + str(value-new_value) + "acceptance
probability: " + str(acceptance_probability))
        rand_var = random.random()

        #worse += 1
        if rand_var < acceptance_probability:
            value = new_value
            weight = new_weight
            solution = new_solution
            #accepted += 1

    best_values.append(max_value)
    current_values.append(value)

    temperature *= alpha
    iteration += 1
    #print(accepted, worse, accepted/worse, iteration, temperature, value,
max_value)

    # fig will include cooling schedule against iterations, current_solution and
best_solution, greedy if there
    end_time = time.perf_counter()
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)
    iterations_list = list(range(iteration))

    axis.set_title('Simulated Annealing - 0/1 Knapsack Problem')
    axis.set_xlabel('Iterations')
    axis.set_ylabel('Value')
    axis.plot(iterations_list, best_values, marker='o', label='Best Value')
    axis.plot(iterations_list, current_values, label='Current Value', alpha=0.5,

```

```

color='grey')

    axis2 = axis.twinx()
    axis2.set_ylabel('Temperature')
    axis2.plot(iterations_list, temperature_list, label='Temperature',
color="red")

    #Add legends for both axes
    lines1, labels1 = axis.get_legend_handles_labels()
    lines2, labels2 = axis2.get_legend_handles_labels()
    axis.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

    return max_value, best_solution, fig, end_time

```

Tabu search is closely related to simulated annealing so follows the same code structure. To implement the tabu_list I just used a normal python list data structure and used a dynamic tabu tenure, which adjusts the length of the tabu list based on the search process. The algorithm runs correctly and thereby completes the **2.6** Tabu Search objective. There are much more complex and hybrid tabu search algorithms, which shows this algorithm has the potential to grow much further.

```

def solve_01_knapsack_Tabu(self):
    n = len(self.weights)
    value, solution, weight =
self.generate_starting_solution(self.algorithm_data['is_greedy_tabu'])

    max_value = value
    best_solution = solution

    tabu_list = []
    tabu_list_maxsize = self.algorithm_data['tabu_list_size']
    ITERATIONS = self.algorithm_data['max_iterations_tabu_search']

    best_values = []
    current_values = []
    tabu_length_list = []

    for y in range(ITERATIONS):
        #Efficiently finding next move/neighbour by evaluating possible new
solutions and selecting the best non-tabu solution
        best_neighbour = ''
        best_neighbour_weight = 0
        best_neighbour_value = 0
        worse = False
        for x in range(n):
            #There are n neighbours
            if solution[x] == '0':
                #We flip to 1
                if weight + self.weights[x] <= self.capacity:
                    new_neighbour_value = value + self.values[x]
                    new_neighbour = solution[:x] + '1' + solution[x+1:]
                    new_neighbour_weight = weight + self.weights[x]
                    if new_neighbour_value > best_neighbour_value and
int(new_neighbour, 2) not in tabu_list:
                        best_neighbour_value = new_neighbour_value
                        best_neighbour = new_neighbour
                        best_neighbour_weight = new_neighbour_weight
            elif solution[x] == '1':
                #We flip to 0
                new_neighbour_value = value - self.values[x]
                new_neighbour = solution[:x] + '0' + solution[x+1:]
                new_neighbour_weight = weight - self.weights[x]
                if new_neighbour_value > best_neighbour_value and
int(new_neighbour, 2) not in tabu_list:
                    best_neighbour_value = new_neighbour_value
                    best_neighbour = new_neighbour
                    best_neighbour_weight = new_neighbour_weight

```

```

        worse = True

    if best_neighbour_value > max_value:
        max_value = best_neighbour_value
        best_solution = best_neighbour
        if worse: print("accepted worse " + str(y))

    value = best_neighbour_value
    solution = best_neighbour
    weight = best_neighbour_weight

    tabu_list.append(int(solution, 2))

    while len(tabu_list) > tabu_list_maxsize:
        tabu_list.pop(0)

    best_values.append(max_value)
    current_values.append(value)
    tabu_length_list.append(len(tabu_list))

end_time = time.perf_counter()
#In the graph for tabu search we will include the length of the tabu list,
the max value, the current value as they are the relevant properties
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)
iterations_list = list(range(ITERATIONS))
axis.set_title('Tabu Search - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')
axis.plot(iterations_list, best_values, marker='o', label='Best Value')
axis.plot(iterations_list, current_values, label='Current Value', alpha=0.5,
color='grey')

axis2 = axis.twinx()
axis2.set_ylabel('Length')
axis2.plot(iterations_list, tabu_length_list, label='Length of Tabu List',
color="green")

# Optional: Add legends for both axes
lines1, labels1 = axis.get_legend_handles_labels()
lines2, labels2 = axis2.get_legend_handles_labels()
axis.legend(lines1 + lines2, labels1 + labels2, loc='lower right')

return max_value, best_solution, fig, end_time

```

I developed ant colony optimisation through again applying the formulation uncovered in the documented design under ACO. This includes implementing a feasible set of items from which an ant can choose from, as well as generating probabilities for each item based upon attractiveness and pheromone strength. At the end of each iteration pheromones are evaporated by decreasing by a fixed rate. I thought it would be interesting to show on the graph each individual pheromone's growth rate, so that the user could see that they are accumulating correctly. This satisfies my **2.4** Ant Colony Optimisation objective.

```

def solve_01_knapsack_ACO(self):
    n = len(self.weights)
    ITERATIONS = self.algorithm_data['max_iterations_ACO']
    ALPHA = self.algorithm_data['alpha_ACO']
    BETA = self.algorithm_data['beta_ACO']
    number_of_ants = self.algorithm_data['ants']
    evaporation_rate = self.algorithm_data['evaporation_rate']

    global_best_combination = ['0'] * n
    max_value = 0

    attractiveness = [self.values[i] / (self.weights[i]) for i in range(n)]

    pheromones = [0.01] * n

    best_value_list = []
    pheromones_storage_list = []

    for x in range(ITERATIONS):
        #Generate probabilities for each iteration
        probabilities = [0] * n
        for i in range(n):
            probabilities[i] = (pheromones[i]**ALPHA) * (attractiveness[i]**BETA)

        #Generate solution for each ant, and update pheromones for next iteration
        for y in range(number_of_ants):
            binary_string = ['0'] * n
            current_weight = 0
            current_value = 0

            feasible_set = list(range(n))

            while feasible_set:
                feasible_set = [i for i in feasible_set if current_weight +
self.weights[i] <= self.capacity]
                if not feasible_set: break

                feasible_probabilities = [probabilities[i] for i in feasible_set]
                total = sum(feasible_probabilities)

                if total == 0: break

                Random_Variable = random.random() * total
                cumulative_sum = 0
                item_index = 0

```

```

        for index, p in zip(feasible_set, feasible_probabilities):
            cumulative_sum += p
            if Random_Variable <= cumulative_sum:
                item_index = index
                break

        binary_string[item_index] = '1'
        current_weight += self.weights[item_index]
        current_value += self.values[item_index]
        feasible_set.remove(item_index)

    if current_value > max_value:
        global_best_combination = binary_string[:]
        max_value = current_value

    try:
        #Update pheromones
        pheromone_strength = 1 / (1 + ((max_value - current_value) /
max_value))
    except:
        continue

    for w in range(n):
        if binary_string[w] == '1':
            pheromones[w] += pheromone_strength

    best_value_list.append(max_value)
    pheromones_storage_list.append(pheromones)

    #Evapouration
    pheromones = [evapouration_rate * pheromone for pheromone in pheromones]

end_time = time.perf_counter()
#Graph for ACO will have the following components:
#Pheromone Strength, Attractiveness, probabilities, best value
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

iterations_list = list(range(ITERATIONS))

axis.plot(iterations_list, best_value_list, marker='o', label='Best Value')

axis.set_title('Ant Colony Optimisation - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')

```

```

axis2 = axis.twinx()
axis2.set_ylabel('Pheromone Strength')
for x in range(n):
    axis2.plot(iterations_list, [pheromones_storage_list[y][x] for y in
range(ITERATIONS)], alpha=0.7)

# Create a proxy artist for the legend
multi_coloured_proxy = Line2D([0], [0], color='black', linewidth=2)
# Add legend manually
lines1, labels1 = axis.get_legend_handles_labels()
axis.legend(lines1 + [multi_coloured_proxy], labels1 + ["(Multicoloured)
Pheromones"], loc="upper left")

return max_value, "".join(global_best_combination), fig, end_time

```

Particle swarm optimisation is a heavily mathematical algorithm and was also underlined in the document design section under BPSO. In this implementation I used integer lists due to the increased efficiency over a binary string when repeatedly updating position, velocity, pbest and gbest vectors. The repair function helps refine this algorithm by ensuring solutions are normalised in a way that achieves their maximum value. The most important equation described below is the velocity update equation $particle.velocity[x] = (inertia * particle.velocity[x]) + (cognitive_factor * random.random() * ((particle.pbest[x] - (particle.position[x]))) + (social_factor * random.random() * ((gbest[x] - (particle.position[x])))$ which describes how the particles' velocity vector changes over time as they converge on a global best solution. This marks the **2.7** Particle Swarm Optimisation objective.

```

def solve_01_knapsack_PS0(self):
    def repair(solution, greedy_ordering):
        #ADD phase
        total_weight = sum([self.weights[x] for x in range(len(self.weights)) if
solution[x] == 1])
        gap = self.capacity - total_weight
        i = 0
        new_solution = list(solution).copy()

        while gap > 0 and i < n:
            index = greedy_ordering[i]
            if solution[index] == 0 and gap >= self.weights[index]:
                new_solution[index] = 1
                gap -= self.weights[index]
                total_weight += self.weights[index]
                i += 1

        #DROP phase
        over = total_weight - self.capacity
        i = 0
        while over > 0 and i < n:
            index = greedy_ordering[n-1-i]
            if solution[index] == 1:
                new_solution[index] = 0
                over -= self.weights[index]
                i += 1

        return new_solution

    def evaluate_PS0(integer_list):
        #uses integer lists
        total_weight = sum([self.weights[x] for x in range(len(self.weights)) if
integer_list[x] == 1])
        total_value = sum([self.values[x] for x in range(len(self.values)) if
integer_list[x] == 1])
        return total_value if total_weight <= self.capacity else 0

    #Uses list of integers until very end
    n = len(self.weights)
    max_iterations = self.algorithm_data['max_iterations_PS0']
    population = self.algorithm_data['population_size_PS0']
    inertia = self.algorithm_data['inertia']
    cognitive_factor = self.algorithm_data['cognitive_factor']
    social_factor = self.algorithm_data['social_factor']
    steepness = self.algorithm_data['steepness']
    max_velocity = self.algorithm_data['max_velocity']

```

```

best_value_list = []

swarm = [Particle(n) for _ in range(population)]
iteration = 0
gbest = []
gbest_fitness = 0

#Compute greedy ordering
items = [None] * n
for x in range(n):
    items[x] = (self.values[x]/self.weights[x], x)
items = sorted(items, reverse=True)
greedy_ordering = [b for (a, b) in items]

while iteration < max_iterations:
    for particle in swarm:
        particle.position = repair(particle.position, greedy_ordering).copy()
        new_fitness = evaluate_PSO(particle.position)

        if new_fitness > particle.pbest_fitness:
            particle.pbest = particle.position[:]
            particle.pbest_fitness = new_fitness

        if particle.pbest_fitness > gbest_fitness:
            gbest = particle.pbest[:]
            gbest_fitness = particle.pbest_fitness

        particle.pbest_list.append(particle.pbest_fitness)

    for particle in swarm:
        for x in range(n):
            #UPDATE VELOCITY
            particle.velocity[x] = (inertia * particle.velocity[x]) +
(cognitive_factor * random.random() * ((particle.pbest[x]) - (particle.position[x])))
+ (social_factor * random.random() * ((gbest[x]) - (particle.position[x])))
            particle.velocity[x] = max(min(particle.velocity[x],
max_velocity), -max_velocity)
            #print(particle.velocity[x])
            #UPDATE POSITION
            if random.random() < (1 / (1 + math.exp(-steepness *
particle.velocity[x]))):
                particle.position[x] = 1
            else:
                particle.position[x] = 0

        particle.velocity_list.append(sum(particle.velocity) /

```

```

len(particle.velocity))

        best_value_list.append(gbest_fitness)
        iteration += 1

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

iterations_list = list(range(max_iterations))

axis.plot(iterations_list, best_value_list, marker='o', label='Best Value')

for particle in swarm:
    axis.plot(iterations_list, particle.pbest_list, color='red', alpha=0.5,
marker='s', markersize=1, label='Personal Best Value')

axis.set_title('Particle Swarm Optimisation - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')

axis2 = axis.twinx()
axis2.set_ylabel('Velocity')
for particle in swarm:
    axis2.plot(iterations_list, particle.velocity_list, color='green',
label='Average Velocity')

# Get handles and labels from both axes
lines1, labels1 = axis.get_legend_handles_labels()
lines2, labels2 = axis2.get_legend_handles_labels()

# Merge them, removing duplicates by label
by_label = dict(zip(labels1 + labels2, lines1 + lines2))

axis.legend(by_label.values(), by_label.keys(), loc='center right')

return gbest_fitness, "".join(map(str, gbest)), fig, end_time

```

Below shows 2 generic common functions including *evaluate()* which returns the total value (and weight) of a solution implemented from documented design and *generate_starting_solution()* which is called by simulated annealing, tabu search and the genetic algorithm etc. in order to get a random or greedy initial solution. These functions are called significantly and so make my code much more compact.

```

def evaluate(self, binary_string, get_total_weight=False):
    total_value = sum([self.values[x] for x in range(len(self.values)) if
binary_string[x] == "1"])
    total_weight = sum([self.weights[x] for x in range(len(self.weights)) if
binary_string[x] == "1"])

    if total_weight > self.capacity:
        total_value = 0

    if get_total_weight:
        return total_value, total_weight
    else:
        return total_value

def generate_starting_solution(self, is_greedy):
    if is_greedy:
        value, solution, weight =
self.solve_01_knapsack_GApprox(get_total_weight=True)
    else:
        n = len(self.weights)
        solution = bin(random.randint(0, (2**n)-1))[2:].zfill(n)
        value, weight = self.evaluate(solution, get_total_weight=True)
        while value == 0:
            solution = bin(random.randint(0, (2**n)-1))[2:].zfill(n)
            value, weight = self.evaluate(solution, get_total_weight=True)

    return value, solution, weight

```

Here are the global functions held outside any class, and imported by [index.py](#). The *time_and_run_binaryKP* function is associated with the **9.1** Comparative Data objective while the *generate_input_data_binaryKP* is associated with the **6.1** Generate Random objective. The first function simply calls and times the selected algorithm based on the *function_index* variable, while the second function produces random data for the problem as discussed in Approach 2 in the 'Generating input data' documented design section.

```

#Function called by index.py
def time_and_run_binaryKP(function_index, weights, values, capacity, algorithm_data):
    problem = BinaryKP(weights, values, capacity, algorithm_data)
    functions = [problem.solve_01_knapsack_random,
problem.solve_01_knapsack_BF_iterative, problem.solve_01_knapsack_BF_recursive,
problem.solve_01_knapsack_GApprox, problem.solve_01_knapsack_DP_Top_Down,
problem.solve_01_knapsack_DP_Bottom_Up, problem.solve_01_knapsack_BnB,
problem.solve_01_knapsack_GeneticAlg, problem.solve_01_knapsack_ACO,
problem.solve_01_knapsack_SA, problem.solve_01_knapsack_Tabu,
problem.solve_01_knapsack_PSO]
    start_time = time.perf_counter()
    function = functions[function_index]
    if function == problem.solve_01_knapsack_GApprox:
        Answer = problem.solve_01_knapsack_GApprox(generate_graph=True)
    else:
        Answer = function()
    end_time = Answer[-1]
    elapsed_time = end_time - start_time

    fig = None
    if len(Answer) > 3:
        #There is a figure
        fig = Answer[2]

    solution_value, solution_weight = problem.evaluate(Answer[1],
get_total_weight=True)
    return Answer[0], solution_weight, Answer[1], elapsed_time, Answer[0] ==
solution_value, fig

#Function called by index.py
def generate_input_data_binaryKP(n, range_of_values, correlation_strength,
capacity_load):
    #request.form['number_of_items'], request.form['range_of_values'],
request.form['correlation_strength'], request.form['capacity_load']
    #Input data parameters
    weights = [0] * n
    values = [0] * n
    capacity = 0

    index = 0

    for index in range(n):
        weight = random.randint(1, range_of_values)
        weights[index] = weight

        if correlation_strength == 0:

```

```

        value = random.randint(1, range_of_values)
    else:
        value = max(1, min(range_of_values,
random.randint(int(correlation_strength * weight), int((2 - correlation_strength) *
weight))))
        #When correlation strength is 0 between 1 and 2W (still weakly
constrained)
        values[index] = value

    capacity = max(1, min(sum(weights), int(capacity_load * sum(weights))))

    return weights, values, capacity

```

3 - index.py

This program is the main application script. It contains the Flask application instance/entry point, route definitions, and the logic to start the development server. This therefore directs the workflow and connects the backend to the frontend. I use dictionaries to group data together based upon when they are processed. It also raises any input/user errors here, preventing them from interrupting/crashing when being run and achieving **3.1** Manual Input Error & **3.2** CSV Input Error.

The *index()* is the entry point for the home page (index.html) and is initially loaded when you access the website. It is divided into 2 sub-sections, the 'POST' method used for loading data into the interface and the 'GET' method that simply returns the index.html render template with the session's values.

```

#Imported libraries
from flask import *
from matplotlib.backends.backend_agg import FigureCanvasAgg as FigureCanvas
import pandas as pd
import math
import ast
import io
import base64
import numpy
import json

app = Flask(__name__)

app.secret_key = "AARAV_AND_ISHA" # Needed for part of functionality

#Import functions from Algorithms.py (encapsulation) [Relative import for vercel]
from .Algorithms import time_and_run_binaryKP
from .Algorithms import generate_input_data_binaryKP

@app.route("/", methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        #Generate data/submit data/load data
        #redirect(url_for('index')) redirets the user back to the homepage (get
method)

        input_data = request.form['input_data']
        if input_data == "generate_random":
            #Calls random data function
            weights, values, capacity =
generate_input_data_binaryKP(int(request.form['number_of_items']),
int(request.form['range_of_values']), float(request.form['correlation_strength']),
float(request.form['capacity_load']))
            elif input_data == "input_manually":
                try:
                    #Strips the input, splits the data by " ", and converts to integer
lists for the 6.2 Input Manually objective
                    weights = list(map(int, request.form['weights'].strip().split()))
                    values = list(map(int, request.form['values'].strip().split()))
                    capacity = int(request.form['capacity'].strip())
                except:
                    session['error'] = "Please enter integers seperated by spaces for
weights and values and a single integer for capacity."
                    return redirect(url_for('index'))
            elif input_data == "import_csv":
                #Gets CSV file, 6.3 Import CSV objective
                file = request.files.get("csv_file")

```

```

if file and file.filename.endswith(".csv"):
    #Reads CSV file using pandas. Calls any error before converting
    df = pd.read_csv(file)
    #Check file contains individual columns
    if 'weights' not in df.columns:
        session['error'] = "Column 'weights' not found in CSV."
        return redirect(url_for('index'))
    elif 'values' not in df.columns:
        session['error'] = "Column 'values' not found in CSV."
        return redirect(url_for('index'))
    elif 'capacity' not in df.columns:
        session['error'] = "Column 'capacity' not found in CSV."
        return redirect(url_for('index'))
else:
    #File type is not CSV file
    session['error'] = "Not CSV File."
    return redirect(url_for('index'))

try:
    weights = df['weights'].tolist()
    values = df['values'].tolist()
    capacity_list = df['capacity'].tolist()
    #Cleans capacity list so that actual value can be identified
    cleaned_list = [x for x in capacity_list if not math.isnan(x)]
except:
    session['error'] = "Column 'weights', 'values' and 'capacity' all
must contain integer values."
    return redirect(url_for('index'))

if len(cleaned_list) == 0:
    #No value found in capacity
    session['error'] = "No valid (non-NaN) values found in 'capacity'."
    return redirect(url_for('index'))

capacity = int(cleaned_list[0])

if len(weights) != len(values):
    session['error'] = "The number of weights inputted doesn't match the
number of values."
    return redirect(url_for('index'))

# Store results in session or temp storage
session['weights'] = weights
session['values'] = values
session['capacity'] = capacity

```

```

    session['number_of_items'] = request.form.get('number_of_items', 450)
    session['range_of_values'] = request.form.get('range_of_values', 200)
    session['correlation_strength'] = request.form.get('correlation_strength',
0.6)

    session['capacity_load'] = request.form.get('capacity_load', 0.75)

    # Redirect to GET route
    return redirect(url_for('index'))

# GET request: load stored data or show empty
weights = session.pop('weights', [])
values = session.pop('values', [])
capacity = session.pop('capacity', 0)
error = session.pop('error', "")

number_of_items = session.pop('number_of_items', 450)
range_of_values = session.pop('range_of_values', 200)
correlation_strength = session.pop('correlation_strength', 0.6)
capacity_load = session.pop('capacity_load', 0.75)

#Returns the index.html render template with the newly generated weights, value
and capacity
#Additionally stores the generate random data parameters if new data with same
parameters needs to be generated
    return render_template(
        "index.html",
        weights=weights,
        values=values,
        capacity=capacity,
        number_of_items=number_of_items,
        range_of_values=range_of_values,
        correlation_strength=correlation_strength,
        capacity_load=capacity_load,
        error=error
    )

```

Below is the *submit()* method which is the entry point for the result page (submit.html). It identifies the algorithm, collates the parameters, calls the function and then returns the render template with the returned values passed in. It will also importantly convert the *matplotlib.figure* into a png and encode the value (for the **8.2** Plotted Graph), as well as converting the numpy array into json (for the **8.3** DP Tabulation Table).

```

@app.route('/submit', methods=['POST'])
def submit():
    #.form[''] returns the value of the option from dropdown
    algorithm_type = request.form['algorithm_type']
    function_names = ['Randomised Search', 'Brute Force (iterative)', 'Brute Force
(recursive)', 'Greedy Approximation', 'DP (Memoization)', 'DP (Tabulation)', 'Branch
and Bound', 'Genetic Algorithm', 'Ant Colony Optimisation', 'Simulated Annealing',
'Tabu Search', 'Particle Swarm Optimisation']
    #Gets the index of the algorithm selected
    function_index = function_names.index(algorithm_type)
    #Efficiently converts string of weights and values into an actual python stored
integer list
    weights = ast.literal_eval(request.form['weights'])
    values = ast.literal_eval(request.form['values'])
    capacity = int(request.form['capacity'])

    if len(weights) == 0:
        #No data submitted
        return render_template('index.html', weights=[], values=[], capacity=0,
error="No data value submitted")

    #Dictionary of all algorithm parameters. All are grouped together, however only
the selected algorithm's parameters will be used by Algorithm.py
    algorithm_data = {
        "time_limit" : int(request.form['time_limit']),
        "max_iterations_iterative" : int(request.form['max_iterations_iterative']),
        "max_iterations_recursive" : int(request.form['max_iterations_recursive']),
        "max_iterations_tabu_search" :
int(request.form['max_iterations_tabu_search']),
        "tabu_list_size" : int(request.form['tabu_list_size']),
        "initial_temperature" : float(request.form['initial_temperature']),
        "minimum_temperature" : float(request.form['minimum_temperature']),
        "alpha" : float(request.form['alpha']),
        "max_iterations_SA" : int(request.form['max_iterations_SA']),
        "max_iterations_ACO" : int(request.form['max_iterations_ACO']),
        "ants" : int(request.form['ants']),
        "alpha_ACO" : int(request.form['alpha_ACO']),
        "beta_ACO" : int(request.form['beta_ACO']),
        "evapouration_rate" : float(request.form['evapouration_rate']),
        "max_iterations_GA" : int(request.form['max_iterations_GA']),
        "population_size_GA" : int(request.form['population_size_GA']),
        "selection_type_GA" : request.form['selection_type_GA'],
        "tournament_size_GA" : int(request.form['tournament_size_GA']),
        "cross_over_rate" : float(request.form['cross_over_rate']),
        "mutation_rate" : float(request.form['mutation_rate']),
        "elitism_rate" : float(request.form['elitism_rate']),

```

```

    "max_iterations_PSO" : int(request.form['max_iterations_PSO']),
    "population_size_PSO" : int(request.form['population_size_PSO']),
    "inertia" : float(request.form['inertia']),
    "cognitive_factor" : float(request.form['cognitive_factor']),
    "social_factor" : float(request.form['social_factor']),
    "steepness" : float(request.form['steepness']),
    "max_velocity" : float(request.form['max_velocity']),
    "is_greedy_SA" : True if "is_greedy_SA" in request.form.keys() else False,
    "is_greedy_tabu" : True if "is_greedy_tabu" in request.form.keys() else False
}

#Processes the forms data
#Calls algorithm on the loaded data
generated_answer, total_weight, binary_string, elapsed_time, valid, fig =
time_and_run_binaryKP(function_index, weights, values, capacity, algorithm_data)
if type(fig) == type(numpy.array([0])):
    #If fig is from DP Tabulation, convert to json
    #8.3 DP Tabulation Table objective
    l_fig = fig.tolist()
    r_fig = json.dumps(l_fig)
else:
    #If fig is a plotted graph, convert using base64 into an image that can be
    embedded in the html
    #8.2 Plotted Graph objective
    output = io.BytesIO()
    FigureCanvas(fig).print_png(output)
    r_fig = base64.b64encode(output.getvalue()).decode('utf-8')

#Dictionary of grouped comparative data that needs to be passed into submit.html
Data = {
    "approach" : function_names[function_index],
    "total_value" : generated_answer,
    "total_weight" : total_weight,
    "time" : elapsed_time,
    "valid" : valid,
    "binary_string" : binary_string,
    "weights" : weights,
    "values" : values,
    "r_fig" : r_fig
}

return render_template("submit.html", Data=Data)

#app.run(debug=True) – This is used in testing not in deployed version on vercel

```

4 - base.html

This represents the root html file which both index.html and submit.html derive from laying out the essential book keeping and reducing repetitiveness.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!--Fixed attributes across index and submit html files-->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static', filename = 'css/main.css')}}">
    <link rel="icon" type="image/x-icon" href="{{ url_for('static',
filename='images/KnapSackTitleImage.png') }}">
    {% block head %}{% endblock %}
</head>
<body>
    {% block body %}{% endblock %}
</body>
</html>
```

This code uses Jinja2 syntax in order to create code blocks which can be inherited from. This will avoid repeated redundant code at the start of each individual html file. In the header it shows the html file connected to the stylesheet (main.css) and the fixed icon that is displayed on the tab. This file is sectioned into the block head and block body for the inherited html files (index.html and submit.html) to insert code.

5 - index.html

This is the page that the user first interacts with to input the parameters of the algorithms and data. There are several repeated interactive elements due to the number of different algorithms available to choose from. There is also a brief explanation/description of each algorithm and problem in order to guide less familiar users. Most of the code is quite repetitive and vanilla due to the amount of algorithms and their associated parameters. Where objectives have been completed, I have marked it from within the comments below.

There are 2 forms: the '*algorithm form*' which when submitted is used to run the algorithm with their parameters, and the '*data form*' which when run generates/submits data to be loaded into the software. I have split the html below between these forms.

```

{% extends 'base.html' %}

{% block head %}
<title>Knapsack Algorithm Learner Tool</title>

<script>
//Math notation should be displayed on the left of the display and connect MathJax
notation for Latex rendering
window.MathJax = {
  chtml: {
    displayAlign: "left",
    displayIndent: "0"
  }
};
</script>
<script src="https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-chtml.js"></script>

{% endblock %}

{% block body %}

<div id="title">
  <!--Title and Title Image-->
  
  <h1>A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the
Knapsack Problem</h1>
</div>
<div class="row">
  <div class="column1">
    <div class="algorithm form">
      <form action="/submit" method="post" id="algorithm form"
onsubmit="show_text()">
        <div>
          <div id="problem_description_01">
            <!--0/1 Optimisation version of the knapsack problem 4.1
Problem Description objective -->
            <strong>Problem Description: </strong>
            <p id="problem_description">Given a set of 'n' items where
each item 'i' has a weight and a value (w<sub>i</sub>, v<sub>i</sub>) and a knapsack
with a maximum capacity 'W', what subset of items maximises the total value of the
knapsack without exceeding it's weight limit.</p>
            <p id="math_notation">
              \[
              \begin{aligned}
&\text{Maximize} \quad \sum_{i=1}^n v_i x_i \\
&\text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W,
\end{aligned}

```

```

&& x_i \in \{0,1\}, \quad i = 1, \dots, n
\end{aligned}
\]
</p>
</div>
</div>
<strong>Algorithm: </strong>
<br>
<label for="algorithm_type">Choose an algorithm:</label>
<select name="algorithm_type" id="algorithm_type">
  <!--All the available algorithms loosely ordered by exact ->
  heuristic-->
    <option value="Randomised Search">Randomised Search</option>
    <option value="Brute Force (iterative)">Brute Force (iterative)
</option>
    <option value="Brute Force (recursive)">Brute Force (recursive)
</option>
    <option value="Greedy Approximation">Greedy
Approximation</option>
    <option value="DP (Memoization)">DP (Memoization)</option>
    <option value="DP (Tabulation)">DP (Tabulation)</option>
    <option value="Branch and Bound">Branch and Bound</option>
    <option value="Genetic Algorithm">Genetic Algorithm</option>
    <option value="Ant Colony Optimisation">Ant Colony
Optimisation</option>
    <option value="Simulated Annealing">Simulated Annealing</option>
    <option value="Tabu Search">Tabu Search</option>
    <option value="Particle Swarm Optimisation">Particle Swarm
Optimisation</option>
  </select>
  <br>
  <div id="0/1algorithms">
    <!--All algorithm descriptions 4.2 Algorithm Descriptions
    objective-->
      <!--Randomised Search-->
      <div class="algorithm" id="Randomised Search">
        <p class="algorithm-description">Uniformly generates random
solutions over a given time period, and stores the best valid solution.</p>
        <!--Example of 5.1 Slider Visual-->
        <div class="variable">
          <label for="time_limit">Search time limit: </label>
          <div class="slidecontainer">
            <input type="range" min="1" max="30" value="2"
class="slider" name="time_limit">
          </div>
          <input type="number" class="slidevalue" min="1" max="30"

```

```

value="2">
    </div>
</div>
<!--Brute Force Iterative-->
<div class="algorithm" id="Brute Force (iterative)"
style="display:none">
    <p class="algorithm-description">Loops in binary from 0 to
2<sup>n-1</sup> (over all possible individual subsets) and stores the best valid
solution.</p>
    <p class="time-complexity">Time Complexity:  $(2^n)$ </p>
    <div class="variable">
        <label for="max_iterations_iterative">Max iterations:
</label>
        <div class="slidecontainer">
            <input type="range" min="100" max="25000000"
value="200000" class="slider" name="max_iterations_iterative">
            </div>
            <input type="number" class="slidevalue" min="100"
max="25000000" value="200000">
        </div>
    </div>
<!--Brute Force Recursive-->
<div class="algorithm" id="Brute Force (recursive)"
style="display:none">
    <p class="algorithm-description">Recursively solves the
problem, by splitting it into smaller sub-problems, using the recursive idea below:
</p>
    <!--Recursive description of the algorithm/problem-->
    <p>
        $$
        k(W, n) =
        \begin{cases}
        0, & \text{if } W \leq 0 \text{ or } n \geq N, \\[6pt]
        k(W, n+1), & \text{if } w_n > W, \\[6pt]
        \max\left(
        \underbrace{v_n + k(W - w_n, n+1)}_{\text{Include item}}, \right. \\
        \left. \underbrace{k(W, n+1)}_{\text{Exclude item}} \right), & \text{otherwise.}
        \end{cases}
        $$
    </p>
    <p class="time-complexity">Time Complexity:  $(2^n)$ </p>
    <div class="variable">

```

```

        <label for="max_iterations_recursive">Recursion limit:
</label>

        <div class="slidecontainer">
            <input type="range" min="1000" max="10000000"
value="10000000" class="slider" name="max_iterations_recursive">
        </div>
            <input type="number" class="slidevalue" min="1000"
max="10000000" value="10000000">
        </div>
    </div>
    <!--Greedy Approximation-->
    <div class="algorithm" id="Greedy Approximation"
style="display:none">
        <p class="algorithm-description">Sorts the items in
descending order based upon their value/weight ratio. The algorithm then loops
through the list until no more items can be packed.</p>
        <p class="time-complexity">Time Complexity:  $O(n \log n)$ </p>
    </div>
    <!--Dynamic Programming Memoization-->
    <div class="algorithm" id="DP (Memoization)"
style="display:none">
        <p class="algorithm-description">Due to the problem having an
overlapping sub-problem structure, the memoization technique stems from the recursive
approach but caches the results of previous sub-problems, and looks them up before
recomputing therefore avoiding redundant calculations.</p>
        <p class="time-complexity">Time Complexity:  $O(nW)$ </p>
    </div>
    <!--Dynamic Programming Tabulation-->
    <div class="algorithm" id="DP (Tabulation)" style="display:none">
        <p class="algorithm-description">Tabulation creates a table
and builds it up starting with resolving the smallest sub-problems until the main
problem is solved.</p>
        <p class="time-complexity">Time Complexity:  $O(nW)$ </p>
    </div>
    <!--Branch and Bound-->
    <div class="algorithm" id="Branch and Bound"
style="display:none">
        <p class="algorithm-description">We use the greedy approach
to generate upper and lower bounds for the knapsack problem, and then prune branches
to reduce the search space. We always explore the most promising sub-problems first.
</p>
    </div>
    <!--Simulated Annealing-->

```

```

<div class="algorithm" id="Simulated Annealing"
style="display:none">
    <p class="algorithm-description">Annealing in metallurgy is
when a material is first heated up to a high temperature and then cooled down slowly
over time to achive a stable crystalline structure. The algorithm takes inspiration
from this with the 'temperature' variable corresponding to the degree of randomness
in the search process which will decrease over time to refine the solution. 'Alpha'
is the factor by which the temperature reduces per iteration.</p>
    <label class="checkbox-container">Greedy Initialisation:
        <input name="is_greedy_SA" type="checkbox" checked>
        <span class="checkmark"></span>
    </label>
    <div class="variable">
        <label for="initial_temperature">Initial temperature:
</label>
        <div class="slidecontainer">
            <input type="range" min="1" max="10" step="0.1"
value="3.5" class="slider" name="initial_temperature">
        </div>
        <input type="number" class="slidevalue" min="1" max="10"
step="0.1" value="3.5">
    </div>
    <div class="variable">
        <label for="minimum_temperature">Minimum temperature:
</label>
        <div class="slidecontainer">
            <input type="range" min="0" max="1" step="0.1"
value="0.4" class="slider" name="minimum_temperature">
        </div>
        <input type="number" class="slidevalue" min="0" max="1"
step="0.1" value="0.4">
    </div>
    <div class="variable">
        <label for="alpha">Alpha: </label>
        <div class="slidecontainer">
            <input type="range" min="0.9" max="0.9999"
step="0.0001" value="0.9995" class="slider" name="alpha">
        </div>
        <input type="number" class="slidevalue" min="0.9"
max="0.9999" step="0.0001" value="0.9995">
    </div>
    <div class="variable">
        <label for="max_iterations_SA">Max iterations: </label>
        <div class="slidecontainer">
            <input type="range" min="1000" max="10000"
value="3300" class="slider" name="max_iterations_SA">

```

```

        </div>
        <input type="number" class="slidevalue" min="1000"
max="10000" value="3300">
    </div>
</div>
<!--Tabu Search-->
<div class="algorithm" id="Tabu Search" style="display:none">
    <p class="algorithm-description">Starts with a candidate
solution and repeatedly finds the best neighbouring solution. Stores previously
visited solutions in a tabu list to avoid getting trapped in a local optima.</p>
    <label class="checkbox-container">Greedy Initialisation:
        <input name="is_greedy_tabu" type="checkbox" checked>
        <span class="checkmark"></span>
    </label>
    <div class="variable">
        <label for="max_iterations_tabu_search">Max iterations:
</label>
        <div class="slidecontainer">
            <input type="range" min="250" max="3000" value="1500"
class="slider" name="max_iterations_tabu_search">
        </div>
        <input type="number" class="slidevalue" min="250"
max="3000" value="1500">
    </div>
    <div class="variable">
        <label for="tabu_list_size">Tabu list size: </label>
        <div class="slidecontainer">
            <input type="range" min="10" max="1000" value="500"
class="slider" name="tabu_list_size">
        </div>
        <input type="number" class="slidevalue" min="10"
max="1000" value="500">
    </div>
</div>
<!--Particle Swarm Optimisation (PSO)-->
<div class="algorithm" id="Particle Swarm Optimisation"
style="display:none">
    <p class="algorithm-description">Inspired by natural life
processes such as the social behaviour of birds flocking or fish schooling, a
population (swarm) of candidate solutions is called particles, where each particle
has a position and velocity value over the search space. Each particle is influenced
by its own best and the global best known solution in order to explore the search
space.</p>
    <div class="variable">
        <label for="max_iterations_PSO">Max iterations: </label>
        <div class="slidecontainer">

```

```

        <input type="range" min="10" max="300" value="55"
class="slider" name="max_iterations_PSO">
    </div>
    <input type="number" class="slidevalue" min="10"
max="300" value="55">
    </div>
    <div class="variable">
        <label for="population_size_PSO">Number of particles:
</label>

        <div class="slidecontainer">
            <input type="range" min="10" max="300" value="35"
class="slider" name="population_size_PSO">
            </div>
            <input type="number" class="slidevalue" min="10"
max="300" value="35">
            </div>
            <div class="variable">
                <label for="inertia">Inertia: </label>
                <div class="slidecontainer">
                    <input type="range" min="0" max="1" value="0.8"
step="0.01" class="slider" name="inertia">
                    </div>
                    <input type="number" class="slidevalue" min="0" max="1"
value="0.8" step="0.01">
                    </div>
                    <div class="variable">
                        <label for="cognitive_factor">Cognitive factor: </label>
                        <div class="slidecontainer">
                            <input type="range" min="1" max="5" value="1.2"
step="0.1" class="slider" name="cognitive_factor">
                            </div>
                            <input type="number" class="slidevalue" min="1" max="5"
value="1.2" step="0.1">
                            </div>
                            <div class="variable">
                                <label for="social_factor">Social factor: </label>
                                <div class="slidecontainer">
                                    <input type="range" min="1" max="5" value="3"
step="0.1" class="slider" name="social_factor">
                                    </div>
                                    <input type="number" class="slidevalue" min="1" max="5"
value="3" step="0.1">
                                    </div>
                                    <div class="variable">
                                        <label for="steepness">Sigmoid function steepness:
</label>

```

```

        <div class="slidecontainer">
            <input type="range" min="1" max="3" value="2.2"
step="0.1" class="slider" name="steepness">
        </div>
        <input type="number" class="slidevalue" min="1" max="3"
value="2.2" step="0.1">
    </div>
    <div class="variable">
        <label for="max_velocity">Maximum velocity: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="10" value="4.3"
step="0.1" class="slider" name="max_velocity">
        </div>
        <input type="number" class="slidevalue" min="1" max="10"
value="4.3" step="0.1">
    </div>
</div>
<!--Genetic Algorithm (GA)-->
<div class="algorithm" id="Genetic Algorithm"
style="display:none">
    <p class="algorithm-description">Simulates natural selection
across multiple candidate solutions. Each generation, crossover between solutions and
mutation occurs. Selection techniques combined with elitism are used to pick the best
solutions for the next generation. This process repeats over a large number of
generations, keeping track of the best valid solution.</p>
    <div class="variable">
        <label for="max_iterations_GA">Generations: </label>
        <div class="slidecontainer">
            <input type="range" min="50" max="300" value="100"
class="slider" name="max_iterations_GA">
        </div>
        <input type="number" class="slidevalue" min="50"
max="300" value="100">
    </div>
    <div class="variable">
        <label for="population_size_GA">Population size: </label>
        <div class="slidecontainer">
            <input type="range" min="10" max="300" value="50"
class="slider" name="population_size_GA">
        </div>
        <input type="number" class="slidevalue" min="10"
max="300" value="50">
    </div>
    <label for="selection_type_GA">Selection type:</label>
    <select name="selection_type_GA" id="selection_type_GA">
        <option value="Tournament selection">Tournament

```

```

selection</option>
        <option value="Roulette wheel selection">Roulette wheel
selection</option>
    </select>
    <div class="variable" id="tournament_size_variable">
        <label for="tournament_size_GA">Tournament size: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="20" value="9"
class="slider" name="tournament_size_GA">
        </div>
        <input type="number" class="slidevalue" min="1" max="20"
value="9">
    </div>
    <div class="variable">
        <label for="cross_over_rate">Cross over rate: </label>
        <div class="slidecontainer">
            <input type="range" min="0" max="1" value="0.6"
step="0.01" class="slider" name="cross_over_rate">
        </div>
        <input type="number" class="slidevalue" min="0" max="1"
value="0.6" step="0.01">
    </div>
    <div class="variable">
        <label for="mutation_rate">Mutation rate: </label>
        <div class="slidecontainer">
            <input type="range" min="0" max="0.2" value="0.05"
step="0.01" class="slider" name="mutation_rate">
        </div>
        <input type="number" class="slidevalue" min="0"
max="0.2" value="0.05" step="0.01">
    </div>
    <div class="variable">
        <label for="elitism_rate">Elitism rate: </label>
        <div class="slidecontainer">
            <input type="range" min="0" max="0.2" value="0.05"
step="0.01" class="slider" name="elitism_rate">
        </div>
        <input type="number" class="slidevalue" min="0" max="0.2"
value="0.05" step="0.01">
    </div>
</div>
<!--Ant Colony Optimisation (ACO)-->
<div class="algorithm" id="Ant Colony Optimisation"
style="display:none">
    <p class="algorithm-description">ACO is inspired by the
behaviour of ants when searching for food. Each ant lays down a pheromone trail (a

```

type of hormone) when exploring a particular path. Over time, good/frequently chosen paths will accumulate pheromone, attracting more ants. Additionally, the choices a 'path' takes is influenced by other heuristic information (attractiveness). Evaporation of pheromone occurs, so that older suboptimal paths lose influence over time.

```

        <div class="variable">
            <label for="max_iterations_ACO">Max iterations: </label>
            <div class="slidecontainer">
                <input type="range" min="1" max="100" value="11"
class="slider" name="max_iterations_ACO">
            </div>
            <input type="number" class="slidevalue" min="1" max="100"
value="11">
        </div>
        <div class="variable">
            <label for="ants">Number of ants: </label>
            <div class="slidecontainer">
                <input type="range" min="1" max="20" value="5"
class="slider" name="ants">
            </div>
            <input type="number" class="slidevalue" min="1" max="20"
value="5">
        </div>
        <div class="variable">
            <label for="alpha_ACO">Alpha: </label>
            <div class="slidecontainer">
                <input type="range" min="1" max="100" value="5"
class="slider" name="alpha_ACO">
            </div>
            <input type="number" class="slidevalue" min="1" max="100"
value="5">
        </div>
        <div class="variable">
            <label for="beta_ACO">Beta: </label>
            <div class="slidecontainer">
                <input type="range" min="1" max="100" value="1"
class="slider" name="beta_ACO">
            </div>
            <input type="number" class="slidevalue" min="1" max="100"
value="1">
        </div>
        <div class="variable">
            <label for="evaporation_rate">Evaporation rate:
</label>
            <div class="slidecontainer">
                <input type="range" min="0" max="1" step="0.01"

```

```

value="0.9" class="slider" name="evapouration_rate">
    </div>
    <input type="number" class="slidevalue" min="0" max="1"
step="0.01" value="0.9">
    </div>
</div>
</div>
<!--Used to store data values within the page (this is hidden from
the user)-->
<div id="data" style="display:none">
    <textarea name="weights">{{ weights }}</textarea>
    <textarea name="values">{{ values }}</textarea>
    <textarea name="capacity">{{ capacity }}</textarea>
</div>
</form>
</div>
<br>

```

Here is the *data form*. It provides the interface for the **6.X** Inputting Data objective. Importantly for the generate random section, in this form I set the value of the input to the flask variable in order to maintain these variables whenever the page refreshes. I thought this would be beneficial if the user wants to do repeats on the same 'style' of random data, i.e. with the same range of values. This same method could also be applied to the algorithm parameters, however I thought it was necessary to preserve the default settings for consistency.

```

<strong>Data: </strong>
<div class="data form">
  <!--Form initially submitted to generate/submit/upload data-->
  <form method="post" id="data form" enctype="multipart/form-data" action =
"/">

    <label for="input_data">Input data:</label>
    <select name="input_data" id="input_data">
      <option value="generate_random">Generate random</option>
      <option value="input_manually">Input manually</option>
      <option value="import_csv">Import CSV</option>
    </select>
    <div id="generate_random">
      <!-- This will having the following control variables: Number of
items, Range of values, Correlation strength, Capacity Load-->
      <div class="variable">
        <label for="number_of_items">Number of items: </label>
        <div class="slidecontainer">
          <input type="range" min="5" max="1000" value="{{
number_of_items }}" class="slider" name="number_of_items">
        </div>
        <input type="number" class="slidevalue" min="5" max="1000"
value="{{ number_of_items }}">
        </div>
        <div class="variable">
          <label for="range_of_values">Range of values: </label>
          <div class="slidecontainer">
            <input type="range" min="10" max="1000" value="{{
range_of_values }}" class="slider" name="range_of_values">
          </div>
          <input type="number" class="slidevalue" min="10" max="1000"
value="{{ range_of_values }}">
          </div>
          <div class="variable">
            <label for="correlation_strength">Correlation strength:
</label>
            <div class="slidecontainer">
              <input type="range" min="0" max="1" value="{{
correlation_strength }}" class="slider" name="correlation_strength" step="0.01">
            </div>
            <input type="number" class="slidevalue" min="0" max="1"
value="{{ correlation_strength }}" step="0.01">
            </div>
            <div class="variable">
              <label for="capacity_load">Capacity load: </label>
              <div class="slidecontainer">
                <input type="range" min="0.1" max="0.9" value="{{

```

```

capacity_load }}" class="slider" name="capacity_load" step="0.01">
    </div>
    <input type="number" class="slidevalue" min="0.1" max="0.9"
value="{{ capacity_load }}" step="0.01">
    </div>
</div>
<div id="input_manually" style="display:none">
    <br>
    <label for="weights">Enter the weights: </label>
    <input type="text" name="weights" placeholder="17 20 87 54 34">
    <br>
    <label for="values">Enter the values: </label>
    <input type="text" name="values" placeholder="20 24 12 87 45">
    <br>
    <label for="capacity">Enter the capacity:</label>
    <input type="text" name="capacity" placeholder="135">
    <br>
</div>
<div id="import_csv" style="display:none">
    <br>
    <p>You can upload a .csv file as input data when layed out in the
following format: columns ('weights', 'values', 'capacity')</p>
    <input type="file" name="csv_file" accept=".csv">
    <p style="color: blue;">
        Choose csv file to upload
    </p>
</div>
</form>
</div>
</div>
<!--Capacity marker and scrollbar-->
<div class="column2">
    <strong>Capacity:</strong> {{ capacity }}
    <br>
    <strong>Items (w<sub>i</sub>, v<sub>i</sub>):</strong>
    <div class="scroll-box">
        <!--Shows all the items loaded into the website in the vertical
scrollbox-->
        {% for i in range(weights|length) %}
            <div class="item">Item {{ i + 1 }}: ({{ weights[i] }}, {{ values[i]
}}})</div>
        {% endfor %}
    </div>
</div>
</div>
<br>

```

```

<div class="button_controls">
  <button type="submit" form="data form" id="generate_data_button">Generate
data</button>
  <button type="submit" form="algorithm form">Run</button>
</div>
<div style="display:none" id="waiting_text">
  <!--Running algorithm message is displayed to the user as some algorithms may
take a significant time to load-->
  <p class="message-text" >Running algorithm...</p>
</div>
<!--Error handling (Specific error message is inserted below)-->
{% if error %}
<p class="message-text">{{ error }}</p>
{% endif %}
<br>

```

Below is the javascript element from the index.html file. It manages all the different menus using the *display* css property in order to select what items are shown to the user. It uses many event listeners in order to react to the different select dropdowns. It also clears the waiting message once the page is reloaded and prevents the enter key from submitting the form, in order to enhance the user's experience. The *getElementById()* function is used in order to get/connect to a specific html element, while the *addEventListener()* functions is used link a function to an event such as a key press.

```

<script>
    //Used to dynamically update the menus

    //Stops submission on enter key press due to their being 2 forms
    document.getElementById("algorithm form").addEventListener("keydown",
function(event) {
    if (event.key === "Enter"){
        event.preventDefault()
    }
});

    document.getElementById("data form").addEventListener("keydown", function(event)
{
    if (event.key === "Enter"){
        event.preventDefault()
    }
});

    //Matches the algorithm parameters to the algorithm selected
    function change_algorithm_parameters () {

document.getElementById("0/1algorithms").querySelectorAll(".algorithm").forEach(algorithm
=> {
    if (algorithm_setting.id === dropdown_binary_algorithm.value) {
        algorithm_setting.style.display = "inline";
    }
    else {
        algorithm_setting.style.display = "none";
    }
});
}

    const dropdown_binary_algorithm = document.getElementById("algorithm_type");
    dropdown_binary_algorithm.addEventListener("change",
change_algorithm_parameters);

    //Manually uses switch statement in order to match the form parameters for the
input data type
    const dropdown_input_data = document.getElementById("input_data");
    const generate_random_menu = document.getElementById("generate_random");
    const input_manually_menu = document.getElementById("input_manually");
    const import_csv_menu = document.getElementById("import_csv");
    const generate_data_button = document.getElementById("generate_data_button");

    dropdown_input_data.addEventListener("change", function() {
        generate_random_menu.style.display = "none";

```

```

input_manually_menu.style.display = "none";
import_csv_menu.style.display = "none";

if (this.value === 'generate_random') {
    generate_random_menu.style.display = "inline";
    generate_data_button.innerHTML = "Generate data";
}
else if (this.value === 'input_manually') {
    input_manually_menu.style.display = "inline";
    generate_data_button.innerHTML = "Submit data";
}
else if (this.value === 'import_csv') {
    import_csv_menu.style.display = "inline";
    generate_data_button.innerHTML = "Upload csv";
}
});

//Specific menus for genetic algorithms
const selection_type_GA = document.getElementById("selection_type_GA");
const tournament_size_variable =
document.getElementById("tournament_size_variable");
selection_type_GA.addEventListener("change", function() {
    if (this.value === "Tournament selection") {
        tournament_size_variable.style.visibility = "initial";
    }
    else if (this.value === "Roulette wheel selection") {
        tournament_size_variable.style.visibility = "hidden";
    }
});

//Sync slider and textbox value
document.querySelectorAll(".variable").forEach(group => {
    const slider =
group.querySelector(".slidecontainer").querySelector("input[type=range]");
    const number_input = group.querySelector(".slidevalue");

    // Update number when slider changes
    slider.addEventListener('input', function() {
        number_input.value = slider.value;
    });

    // Update slider when number changes (with validation)
    number_input.addEventListener('input', function() {
        let value = number_input.value;
        //Clamps the value between slider.min and slider.max
        if (value < Number(slider.min)) value = slider.min;
    });
});

```

```

        if (value > Number(slider.max)) value = slider.max;
        slider.value = value;
        number_input.value = value;
    });
});

const waiting_text = document.getElementById("waiting_text");

function show_text() {
    waiting_text.style.display = "inline";
}

// Run when DOM is fully loaded (initial load)
document.addEventListener("DOMContentLoaded", function () {
    runOnLoadOrBack();
});

// Run when page is shown
window.addEventListener("pageshow", function (event) {
    if (event.persisted || performance.getEntriesByType("navigation")[0].type ===
"back_forward") {
        runOnLoadOrBack();
    }
});

//Hides the waiting text
function runOnLoadOrBack() {
    waiting_text.style.display = "none";
    change_algorithm_parameters();
}

</script>

{% endblock %}

```

6 - submit.html

This is the page that displays the result data of the run algorithm. The page is adapted based upon on what algorithm the user has chosen, however will always have both statistics and a visual elements.

Below is the html, storing the key information, including 2 scroll boxes and a canvas/image element. Values from [index.py](#) are passed into the html, for example `{{ Data['approach'] }}` for the name of the

algorithm which is then rendered in the html. This allows for a continuity between the front end and back end and satisfies **9.1** Comparative Data criteria.

```

{% extends 'base.html' %}

{% block head %}
<title>Submitted</title>
{% endblock %}

{% block body %}
<h1>{{ Data['approach'] }} (Result Data)</h1> <!--Name of the algorithm-->
<div class="row">
    <div class="column3">
        <div id="result_data">
            <strong>Value: </strong> {{ Data['total_value'] }} <!--Value of solution-->
            <br>
            <strong>Weight: </strong> {{ Data['total_weight'] }} <!--Weight of
solution-->
            <br>
            <strong>Valid: </strong> {{ Data['valid'] }} <!--As a debugging check to
test if the solution is valid-->
            <br>
            <strong>Time(s): </strong> {{ Data['time'] }} <!--Execution time-->
            <br>
            <strong>Items (w<sub>i</sub>, v<sub>i</sub>) included: </strong>
            <br>
            <div class="scroll-box">
                <!--All items in binary_string that are '1'-->
                {% for i in range(Data['binary_string']|length) %}
                    {% if Data['binary_string'][i] == '1': %}
                        <div class="item">Item {{ i + 1 }}: ({{ Data['weights'][i]
}}, {{ Data['values'][i] }})</div>
                    {% endif %}
                {% endfor %}
            </div>

            <strong>Items code: </strong>
            <br>
            <div class="scroll-box-mini">
                <!--Actual binary string returned by program-->
                <div class="item">{{ Data['binary_string'] }}</div>
            </div>
        </div>
    </div>
    <div class="column4">
        <!--Change display output if it is plotted graph or DP table-->
        {% if Data['approach'] == 'DP (Tabulation)': %}
            <!--DP table description-->

```

```

    <p>Below shows the first 250(Capacity/Columns) x 250(Items Included/Rows)
in the dp table as previously explained in the algorithm:</p>
    <div id="canvas-container">
        <canvas id="dp-canvas"></canvas>
    </div>
    {% else: %}
    <div id="graphs">
        <!--Insert encoded image-->
        
        <!--Download image linked to the same source-->
        <a id="download_button" href="data:image/png;base64,{{ Data['r_fig']
}}" download>Download Image</a>
    </div>
    {% endif %}
</div>
<br>

```

Below is the embedded javascript part of the submit.html code. It is used for the virtual table for the DP table visualisation element achieving the **8.3** DP Tabulation Table objective. It uses the *drawVisible* function to act as a listener to draw the *dpData* on to the canvas element. It calculates the bounds of the cells that need to be drawn, and then loops and draws all cells using the *fillRect()/strokeRect()* functions. Different colours are used in order to separate the headers of the table (capacityxitems included) from the actual data within the table.

```

<script>
  //parse json file
  const dpData = JSON.parse('{ Data["r_fig"]|safe }}');

  const cellWidth = 40; //width of cell
  const cellHeight = 20; //height of cell

  //rows and cols may not be 250 if not maximised
  const rows = dpData.length;
  const cols = dpData[0].length;

  const container = document.getElementById("canvas-container");
  const canvas = document.getElementById("dp-canvas");
  const ctx = canvas.getContext("2d");

  //Set virtual canvas size
  canvas.width = Math.min(cellWidth * cols, 10000);
  canvas.height = Math.min(cellHeight * rows, 5000);

  //console.log(canvas.width, canvas.height)

  //Function that draws the current displayed output in the container
  function drawVisible() {

    const scrollTop = container.scrollTop;
    const scrollLeft = container.scrollLeft;

    const startRow = Math.floor(scrollTop / cellHeight);
    const endRow = Math.min(rows, startRow + Math.ceil(container.clientHeight /
cellHeight) + 1);

    const startCol = Math.floor(scrollLeft / cellWidth);
    const endCol = Math.min(cols, startCol + Math.ceil(container.clientWidth /
cellWidth) + 1);

    //clear output before drawing cells
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    //Output text format settings
    ctx.font = "12px Arial";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";

    for (let r = startRow; r < endRow; r++) {
      for (let c = startCol; c < endCol; c++) {
        //x, y coordinates of the cell

```

```

const x = c * cellWidth;
const y = r * cellHeight;

//Draw the cells with appropriate colours

if (r === 0 || c === 0)
{
    ctx.fillStyle = "#fff";
    //Coloured Rectangle
    ctx.fillRect(x, y, cellWidth, cellHeight);
}

ctx.strokeStyle = "#000";
//Bordered Rectangle
ctx.strokeRect(x, y, cellWidth, cellHeight);

ctx.fillStyle = "#000";
if (!(r === 0 && c === 0))
{
    //Fill text with dp array
    ctx.fillText(dpData[r][c], x + cellWidth / 2, y + cellHeight /
2);
}
else
{
    //Top left cell should be unmarked
    ctx.fillText('#', x + cellWidth / 2, y + cellHeight / 2)
}
}
}

container.addEventListener("scroll", drawVisible);
drawVisible();
</script>
</body>
</html>

{% endblock %}

```

7 - main.css

I only have only one CSS file that takes care of all the styling through every html page (linked to base.html). This is used both for arranging and decorating elements, improving the user experience

and ease of use. Most of the code is quite simple and just for aesthetic purposes. There are simple hover animations for buttons and checkmarks while remaining with a constant colour theme throughout. The CSS is not overly complex, however does improve over the plain html.

```

/* Fixed Title Image*/
#title img{
    width: 160px;
    height: 150px;
    margin: 10px;
}

html {
    zoom: 1.25;
}

body {
    margin: 10px;
    font-family: sans-serif;
    line-height: 2;
    background-color: rgb(195, 223, 240);
}

#title {
    display: flex;
    gap: 20px;
}

select {
    width: 200px;
    height: 25px;
    line-height: normal;
}

p, h1, option {
    line-height: normal;
}

h1 {
    margin-bottom: 10px;
}

.variable {
    display: flex;
    gap: 7px;
}

.slidecontainer {
    width: 150px; /* Width of the outside container */
}

```

```

.variable input[type="number"] {
    border: none;
    color: green;
    margin: 3px;
    border-radius: 3px;
}

/* checkbox*/

.checkbox-container {
    display: inline-block;
    position: relative;
    margin: 10px 0;
    cursor: pointer;
    font-size: 16px;
    user-select: none;
}

/* Hide the default checkbox */
.checkbox-container input {
    display: none;
}

/* Custom checkbox square */
.checkmark {
    position: absolute;
    top: 5px;
    left: 150px;
    height: 20px;
    width: 20px;
    background-color: #eee;
    border: 2px solid #ccc;
    border-radius: 4px;
    transition: all 0.2s ease;
    transform: scale(0.8);
}

.checkbox-container:hover input ~ .checkmark {
    background-color: #ddd;
}

/* Checked state */
.checkbox-container input:checked ~ .checkmark {
    background-color: #4CAF50;
    border-color: #4CAF50;
}

```

```

}

.checkmark:after {
    content: "";
    position: absolute;
    display: none;
}

.checkbox-container input:checked ~ .checkmark:after {
    display: block;
}

/* Tick styling */
.checkbox-container .checkmark:after {
    left: 6px;
    top: 2px;
    width: 5px;
    height: 10px;
    border: solid white;
    border-width: 0 2px 2px 0;
    transform: rotate(45deg);
}

/* The slider itself */
.slider {
    -webkit-appearance: none;
    width: 100%;
    height: 10px;
    border-radius: 3px;
    background: #d3d3d3;
    outline: none;
    opacity: 0.7;
    -webkit-transition: .2s;
    transition: opacity .2s;
}

.slider::-webkit-slider-thumb {
    -webkit-appearance: none;
    appearance: none;
    width: 12px;
    height: 12px;
    border-radius: 50%;
    background: green;
    cursor: pointer;
}

```

```

.slider::-moz-range-thumb {
    width: 12px;
    height: 12px;
    border-radius: 50%;
    background: green;
    cursor: pointer;
}

.message-text {
    color: red;
}

.variable label {
    display: inline-block;
    white-space: nowrap;
    width: auto;
    max-width: 200px;
}

button {
    background-color: rgb(51, 197, 51);
    border-radius: 5px;
    color: white;
    height: 25px;
    width: auto;
    border: none;
    transform: scale(1.15);
}

input[type="file"] {
    background-color: rgb(51, 197, 51);
    color: white;
    padding: 4px 8px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    margin: 5px;
}

/* vertical scrollbar */
.scroll-box {
    border: 1px solid #ccc;
    padding: 10px;
    max-width: 200px;
    width: auto;
    height: 300px;
}

```

```

    overflow-y: scroll;
    background-color: #f9f9f9;
    border-radius: 3px;
}

/* horizontal scrollbar */
.scroll-box-mini {
    border: 1px solid #ccc;
    padding: 10px;
    max-width: 200px;
    width: auto;
    height: 45px;
    overflow-x: scroll;
    background-color: #f9f9f9;
    border-radius: 3px;
}

.row {
    display: flex;
    gap: 15px;
}

.column1 {
    width: 75%;
}

.column2 {
    width: 25%;
    padding: 10px;
}

.column3 {
    width: 25%;
}

.column4 {
    width: 75%;
}

.button_controls {
    display: flex;
    gap: 20px;
    margin-left: 15px;
}

#graphs img {

```

```

    max-width: 100%;
    max-height: 100%;
    display: block;
    margin: auto;
    border-radius: 5px;
}

#graphs {
    width: 90%;
    margin-left: auto;
    margin-right: auto;
    text-align: right;
}

#download_button {
    display: inline-block;
    padding: 0px 4px;
    background-color: #007BFF;
    color: white;
    text-decoration: none;
    border-radius: 5px;
    font-family: Arial, sans-serif;
    font-size: 16px;
    transition: background-color 0.3s ease;
    border: 5px;
    margin-top: 5px;
}

#download_button:hover {
    background-color: #0056b3; /* Darker on hover */
}

/* table */

#canvas-container {
    width: 100%;
    height: 500px;
    overflow: auto;
    position: relative;
}

canvas {
    position: absolute;
    top: 0;
    left: 0;
}

```

```
.time-complexity {  
    font-style: italic;  
}
```

The styling is adaptive to the screen size and changes to better suit the user for example if they are on a phone or a smaller tablet. Below shows the css for the smaller screen size, thereby completing the **7.1** Design Interface objective. It overrides some of the previous styling functions including the directions of the columns and *width, padding and margin etc.*

```

/* 893 px width aspect ratio changes */

@media (max-width: 893px) {
  body {
    margin: none;
    margin-left: 5px;
  }

  #title {
    flex-direction: column-reverse;
    align-items: center;      /* Vertical */
    text-align: center;
  }

  .row {
    flex-direction: column;
  }

  .column1, .column2, .column3, .column4 {
    width: 100%;
    padding: 10px;
  }

  #title h1 {
    margin-bottom: 0;
  }

  #title img {
    width: 192px;
    height: 180px;
  }

  #graphs {
    width: 96%;
    margin: 0px;
  }
}

```

8 - requirements.txt

In order to deploy my website so that it can be used by others and shared during my evaluation section I bought the domain name using namecheap and deployed it using vercel. This is a list

generated through `pip freeze > requirements.txt`, of a list of all the requirements used needed when deploying this live.

```
blinker==1.9.0
cachelib==0.13.0
click==8.3.1
colorama==0.4.6
contourpy==1.3.2
cyclr==0.12.1
Flask==3.1.3
fonttools==4.61.1
itsdangerous==2.2.0
Jinja2==3.1.6
kiwisolver==1.4.9
MarkupSafe==3.0.3
matplotlib==3.10.8
msgspec==0.20.0
numpy==2.2.6
packaging==26.0
pandas==2.3.3
pillow==12.1.1
pyparsing==3.3.2
python-dateutil==2.9.0.post0
pytz==2025.2
six==1.17.0
tzdata==2025.3
Werkzeug==3.1.6
```

9 - vercel.json

This file is used to direct an incoming request to the home page and is needed by vercel when deploying a website.

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/api/index.py" }
  ]
}
```

10 - Bibliography (Links and Documentation)

- 'Learn Flask for Python - Full Tutorial'. [freeCodeCamp.org](https://www.youtube.com/watch?v=Z1RJmh_OqeA). From https://www.youtube.com/watch?v=Z1RJmh_OqeA **Used to learn basic flask syntax and set up project**
- 'How TO - Range Sliders'. w3schools. From https://www.w3schools.com/howto/howto_js_rangeslider.asp **Used when implementing html for sliders with textbox**
- 'What is the difference between visibility:hidden and display:none ?'. geeksforgeeks. From <https://www.geeksforgeeks.org/css/what-is-the-difference-between-visibilityhidden-and-displaynone/> **Used when toggling different menus**
- 'Top 3 Ways to Call a JavaScript Function on Page Load'. sqlpey. From <https://sqlpey.com/javascript/top-3-ways-to-call-a-javascript-function-on-page-load/#method-1-using-windowonload> **Used to reset page on reload**
- 'Uploading and Reading a CSV File in Flask'. geeksforgeeks. From <https://www.geeksforgeeks.org/python/uploading-and-reading-a-csv-file-in-flask/> **Used when learning how to implement upload CSV feature**
- 'Basics of MathJax (+ How to use it?)'. iq.opengenus.org. <https://iq.opengenus.org/mathjax/> **Used when inserting math notation on home page**
- 'Latex Equation Editor'. Lagrida. From <https://latexeditor.lagrida.com/> **When getting latex for custom math notation**
- 'What Is Gray Code?'. FINtechnician. From <https://www.youtube.com/watch?v=58HqmMYhfNw> **Used to optimise the brute force iterative approach**
- 'How to Deploy a Flask App to Vercel | Flask Vercel Deployment'. Arpan Neupane. From <https://www.youtube.com/watch?v=LaMVBDtUtMA> **Used when deploying web app online**

A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the Knapsack Problem

Testing Section

Name: Aarav Bhatnagar - Centre Number: 39253 - Candidate Number: 8015

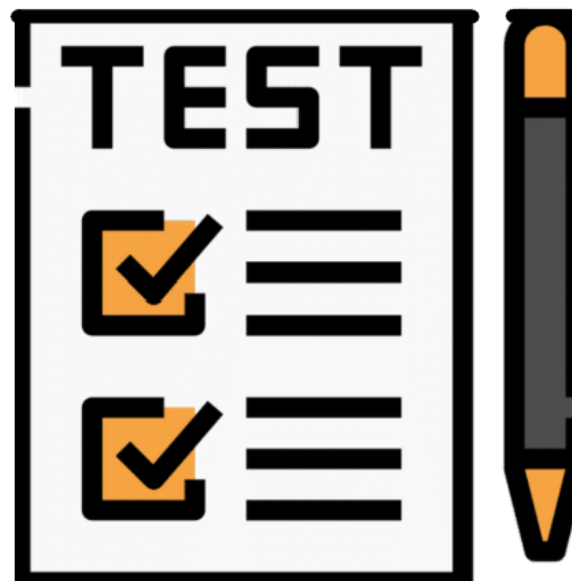


Table of Contents

- [1 - Introduction](#) (pg 136)
- [2 - Testing Plan](#) (pg 137)
- [3 - Conclusion](#) (pg 167)

1 - Introduction

While I am developing my technical solution, I will follow a precise and pre-defined testing plan in order to ensure that all my objectives are complete. This will also increase the robustness of my code and help me inform future design decisions. I will perform tests on my code both during the development process and on the complete product. Below is my testing plan which will include multiple testing tables, linked code and visual evidence of results. I also made a testing video, which shows a detailed overview of the entire final product. Below is the QR code/video link and time-stamps for specific tests are giving in the tables where appropriate.

I also did erroneous and boundary testing on specific input handling tests including 25 & 26, however most of my tests are with respect to algorithms and visual GUI, without any specific input data.

QR Code for Video



www.youtube.com/watch?v=4iJZjXkPq9M

2 - Testing Plan

I separated the testing plan across multiple sub-tables due to formatting issues. Each test has a given number, name, objective, reason, expected result, actual result, visual evidence. The unsuccessful tests have a further explanation row and the successful tests are included in the testing video with their time stamps as seen below.

Testing no.	1
Test Name	Evaluate Function
Objective being tested	1.X and 2.X: Exact Algorithms and Heuristic Algorithms
Reason for test	Subroutine to return the fitness/value of a solution
Expected result	Takes a binary string and returns the sum of the values of items in the knapsack; if the sum of the items weights are above the capacity, it will return 0
Actual result	● SUCCESSFUL
Visual Evidence	<pre>def evaluate(self, binary_string, get_total_weight=False): total_value = sum([self.values[x] for x in range(len(self.values)) if binary_string[x] == "1"]) total_weight = sum([self.weights[x] for x in range(len(self.weights)) if binary_string[x] == "1"]) if total_weight > self.capacity: total_value = 0 if get_total_weight: return total_value, total_weight else: return total_value</pre>

```
problem = BinaryKP([1, 5, 2, 4, 2], [4, 3, 1, 2, 4], 13, algorithm_data=None)
print(problem.evaluate("01101"))
print(problem.evaluate("11111"))
```

Testing no.	2
Test Name	Brute Force - Iterative
Objective being tested	1.1: Exact Algorithms => Brute Force
Reason for test	The simplest and naive approach to solving the problem
Expected result	Testing code structure and the basic algorithm on small values of 'n' (runs too slowly on larger datasets)
Actual result	 SUCCESSFUL

Visual Evidence

```
def solve_01_knapsack_BF_iterative(self):
    #graph max_length
    max_length = 10000
    #loop through all subsets/combinations of items and return one with the highest total value that doesn't exceed self.capacity
    #A perhaps interesting way of doing this is counting upwards in binary
    #In order to count up efficiently, we must use gray codes so that between each iteration we only flip one bit, thereby the O(n) evaluate function is not called n times.
    n = len(self.weights)
    max_value = 0
    best_mask = 0 # to store the Gray code subset

    total_weight = 0
    total_value = 0

    best_values = []
    current_values = []
    iterations_list = []

    iterations = min((2**n), self.algorithm_data['max_iterations_iterative'] + 1)
    # calculate step size to reduce list to <= max_length
    step = max(1, (iterations - 1) // max_length)

    for mask in range(1, iterations):
        #max loops from 1 to (2**n) - 1
        gray = mask ^ (mask >> 1) # compute Gray code i.e. 1110

        # Previous Gray Code
        prev_gray = (mask - 1) ^ ((mask - 1) >> 1) #sub mask -1 into formula i.e. 1010
        # Bit that changed
        diff = gray ^ prev_gray #XOR because 1110 1010 gives 0100
        i = n - ((diff).bit_length()) # index of changed bit (bit length returns the number of bits to store number there 00010 returns 2)

        if gray & diff != 0: # item added prev_gray = 001, gray = 011 therefore diff = 010: gray & diff = 011 & 010 = 010 which is not zero => item is added
            total_weight += self.weights[i]
            total_value += self.values[i]
        else: # item removed
            total_weight -= self.weights[i]
            total_value -= self.values[i]

        if total_weight <= self.capacity and total_value > max_value:
            max_value = total_value
            best_mask = gray # store best subset mask

    if mask % step == 0:
        best_values.append(max_value)
        current_values.append(total_value if total_weight <= self.capacity else 0)
        iterations_list.append(mask)
```

Video Timestamp

4.57

Testing no.	3
Test Name	Brute Force - Recursive

Objective being tested	1.1: Exact Algorithms => Brute Force
Reason for test	An alternative method that breaks down the problem into smaller chunks
Expected result	Test algorithm on small values of 'n' (runs too slowly on larger datasets)
Actual result	 SUCCESSFUL
Visual Evidence	<pre>def solve_01_knapsack_BF_recursive(weights, values, capacity, n=0, best_combination=''): if n == len(weights) or capacity == 0: return (0, best_combination.zfill(len(weights))) elif weights[n] > capacity: return solve_01_knapsack_BF_recursive(weights, values, capacity, n+1, best_combination + '0') else: including_item, best_combination1 = solve_01_knapsack_BF_recursive(weights, values, capacity-weights[n], n+1, best_combination + '1') including_item += values[n] excluding_item, best_combination2 = solve_01_knapsack_BF_recursive(weights, values, capacity, n+1, best_combination + '0') if including_item > excluding_item: return (including_item, best_combination1) else: return (excluding_item, best_combination2)</pre>
Video Timestamp	6.18

Testing no.	4
Test Name	Randomised Search
Objective being tested	2.1: Heuristic Algorithms => Randomised Search
Reason for test	Should provide a control value for the Heuristic algorithms
Expected result	Outputs on average quite a poor value. As time increases, the solution quality also gradually increases
Actual result	 SUCCESSFUL

Visual Evidence	<pre>def solve_01_knapsack_random(self): n = len(self.weights) RUNTIME = self.algorithm_data['time_limit'] time_interval = min(0.001, RUNTIME / 100) start_time = last_time = time.time() best_combination = '0' * n max_value = 0 best_values = [] current_values = [] times_list = [] counter = 0 while time.time() < start_time + RUNTIME: #Generates a random binary string of fixed length n binary_string = bin(random.randint(0, (2**n)-1))[2:].zfill(n) value = self.evaluate(binary_string) if value > max_value: best_combination = binary_string max_value = value</pre>
Video Timestamp	7.10

Testing no.	5
Test Name	Greedy Approximation
Objective being tested	2.3: Heuristic Algorithms => Greedy Approximation
Reason for test	This is another key algorithm. I needed to make sure that it was sorting correctly by value/weight ratio, and selecting the maximum possible number of items
Expected result	Near optimal value produced quickly due to the inherent greedy nature of the problem
Actual result	 SUCCESSFUL

Visual Evidence	<pre>def solve_01_knapsack_GApprox(self, weights=[], values=[], capacity=[], get_total_weight=False, generate_graph=False): #To get a general approximation (this would be exact for the fractional knapsack problem), we can generate a list of value / weight if len(weights) == 0: weights = self.weights values = self.values capacity = self.capacity n = len(weights) items = [None] * n for x in range(n): weight = weights[x] value = values[x] items[x] = ((value/weight, weight, value, x)) items = sorted(items, reverse=True) total_weight = 0 max_value = 0 best_combination = ['0'] * n final_item_index = 0 for x in range(n): if items[x][1] + total_weight > capacity: final_item_index = x break total_weight += items[x][1] max_value += items[x][2] best_combination[items[x][3]] = '1'</pre>
-----------------	--

Video Timestamp	9.12
------------------------	------

Testing no.	6
Test Name	DP - Memoization
Objective being tested	1.2: Exact Algorithms => Dynamic Programming
Reason for test	To make sure that the algorithm worked with a significant amount of recursive calls without crashing and that the lru_cache decorator correctly caches previously run results
Expected result	Provides an exact solution with similar timed results as tabulation
Actual result	 UNSUCCESSFUL - Doesn't provide full control over how data is cached, and doesn't record additional necessary data for the graph including hits (the result was cached) and misses (the result was not cached)
Visual Evidence	<pre>from functools import lru_cache @lru_cache(maxsize=None) def solve_01_knapsack_BF_recursive(self):</pre>
Further Explanation	The lru_cache decorator is often used in competitive programming to quickly memoize recursive functions. However it does not provide the fine control required to track cache hits and misses needed for the analysis.

Testing no.	7
Test Name	DP - Memoization with manual caching
Objective being tested	1.2: Exact Algorithms => Dynamic Programming
Reason for test	Cache the results of function calls manually using a memo python dictionary based on (W, n)
Expected result	Same results to test 5 but with the appropriate data stored over time
Actual result	 SUCCESSFUL

Visual Evidence	<pre>def solve_01_knapsack_DP_Top_Down(self): memo = {} memo_size = [] hits_list = [] def solve(capacity=0, n=0, best_combination='', plot_pos_x=0, plot_pos_y=0, dx=20): self.recursion_counter += 1 memo_size.append(len(memo)) hits_list.append(self.hits) if n == 0: capacity = self.capacity if n == len(self.weights): return (0, '') elif capacity == 0: return (0, '0' * (len(self.weights) - n - 1)) elif (capacity, n) in memo: self.hits += 1 return memo[(capacity, n)] else: including_item = 0 if self.weights[n] <= capacity: including_item, best_combination1 = solve(capacity-self.weights[n], n+1, best_combination) including_item += self.values[n] excluding_item, best_combination2 = solve(capacity, n+1, best_combination) if including_item > excluding_item: result = (including_item, '1' + best_combination1) else: result = (excluding_item, '0' + best_combination2) memo[(capacity, n)] = result return result value, solution = solve()</pre>
Video Timestamp	9.56

Testing no.	8
Test Name	DP - Tabulation
Objective being tested	1.2: Exact Algorithms => Dynamic Programming
Reason for test	To check if the DP formulisation was correctly implemented
Expected result	Correctly outputs final value and correctly constructs valid optimal solution from table
Actual result	 SUCCESSFUL

Visual Evidence	<pre> def solve_01_knapsack_DP_Bottom_Up(self): #Bottom up (Tabulation) n = len(self.weights) #Rows are items and columns are self.capacity #dp[i][c] = best value using items 0...row-1 with self.capacity x #therefore columns is capacity and rows is number of items dp = [[0] * (self.capacity + 1) for _ in range(n+1)] # Fill table bottom-up for i in range(1, len(dp)):#i-items for c in range(self.capacity + 1):#c-self.capacity if self.weights[i-1] <= c: include_item = dp[i-1][c-self.weights[i-1]] + self.values[i-1]#row above shifted along by the weight of the new item exclude_item = dp[i-1][c] #row above dp[i][c] = max(include_item, exclude_item) else: dp[i][c] = dp[i-1][c] #Carry over from row above max_value = dp[n][self.capacity] #Reconstruct best_combination i = n c = self.capacity best_combination = ['0'] * n while i != 0 and c != 0: if dp[i][c] != dp[i-1][c]:#Different value, therefore item was picked best_combination[i-1] = '1' c -= self.weights[i-1] i -= 1 else: i -= 1 </pre>
Video Timestamp	10.39

Testing no.	9
Test Name	Fractional Knapsack Problem for BnB upper bound
Objective being tested	1.3: Exact Algorithms => Branch and Bound
Reason for test	In order to generate an upper bound for the branch and bound algorithm we must first use the greedy algorithm to generate an exact solution to the fractional knapsack problem. This should be as efficient as possible as it will be called repeatedly in the wider algorithm
Expected result	Solves the fractional knapsack problem tested on many different values of 'n'
Actual result	● SUCCESSFUL

Visual Evidence	<pre> def solve_fractional_knapsack_G(self, weights, values, capacity): n = len(weights) items = [] for x in range(n): weight = weights[x] value = values[x] items.append((value/weight, weight, value)) items = sorted(items, reverse=True) total_weight = 0 max_value = 0 for x in range(n): if items[x][1] + total_weight > capacity: max_value += (items[x][2] * ((capacity - total_weight)/items[x][1])) break total_weight += items[x][1] max_value += items[x][2] return max_value </pre>
-----------------	--

Testing no.	10
Test Name	Branch and Bound
Objective being tested	1.3: Exact Algorithms => Branch and Bound
Reason for test	Test if theoretical algorithm can solve knapsack problem efficiently through use of heap data structure
Expected result	Branch and bound finds an exact solution to larger values of 'n' in a reasonable amount of time
Actual result	● SUCCESSFUL

Visual Evidence	<pre>def solve_01_knapsack_BnB(self): n = len(self.weights) max_value = 0 best_solution = '0' * n priority_queue = [] priority_queue_length_list = [] branches_pruned = 0 branches_pruned_list = [] # Compute initial bounds ub_root = math.floor(self.solve_fractional_knapsack_G(self.weights, self.values, self.capacity)) lb_root, lb_root_solution = self.solve_01_knapsack_GApprox() # Priority queue stores (-ub, lb, weight, value, solution, index, lb_solution) # Negate ub to simulate max-heap (heapq defaults to minheap) heapq.heappush(priority_queue, (-ub_root, lb_root, 0, 0, '', 0, lb_root_solution)) iterations = 0 while priority_queue: iterations += 1 priority_queue_length_list.append(len(priority_queue)) branches_pruned_list.append(branches_pruned) neg_ub, lb, current_weight, current_value, current_solution, i, lb_solution = heapq.heappop(priority_queue) ub = -neg_ub #End of branch if i >= n: continue #Prune branches if current_weight > self.capacity or ub <= max_value: branches_pruned += 1 continue #set max_value if lb > max_value: max_value = lb best_solution = current_solution + lb_solution #including item new_weight = current_weight + self.weights[i] if new_weight <= self.capacity: new_value = current_value + self.values[i] remaining_capacity = self.capacity - new_weight ub_incl = new_value + math.floor(self.solve_fractional_knapsack_G(self.weights[i+1:], self.values[i+1:], remaining_capacity)) lb_incl, lb_incl_solution = self.solve_01_knapsack_GApprox(self.weights[i+1:], self.values[i+1:], remaining_capacity) lb_incl += new_value if ub_incl > max_value: heapq.heappush(priority_queue, (-ub_incl, lb_incl, new_weight, new_value, current_solution + '1', i + 1, lb_incl_solution)) #excluding item ub_excl = current_value + math.floor(self.solve_fractional_knapsack_G(self.weights[i+1:], self.values[i+1:], self.capacity - current_weight)) lb_excl, lb_excl_solution = self.solve_01_knapsack_GApprox(self.weights[i+1:], self.values[i+1:], self.capacity - current_weight) lb_excl += current_value if ub_excl > max_value: heapq.heappush(priority_queue, (-ub_excl, lb_excl, current_weight, current_value, current_solution + '0', i + 1, lb_excl_solution))</pre>
Video Timestamp	11.39

Testing no.	11
Test Name	Genetic Algorithm Tournament Selection and Roulette Wheel Selection
Objective being tested	2.3: Heuristic Algorithms => Genetic Algorithm
Reason for test	Testing individual components of the genetic algorithm and how they operate on a population of candidate solutions
Expected result	Each generation a combination of the best solutions are chosen while variance is still added
Actual result	 SUCCESSFUL

Visual Evidence	<pre> if isTournamentSelection: #Tournament Selection (else roulette wheel selection) for x in range(population_size): binary_string = '0' * n max_value = 0 for y in range(tournament_size): index = random.randint(0, population_size - 1) if fitnesses[index] > max_value: binary_string = population[index] max_value = fitnesses[index] new_population[x] = binary_string else: total_fitness = sum(fitnesses) for x in range(population_size): random_variable = random.random() * total_fitness cumulative_sum = 0 solution_index = 0 for p in range(population_size): cumulative_sum += fitnesses[p] if random_variable <= cumulative_sum: solution_index = p break new_population[x] = population[solution_index] </pre>
-----------------	---

Testing no.	12
Test Name	Genetic Algorithm Crossover
Objective being tested	2.3: Heuristic Algorithms => Genetic Algorithm
Reason for test	Testing individual components of the genetic algorithm and how they operate on a population of candidate solutions
Expected result	Takes in 2 binary strings and switches over the prefix/suffix at a random crossover point
Actual result	 SUCCESSFUL
Visual Evidence	<pre> #Cross over certain pairs of the population for x in range(0, population_size-1, 2): cross_over_random_variable = random.random() if cross_over_random_variable > crossover_rate: continue crossover_point = random.randint(0, n-1) new_population[x], new_population[x+1] = new_population[x][:crossover_point] + new_population[x+1][crossover_point:], new_population[x+1][:crossover_point] + new_population[x][crossover_point:] </pre>

Testing no.	13
Test Name	Genetic Algorithm Elitism

Objective being tested	2.3: Heuristic Algorithms => Genetic Algorithm
Reason for test	Testing individual components of the genetic algorithm and how they operate on a population of candidate solutions
Expected result	Preserves a set number of top solutions each generation
Actual result	● SUCCESSFUL
Visual Evidence	<pre>#Elitism indexes = heapq.nlargest(elitism_number, range(len(fitnesses)), key=fitnesses.__getitem__) for x in range(elitism_number): swapping_index = random.randint(0, population_size-1) new_population[swapping_index] = population[indexes[x]]</pre>

Testing no.	14
Test Name	Genetic Algorithm
Objective being tested	2.3: Heuristic Algorithms => Genetic Algorithm
Reason for test	I wanted to test if the multiple systems like crossover and tournament selection were working effectively together in order to generate an accurate solution
Expected result	Solution values are close to greedy values
Actual result	● SUCCESSFUL

Visual Evidence

```
def solve_01_knapsack_GeneticAlg(self):
    #variables
    n = len(self.weights)

    isTournamentSelection = True if self.algorithm_data['selection_type_GA'] == "Tournament selection" else False
    tournament_size = self.algorithm_data['tournament_size_GA']

    GENERATIONS = self.algorithm_data['max_iterations_GA']
    population_size = self.algorithm_data['population_size_GA']
    crossover_rate = self.algorithm_data['cross_over_rate']
    mutation_rate = self.algorithm_data['mutation_rate']
    elitism_rate = self.algorithm_data['elitism_rate']

    elitism_number = int(elitism_rate * population_size)
    global_best_combination = '0' * n
    global_max_value = 0

    population = [''] * population_size
    fitnesses = [0] * population_size
    average_fitnesses = [0] * GENERATIONS
    best_fitnesses = [0] * GENERATIONS
    lowest_fitnesses = [0] * GENERATIONS
    #Generate random population to begin
    for x in range(population_size):
        value, binary_string, weight = self.generate_starting_solution(is_greedy=False)
        population[x] = binary_string
        fitnesses[x] = value

    for i in range(GENERATIONS):
        new_population = [''] * population_size
        if isTournamentSelection:
            #Tournament Selection (else roulette wheel selection)
            for x in range(population_size):
                binary_string = '0' * n
                max_value = 0
                for y in range(tournament_size):
                    index = random.randint(0, population_size - 1)
                    if fitnesses[index] > max_value:
                        binary_string = population[index]
                        max_value = fitnesses[index]
                new_population[x] = binary_string
            else:
                total_fitness = sum(fitnesses)
                for x in range(population_size):
                    random_variable = random.random() * total_fitness
                    cumulative_sum = 0
                    solution_index = 0
                    for p in range(population_size):
                        cumulative_sum += fitnesses[p]
                        if random_variable <= cumulative_sum:
                            solution_index = p
                            break
                new_population[x] = population[solution_index]

        #Cross over certain pairs of the population
        for x in range(0, population_size-1, 2):
            cross_over_random_variable = random.random()
            if cross_over_random_variable > crossover_rate: continue
            crossover_point = random.randint(0, n-1)

            new_population[x], new_population[x+1] = new_population[x][:crossover_point] + new_population[x+1][crossover_point:],
            new_population[x+1][:crossover_point] + new_population[x][crossover_point:]

        #Mutate to introduce variance
        for x in range(population_size):
            for mutation_bit in range(n):
                mutate_random_variable = random.random()
                if mutate_random_variable > mutation_rate: continue
                new_population[x] = (
                    new_population[x][:mutation_bit] +
                    ('1' if new_population[x][mutation_bit] == '0' else '0') +
                    new_population[x][mutation_bit+1:]
                )

        #Elitism
        indexes = heapq.nlargest(elitism_number, range(len(fitnesses)), key=fitnesses.__getitem__)
        for x in range(elitism_number):
            swapping_index = random.randint(0, population_size-1)
            new_population[swapping_index] = population[indexes[x]]

        average_fitnesses[i] = sum(fitnesses) / population_size
        best_fitnesses[i] = max(fitnesses)
        if best_fitnesses[i] > global_max_value:
            global_max_value = best_fitnesses[i]
            global_best_combination = population[fitnesses.index(global_max_value)]
        lowest_fitnesses[i] = min(x for x in fitnesses if x > 0)

        population = new_population
        fitnesses = list(map(self.evaluate, population))
```

Video
Timestamp

12.18

Testing no.

15

Test Name

Ant Colony Optimisation

Objective being tested	2.4: Heuristic Algorithms => Ant Colony Optimisation
Reason for test	I wanted to test if the pheromone trails where accumulating correctly, leading to a valid solution with items selected from the feasible set over time
Expected result	Near optimal value produced close to the greedy values due to the attractiveness of an item contributing to the cumulative probability
Actual result	● SUCCESSFUL

Visual Evidence	<pre> for x in range(ITERATIONS): #Generate probabilities for iteration probabilities = [0] * n for i in range(n): probabilities[i] = (attractiveness[i]**ALPHA) * (pheremones[i]**BETA) #Generate solution for each ant, and update pheremones for next iteration for y in range(number_of_ants): binary_string = ['0'] * n current_weight = 0 current_value = 0 feasible_set = list(range(n)) while feasible_set: feasible_set = [i for i in feasible_set if current_weight + self.weights[i] <= self.capacity] if not feasible_set: break feasible_probabilities = [probabilities[i] for i in feasible_set] total = sum(feasible_probabilities) if total == 0: break Random_Variable = random.random() * total cumulative_sum = 0 item_index = 0 for index, p in zip(feasible_set, feasible_probabilities): cumulative_sum += p if Random_Variable <= cumulative_sum: item_index = index break binary_string[item_index] = '1' current_weight += self.weights[item_index] current_value += self.values[item_index] feasible_set.remove(item_index) if current_value > max_value: global_best_combination = binary_string[:] max_value = current_value try: #Update pheremones pheremone_strength = 1 / (1 + ((max_value - current_value) / max_value)) except: continue for w in range(n): if binary_string[w] == '1': pheremones[w] += pheremone_strength best_value_list.append(max_value) pheremones_storage_list.append(pheremones) </pre>
Video Timestamp	12.57

Testing no.	16
-------------	----

Test Name	Simulated Annealing
Objective being tested	2.5: Heuristic Algorithms => Simulated Annealing
Reason for test	Check if the randomised acceptance probability formula over a large sample size allowed for SA to escape local optima and improve its solution value
Expected result	I wanted to see if it could generate a near optimal solution either from starting with a greedy or random candidate solution
Actual result	 SUCCESSFUL - performs well when starting with a random solution, however often fails to improve greedy solution
Visual Evidence	<pre> while temperature >= minimum_temperature and iteration < max_iterations: temperature_list.append(temperature) Found = False while Found == False: random_bit = random.randint(0, n-1) if solution[random_bit] == '1': new_solution = solution[:random_bit] + ('0') + solution[random_bit + 1:] new_value = value - self.values[random_bit] new_weight = weight - self.weights[random_bit] Found = True else: new_solution = solution[:random_bit] + ('1') + solution[random_bit + 1:] new_value = value + self.values[random_bit] new_weight = weight + self.weights[random_bit] if new_weight <= self.capacity: Found = True if new_value >= value: value = new_value weight = new_weight solution = new_solution if new_value > max_value: max_value = new_value best_solution = new_solution else: #Probability formula #Delta will normalise the range as input data may vary in terms of 'Range of values:' variable delta = (((max_value - new_value)*1) + ((value - new_value)*0)) / max_item_value #relative positive 'worse' difference acceptance_probability = math.exp(-delta / temperature) #print("Difference: " + str(value-new_value) + " acceptance probability: " + str(acceptance_probability)) rand_var = random.random() #worse += 1 if rand_var < acceptance_probability: value = new_value weight = new_weight solution = new_solution #accepted += 1 best_values.append(max_value) current_values.append(value) temperature *= alpha iteration += 1 #print(accepted, worse, accepted/worse, iteration, temperature, value, max_value) </pre>
Video Timestamp	13.27

Testing no.	17
Test Name	Tabu Search
Objective being tested	2.6: Heuristic Algorithms => Tabu Search
Reason for test	Tabu search should be able to quickly find the best neighbouring solution

Expected result	I wanted to see if it could generate a near optimal solution either from starting with a greedy or random candidate solution
Actual result	● SUCCESSFUL - however sometimes reduces into a hill climbing algorithm with greedy initialisation as starts trapped in local optima
Visual Evidence	<pre> for y in range(ITERATIONS): #Efficiently finding next move/neighbour by evaluating possible new solutions and selecting the best non-tabu solution best_neighbour = '' best_neighbour_weight = 0 best_neighbour_value = 0 worse = False for x in range(n): #There are n neighbours if solution[x] == '0': #We flip to 1 if weight + self.weights[x] <= self.capacity: new_neighbour_value = value + self.values[x] new_neighbour = solution[:x] + '1' + solution[x+1:] new_neighbour_weight = weight + self.weights[x] if new_neighbour_value > best_neighbour_value and int(new_neighbour, 2) not in tabu_list: best_neighbour_value = new_neighbour_value best_neighbour = new_neighbour best_neighbour_weight = new_neighbour_weight elif solution[x] == '1': #We flip to 0 new_neighbour_value = value - self.values[x] new_neighbour = solution[:x] + '0' + solution[x+1:] new_neighbour_weight = weight - self.weights[x] if new_neighbour_value > best_neighbour_value and int(new_neighbour, 2) not in tabu_list: best_neighbour_value = new_neighbour_value best_neighbour = new_neighbour best_neighbour_weight = new_neighbour_weight worse = True if best_neighbour_value > max_value: max_value = best_neighbour_value best_solution = best_neighbour if worse: print("accepted worse " + str(y)) value = best_neighbour_value solution = best_neighbour weight = best_neighbour_weight tabu_list.append(int(solution, 2)) while len(tabu_list) > tabu_list_maxsize: tabu_list.pop(0) </pre>
Video Timestamp	14.01

Testing no.	18
Test Name	Particle Swarm Optimisation
Objective being tested	2.7: Heuristic Algorithms => Particle Swarm Optimisation
Reason for test	I decided to test whether each particle class was instantiating correctly, with their own position, velocity and pbest attributes
Expected result	Near optimal value produced
Actual result	● UNSUCCESSFUL - decayed into randomised search, Velocities not following a fixed pattern. Inefficient due to using strings over lists for particle attributes

Visual Evidence	<pre>def solve_01_knapsack_PSO(self): class Particle: def __init__(self, n): self.position = [random.randint(0, 1) for x in range(n)] #represents a possible solution self.velocity = [random.random() for x in range(n)] self.pbest = self.position[:] self.pbest_fitness = 0 self.pbest_list = [] self.velocity_list = [] for particle in swarm: for x in range(n): #UPDATE VELOCITY particle.velocity[x] = (inertia * particle.velocity[x]) + (cognitive_factor * random.random() * ((particle.pbest[x] - (particle.position[x]))) + (social_factor * random.random() * ((gbest[x] - (particle.position[x])))) particle.velocity[x] = max(min(particle.velocity[x], max_velocity), -max_velocity) #PRINT(particle.velocity[x]) #UPDATE POSITION if random.random() < (1 / (1 + math.exp(-steepness * particle.velocity[x]))): particle.position[x] = 1 else: particle.position[x] = 0</pre>
Further Explanation	When printing out data for PSO as it was running, I noticed that the velocity of particles was moving uncontrollably and the output was sometimes invalid, not giving an accurate result. I was using a penalty factor method initially, but decided to switch to a repair method after these results.

Testing no.	19
Test Name	Repair solution function for PSO
Objective being tested	2.7: Heuristic Algorithms => Particle Swarm Optimisation
Reason for test	In most PSO algorithms a repair function is often added in order to maximise a potential valid unoptimised solution, or adjust an invalid solution to make it valid. In this case, a greedy ordering is used to dictate the order in which items are removed or added from a solution
Expected result	Takes in a solution and improves the value
Actual result	 SUCCESSFUL

Visual Evidence

```
def repair(solution, greedy_ordering):
    #ADD phase
    total_weight = sum([self.weights[x] for x in range(len(self.weights)) if solution[x] == 1])
    gap = self.capacity - total_weight
    i = 0
    new_solution = list(solution).copy()

    while gap > 0 and i < n:
        index = greedy_ordering[i]
        if solution[index] == 0 and gap >= self.weights[index]:
            new_solution[index] = 1
            gap -= self.weights[index]
            total_weight += self.weights[index]
            i += 1

    #DROP phase
    over = total_weight - self.capacity
    i = 0
    while over > 0 and i < n:
        index = greedy_ordering[n-1-i]
        if solution[index] == 1:
            new_solution[index] = 0
            over -= self.weights[index]
            i += 1

    return new_solution
```

Testing no.	20
Test Name	Particle Swarm Optimisation Fixed
Objective being tested	2.7: Heuristic Algorithms => Particle Swarm Optimisation
Reason for test	Correctly ensured that velocity of particles are updated correctly and chose to repair each solution over using a penalty factor method
Expected result	Near optimal value produced
Actual result	● SUCCESSFUL

Visual Evidence	<pre> swarm = [Particle(n) for _ in range(population)] iteration = 0 gbest = [] gbest_fitness = 0 #Compute greedy ordering items = [None] * n for x in range(n): items[x] = (self.values[x]/self.weights[x], x) items = sorted(items, reverse=True) greedy_ordering = [b for (a, b) in items] while iteration < max_iterations: for particle in swarm: particle.position = repair(particle.position, greedy_ordering).copy() new_fitness = evaluate_PSO(particle.position) if new_fitness > particle.pbest_fitness: particle.pbest = particle.position[:] particle.pbest_fitness = new_fitness if particle.pbest_fitness > gbest_fitness: gbest = particle.pbest[:] gbest_fitness = particle.pbest_fitness particle.pbest_list.append(particle.pbest_fitness) for particle in swarm: for x in range(n): #UPDATE VELOCITY particle.velocity[x] = (inertia * particle.velocity[x]) + (cognitive_factor * random() * (items[x][1] - particle.position[x])) particle.velocity[x] = max(min(particle.velocity[x], max_velocity), -max_velocity) #print(particle.velocity[x]) #UPDATE POSITION if random.random() < (1 / (1 + math.exp(-steepness * particle.velocity[x]))): particle.position[x] = 1 else: particle.position[x] = 0 particle.velocity_list.append(sum(particle.velocity) / len(particle.velocity)) best_value_list.append(gbest_fitness) iteration += 1 </pre>
Video Timestamp	14.38

Testing no.	21
Test Name	DP Tabulation rendering the table as image
Objective being tested	8.3: Data Graphs => DP Tabulation Table
Reason for test	Since this is the most popular way of solving the knapsack problem, I want to show the key element to the user (the dp array) so that the user can appreciate what is happening
Expected result	Rendered table image using matplotlib

Actual result	UNSUCCESSFUL - Due to the large size of the table (it would need to be cut severely in order to fit into one image)																																																																																																																																																																											
Visual Evidence	<pre>end_time = time.perf_counter() fig = Figure(dpi=250) axis = fig.add_subplot() max_size = 20 m_data = np.array(dp) data = m_data[:max_size, :max_size] df = pd.DataFrame(data) table = axis.table(cellText=df.values, loc='center', rowLabels=list(range(min(n+1, max_size))), colLabels=list(range(min(self.capacity+1, max_size)))) table.scale(1, 2) axis.axis('off') axis.set_title("DP (Tabulation) - 0/1 Knapsack Problem") axis.text(0.5, 0.85, "Capacity", transform=axis.transAxes, fontsize=12, fontweight="bold", va="center", ha="center",) axis.text(-0.05, 0.5, "Items", transform=axis.transAxes, fontsize=12, fontweight="bold", va="center", ha="center", rotation=90) fig.tight_layout()</pre> DP (Tabulation) (Result Data) Value: 112 Weight: 106 Valid: True Time(s): 0.00032639999699313194 Items (w_i, v_i) included: Item 2: (24, 32) Item 3: (12, 14) Item 4: (40, 30) Item 5: (30, 36) Items code: 01111 DP (Tabulation) - 0/1 Knapsack Problem <table><tr><th colspan="2"></th><th colspan="20">Capacity</th></tr><tr><th colspan="2"></th><th>0</th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th><th>7</th><th>8</th><th>9</th><th>10</th><th>11</th><th>12</th><th>13</th><th>14</th><th>15</th><th>16</th><th>17</th><th>18</th><th>19</th></tr><tr><th rowspan="6">Items</th><th>0</th><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><th>1</th><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><th>2</th><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><th>3</th><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td></tr><tr><th>4</th><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td></tr><tr><th>5</th><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td><td>14</td></tr></table> Download Image			Capacity																						0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	Items	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0	0	0	0	14	14	14	14	14	14	14	14	4	0	0	0	0	0	0	0	0	0	0	0	0	14	14	14	14	14	14	14	14	5	0	0	0	0	0	0	0	0	0	0	0	0	14	14	14	14	14	14	14	14
		Capacity																																																																																																																																																																										
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19																																																																																																																																																							
Items	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																																																																																																																																																							
	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																																																																																																																																																							
	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																																																																																																																																																							
	3	0	0	0	0	0	0	0	0	0	0	0	0	14	14	14	14	14	14	14	14																																																																																																																																																							
	4	0	0	0	0	0	0	0	0	0	0	0	0	14	14	14	14	14	14	14	14																																																																																																																																																							
	5	0	0	0	0	0	0	0	0	0	0	0	0	14	14	14	14	14	14	14	14																																																																																																																																																							
Further Explanation	As shown above, the image is fine for small capacities or a small number of items. However, the DP table can quickly grow to millions of cells when the capacity and number of items increase. Rendering such a large matrix as an image becomes impractical. To address this, instead of generating a single image using matplotlib, a dedicated HTML table element will be dynamically created to render the DP table directly for this algorithm.																																																																																																																																																																											

Testing no.	22
Test Name	DP Tabulation rendering the table using a canvas

Objective being tested	8.3: Data Graphs => DP Tabulation Table
Reason for test	In order to fix the issues that arose in test Z, I decided to cut the table to a maximum viewable size of 250 x 250, so that a total of 62500 cells are available to the user. I used a canvas element to ensure that the table can run efficiently.
Expected result	Scrollable cropped canvas embedded within the submit.html file
Actual result	● SUCCESSFUL
Visual Evidence	<pre> //Function that draws the current displayed output in the container function drawVisible() { const scrollTop = container.scrollTop; const scrollleft = container.scrollleft; const startRow = Math.floor(scrollTop / cellHeight); const endRow = Math.min(rows, startRow + Math.ceil(container.clientHeight / cellHeight) + 1); const startCol = Math.floor(scrollleft / cellWidth); const endCol = Math.min(cols, startCol + Math.ceil(container.clientWidth / cellWidth) + 1); //clear output before drawing cells ctx.clearRect(0, 0, canvas.width, canvas.height); //Output text format settings ctx.font = "12px Arial"; ctx.textAlign = "center"; ctx.textBaseline = "middle"; for (let r = startRow; r < endRow; r++) { for (let c = startCol; c < endCol; c++) { //x, y coordinates of the cell const x = c * cellWidth; const y = r * cellHeight; //Draw the cells with appropriate colours if (r === 0 c === 0) { ctx.fillStyle = "#fff"; //Coloured Rectangle ctx.fillRect(x, y, cellWidth, cellHeight); } ctx.strokeStyle = "#000"; //Bordered Rectangle ctx.strokeRect(x, y, cellWidth, cellHeight); ctx.fillStyle = "#000"; if (!(r === 0 && c === 0)) { //Fill text with dp array ctx.fillText(dpData[r][c], x + cellWidth / 2, y + cellHeight / 2); } else { //Top left cell should be unmarked ctx.fillText('#', x + cellWidth / 2, y + cellHeight / 2) } } } } </pre> <pre> end_time = time.perf_counter() m_data = np.array(dp) items_list = np.array(list(range(n + 1))) m_data = np.column_stack((items_list, m_data)) capacity_list = np.array([0] + list(range(self.capacity + 1))) m_data = np.vstack((capacity_list, m_data)) data = m_data[:250, :250] return max_value, ''.join(best_combination), data, end_time </pre>

Video Timestamp	10.49
-----------------	-------

Testing no.	23
Test Name	Rendering Graphs
Objective being tested	8.2: Data Graphs => Plotted Graphs
Reason for test	I wanted a static graph image rendered and then displayed embedded within the html file. In order to do this I used a the figure entity from matplotlib combined with base 64 encoding to embed the images
Expected result	Each specific algorithm (other than DP tabulation) generates a graph with properties and results shown
Actual result	● SUCCESSFUL
Visual Evidence	<pre> output = io.BytesIO() FigureCanvas(fig).print_png(output) r_fig = base64.b64encode(output.getvalue()).decode('utf-8') </pre> <pre> {% else: %} <div id="graphs"> Download Image </div> {% endif %} </pre>

Testing no.	24
Test Name	Decision Tree
Objective being tested	8.1: Data Graphs => Decision Tree
Reason for test	In order to explain the brute force recursive algorithm I thought a rooted tree would be more suitable than a plotted graph
Expected result	Shows a relevant and specific decision tree with a blue selected line corresponding to the item code
Actual result	● SUCCESSFUL

Visual Evidence

```
#Recursively called plotting methods
def plot_node(self, axis, x, y, lastnode=False):
    node_size = 10
    axis.scatter(x, y, s=node_size, color='black', label='node', zorder=2)

    if lastnode:
        axis.scatter(x, y-0.25, s=2, color='black', marker='d', alpha=0.5, label='hidden nodes')
```

```
def solve(capacity=0, n=0, best_combination='', plot_pos_x=0, plot_pos_y=0, dx=20):
    if n == 0:
        capacity = self.capacity

    self.recursion_counter += 1
    if n == max_tree_layer and items > max_tree_layer and plot_pos_y != 0: self.plot_node(axis, plot_pos_x, plot_pos_y, lastnode=True)
    elif n <= max_tree_layer:
        self.plot_node(axis, plot_pos_x, plot_pos_y)

    if self.recursion_counter >= self.algorithm_data['max_iterations_recursive']:
        return (0, '')

    if n == len(self.weights) or capacity == 0:
        return (0, best_combination.ljust(len(self.weights), '0'))
    else:
        including_item = 0
        if self.weights[n] <= capacity:
            #including item
            xi, yi = plot_pos_x - dx, plot_pos_y - 1
            if n < max_tree_layer:
                line = axis.plot([plot_pos_x, xi], [plot_pos_y, yi], color="green", label="included", zorder=1)
                path_edges[n][best_combination + '1'] = line
                including_item, best_combination1 = solve(capacity-self.weights[n], n+1, best_combination + '1', xi, yi, dx / 2)
                including_item += self.values[n]

            #excluding item
            xe, ye = plot_pos_x + dx, plot_pos_y - 1
            if n < max_tree_layer:
                line = axis.plot([plot_pos_x, xe], [plot_pos_y, ye], color="red", label="excluded", zorder=1)
                path_edges[n][best_combination + '0'] = line
                excluding_item, best_combination2 = solve(capacity, n+1, best_combination + '0', xe, ye, dx/2)

            if including_item > excluding_item:
                return (including_item, best_combination1)
            else:
                return (excluding_item, best_combination2)
```

```
#highlight selected path
for depth in range(min(max_tree_layer, items)):
    prefix = solution[:depth + 1]
    line = path_edges[depth].get(prefix)
    if line:
        line[0].set_color('blue')
        line[0].set_label('selected')
```

Video
Timestamp

6.34

Testing no.

25

Test Name

Input Error Handling

Objective
being tested

3.X, 6.X: Exception Handling and Inputting Data

Reason for
test

Seeing if using raise key error is sufficient for alerting user

Expected result	When user doesn't input valid data, it returns the relevant error
Actual result	 UNSUCCESSFUL - Internal Server Error is displayed rather than specific message to user
Visual Evidence	<pre> #Generate data/submit data/load data input_data = request.form['input_data'] if input_data == "generate_random": weights, values, capacity = generate_input_data_binaryKP(int(request.form['number_of_items']), int(request.form['range_ elif input_data == "input_manually": weights = list(map(int, request.form['weights'].split(" "))) values = list(map(int, request.form['values'].split(" "))) capacity = int(request.form['capacity']) elif input_data == "import_csv": file = request.files.get("csv_file") if file and file.filename.endswith(".csv"): df = pd.read_csv(file) if 'weights' not in df.columns: raise KeyError("Column 'weights' not found in CSV.") elif 'values' not in df.columns: raise KeyError("Column 'values' not found in CSV.") elif 'capacity' not in df.columns: raise KeyError("Column 'capacity' not found in CSV.") else: raise KeyError("Not CSV File") weights = df['weights'].tolist() values = df['values'].tolist() capacity_list = df['capacity'].tolist() cleaned_list = [x for x in capacity_list if not math.isnan(x)] if len(cleaned_list) == 0: raise KeyError("No valid (non-NaN) values found in 'capacity'.") capacity = int(cleaned_list[0]) if len(weights) != len(values): raise KeyError("Number of weights entered doesn't match the number of values entered") </pre> Internal Server Error The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.
Further Explanation	After deployment, I realised that using the method of raising a KeyError was insufficient and resulted in an Internal Server Error. This provided the user with no guidance on why this error was caused, so a tailored method had to be implemented.

Testing no.	26
Test Name	Tailored Input Error Handling
Objective being tested	3.X, 6.X: Exception Handling and Inputting Data
Reason for test	Integrating a built in system for input errors so that system doesn't crash

Expected result	User is notified on how they have inputted data wrong with the opportunity to input it again
Actual result	 SUCCESSFUL
Visual Evidence	<pre> #Generate data/submit data/load data input_data = request.form['input_data'] if input_data == "generate_random": weights, values, capacity = generate_input_data_binaryKP(int(request.form['number_of_items']), int(request.form['range_of_v elif input_data == "input_manually": try: weights = list(map(int, request.form['weights'].split(" "))) values = list(map(int, request.form['values'].split(" "))) capacity = int(request.form['capacity']) except: return render_template('index.html', error="Please enter integers seperated by spaces for weights and values and a sing elif input_data == "import_csv": file = request.files.get("csv_file") if file and file.filename.endswith(".csv"): df = pd.read_csv(file) if 'weights' not in df.columns: return render_template('index.html', error="Column 'weights' not found in CSV.") elif 'values' not in df.columns: return render_template('index.html', error="Column 'values' not found in CSV.") elif 'capacity' not in df.columns: return render_template('index.html', error="Column 'capacity' not found in CSV.") else: return render_template('index.html', error="Not CSV File") weights = df['weights'].tolist() values = df['values'].tolist() capacity_list = df['capacity'].tolist() cleaned_list = [x for x in capacity_list if not math.isnan(x)] if len(cleaned_list) == 0: return render_template('index.html', error="No valid (non-NaN) values found in 'capacity'.") capacity = int(cleaned_list[0]) if len(weights) != len(values): return render_template('index.html', error="The number of weights inputted doesn't match the number of values") </pre>
Video Timestamp	4.03

Testing no.	27
Test Name	Problem and Algorithm Descriptions
Objective being tested	4.X: Ease of use
Reason for test	So that the parameters for each algorithm can update according to the selected option
Expected result	Check if javascript updates webpage after menu selection
Actual result	 SUCCESSFUL

Visual Evidence	<pre>function change_algorithm_parameters () { document.getElementById("0/1algorithms").querySelectorAll(".algorithm").forEach(algorithm_setting => { if (algorithm_setting.id === dropdown_binary_algorithm.value) { algorithm_setting.style.display = "inline"; } else { algorithm_setting.style.display = "none"; } }); } const dropdown_binary_algorithm = document.getElementById("algorithm_type"); dropdown_binary_algorithm.addEventListener("change", change_algorithm_parameters);</pre>
Video Timestamp	0.30

Testing no.	28
Test Name	Sliders
Objective being tested	5.1: Control over parameters => Slider Visual
Reason for test	To allow the user to have full control over each algorithm, I thought it would be best to have a slider for each parameter
Expected result	Slider correctly updates parameter and is intuitive to use
Actual result	● UNSUCCESSFUL - just a single slider seems insufficient for user to be able to adjust a variable as it doesn't give enough control especially to touchscreen/phone users
Visual Evidence	<pre><div class="variable"> <label for="cross_over_rate">Cross over rate: </label> <div class="slidecontainer"> <input type="range" min="0" max="1" value="0.6" step="0.01" class="slider" name="cross_over_rate"> </div> <p class="displayoutput"></p> </div></pre>  Cross over rate: <input type="range" value="0.6"/> 0.6
Further Explanation	Above shows an example UI slider for crossover rate with its paired fixed display output. After implementing this method, it was realised that having a textbox for specific input gives the user full control over parameters.

Testing no.	29
Test Name	Sliders with textbox

Objective being tested	5.1: Control over parameters => Slider Visual
Reason for test	Additional textbox should give the user the option to specifically set the parameter at a target value within a given range
Expected result	Slider and textbox should sync together. They should allow for multiple of these 'slider textbox systems' to be stacked next to each other
Actual result	● SUCCESSFUL
Visual Evidence	<pre> <div class="variable"> <label for="evapouration_rate">Evapouration rate: </label> <div class="slidecontainer"> <input type="range" min="0" max="1" step="0.01" value="0.9" class="slider" name="evapouration_rate"> </div> <input type="number" class="slidevalue" min="0" max="1" step="0.01" value="0.9"> </div> </pre> <pre> document.querySelectorAll(".variable").forEach(group => { const slider = group.querySelector(".slidecontainer").querySelector("input[type=range]"); const number_input = group.querySelector(".slidevalue"); // Update number when slider changes slider.addEventListener('input', function() { number_input.value = slider.value; }); // Update slider when number changes (with validation) number_input.addEventListener('input', function() { let value = number_input.value; if (value < Number(slider.min)) value = slider.min; if (value > Number(slider.max)) value = slider.max; slider.value = value; number_input.value = value; }); }); </pre> 

Testing no.	30
Test Name	Timing parent function
Objective being tested	9.1: Comparative data
Reason for test	In order to get the execution time of each algorithm, I am using the time library in python to record the difference in time between when the function is initially called and when an answer is outputted

Expected result	Time is displayed on submit page that is approximately equal to the time taken for this page to load
Actual result	 SUCCESSFUL
Visual Evidence	<pre>def time_and_run_binaryKP(function_index, weights, values, capacity, algorithm_data): problem = BinaryKP(weights, values, capacity, algorithm_data) functions = [problem.solve_01_knapsack_random, problem.solve_01_knapsack_BF_iterative, problem.solve_01_knapsack_BF_recursive, problem.solve_01_knapsack_GA_iterative, problem.solve_01_knapsack_GA_recursive, problem.solve_01_knapsack_GA_iterative] start_time = time.perf_counter() function = functions[function_index] if function == problem.solve_01_knapsack_GA_iterative: Answer = problem.solve_01_knapsack_GA_iterative(generate_graph=True) else: Answer = function() end_time = time.perf_counter() elapsed_time = end_time - start_time fig = None if len(Answer) > 3: #There is a figure fig = Answer[2] solution_value, solution_weight = problem.evaluate(Answer[1], get_total_weight=True) return Answer[0], solution_weight, Answer[1], elapsed_time, Answer[0] == solution_value, fig</pre>

Testing no.	31
Test Name	Generate Random Input Data
Objective being tested	6.1: Inputting Data => Generate Random
Reason for test	The user should be able to generate new data quickly from within the website
Expected result	When the 'Generate Data' button is clicked a new set of data is loaded within the interface with the parameters specified
Actual result	 SUCCESSFUL
Visual Evidence	<pre>def generate_input_data_binaryKP(n, range_of_values, correlation_strength, capacity_load): #request.form['number_of_items'], request.form['range_of_values'], request.form['correlation_strength'], request.form['capacity_load'] #Input data parameters weights = [0] * n values = [0] * n capacity = 0 index = 0 for index in range(n): weight = random.randint(1, range_of_values) weights[index] = weight if correlation_strength == 0: value = random.randint(1, range_of_values) else: value = max(1, min(range_of_values, random.randint(int(correlation_strength * weight), int((2 - correlation_strength) * weight)))) #When correlation strength is 0 between 1 and 2W (still weakly constrained) values[index] = value capacity = max(1, min(sum(weights), int(capacity_load * sum(weights)))) return weights, values, capacity</pre>
Video Timestamp	1.22

Testing no.	32
Test Name	Input Data Manually

Objective being tested	6.2: Inputting Data => Input Manually
Reason for test	Data should be able to be inputted directly into the interface
Expected result	Values inputted displayed in scroll box
Actual result	● SUCCESSFUL
Visual Evidence	<pre>weights = list(map(int, request.form['weights'].strip().split())) values = list(map(int, request.form['values'].strip().split())) capacity = int(request.form['capacity'].strip())</pre>
Video Timestamp	1.56

Testing no.	33
Test Name	Import csv file
Objective being tested	6.3: Inputting Data => Import CSV
Reason for test	Test if the user can upload a file which is then parsed through pandas and ready to use by an algorithm
Expected result	CSV file in correct format is uploaded and read successfully
Actual result	● SUCCESSFUL
Visual Evidence	<pre><div id="import_csv" style="display:none">
 <p>You can upload a .csv file as input data when layed out in the following format: columns ('weights', 'values', 'capacity')</p> <input type="file" name="csv_file" accept=".csv"> <p style="color: blue;"> Choose csv file to upload </p> </div></pre> <pre>df = pd.read_csv(file) weights = df['weights'].tolist() values = df['values'].tolist() capacity_list = df['capacity'].tolist() cleaned_list = [x for x in capacity_list if not math.isnan(x)] capacity = int(cleaned_list[0])</pre>
Video Timestamp	2.18

Testing no.	34
Test Name	Screen size scaling
Objective being tested	7.1: Design Interface
Reason for test	In order to make sure that the website scales dynamically for different screen sizes and therefore can be used by many different devices
Expected result	When the screen width is reduced the screen formats to better suit the new format
Actual result	 SUCCESSFUL

Visual Evidence

```
/* 893 px width aspect ratio changes */

@media (max-width: 893px) {
  body {
    margin: none;
    margin-left: 5px;
  }

  #title {
    flex-direction: column-reverse;
    align-items: center;    /* Vertical */
    text-align: center;
  }

  .row {
    flex-direction: column;
  }

  .column1, .column2, .column3, .column4 {
    width: 100%;
    padding: 10px;
  }

  #title h1 {
    margin-bottom: 0;
  }

  #title img {
    width: 192px;
    height: 180px;
  }

  #graphs {
    width: 96%;
    margin: 0px;
  }
}
```

A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the Knapsack Problem



Problem Description:

Given a set of 'n' items where each item 'i' has a weight and a value (w_i, v_i) and a knapsack with a maximum capacity 'W', what subset of items maximises the total value of the knapsack without exceeding its weight limit.

$$\begin{aligned} &\text{Maximize} && \sum_{i=1}^n v_i x_i \\ &\text{subject to} && \sum_{i=1}^n w_i x_i \leq W, \\ &&& x_i \in \{0, 1\}, \quad i = 1, \dots, n \end{aligned}$$

Algorithm:

Choose an algorithm:

Randomised Search

Uniformly generates random solutions over a given time period, and stores the best valid solution.

Search time limit:

Data:

Input data:

Number of items:

Range of values:

Correlation strength:

Capacity load:

Capacity: 0

Items (w_i, v_i):

3 - Conclusion

The above testing plan is now finished with code and testing screenshots also included. All unsuccessful tests have been subsequently corrected shown by the test after. This concludes the testing section and now I will begin to evaluate my product.

A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the Knapsack Problem

Evaluation Section

Name: Aarav Bhatnagar - Centre Number: 39253 - Candidate Number: 8015



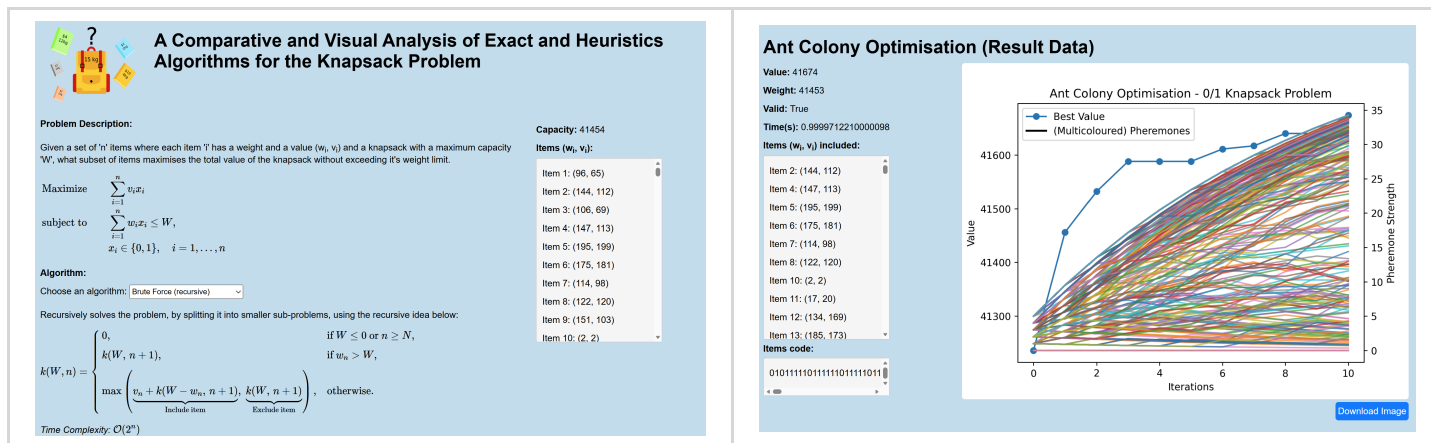
Table of Contents

- [1 - Introduction](#) (pg 168)
- [2 - Final Interview with Primary User](#) (pg 169)
- [3 - Analysis of Feedback Given](#) (pg 170)
- [4 - Marking against Objectives](#) (pg 170)
- [5 - My Findings \(what algorithms work best\)](#) (pg 173)
- [6 - Future Improvements](#) (pg 173)
- [7 - Conclusion](#) (pg 174)

1 - Introduction

In this section I will reflect on my final product and evaluate it against my initial objectives. I will show my website to my primary user Ms Lynn and gain feedback for the product. Additionally, I will myself evaluate what I found when using the website and what algorithms are best suited for this problem.

2 - Final Interview with Primary User



Images taken from knapsackproblem.com

I did a live demonstration of the product, demonstrating most of the algorithms and all of the features of the product. I then talked through what went well and what improvements could be made. Here are the following written questions Ms Lynn answered for me after the interview:

1 - Do you find the user interface intuitive and clear to use?

Yes, the interface is clear and easy to use. As the data set needs to be generated before choosing the algorithm, it would be more intuitive if the menu for this was first, but that is a minor change, and doesn't stop it being usable.

2 - Are the sliders and textboxes helpful when entering data?

Yes, I like the choice of slider or textbox. The sliders are easier if using a touch screen, but having the textbox is good if wanting a specific value, as this can be difficult to achieve with a slider.

3 - Do you think there is sufficient description of each algorithm?

Yes, there is sufficient description for the purpose of this tool, but having a link to further information would be a nice addition, to help if students/teachers are unclear after reading the description, or just want more information. This could be links to external sites.

4 - In your experience, do you think the graphs will help users be able to monitor how each algorithm works?

Yes, graphs are useful, and I liked that the style of graph was chosen to make it relevant to each particular algorithm. It can be hard to visualise what is happening with the dataset in complex algorithms, so having the graphs to refer to would make it easier to explain too.

5 - What aspects of the project did you like?

The simple layout combined with clear explanations, and quick load speeds for the graphs. Having the items included as a list was useful as well.

6 - What aspects of the project do you think I could improve on/add in future iterations?

Just minor changes as mentioned in the answers to 1 and 3 above.

7 - Do you think you will find this project useful in teaching?

Yes, especially as it includes heuristic approaches, as that is always an area that lacks concrete examples in my experience.

3 - Analysis of Feedback Given

After speaking to my primary user and reading the written responses, I am pleased with the feedback I received. In summary, Ms Lynn enjoyed using my product with only a few minor changes that I will discuss in the '*Future Improvements*' section. Ms Lynn highlighted that she liked many parts of both the design interface and the efficiency of the algorithms. Sliders, graphs and clear explanations seems to be the correct decision making. Most importantly, she acknowledged that my product will be useful in teaching, which resolves the initial problem that I first proposed in the analysis section and the goal of my project.

I talked to my computing class, in order to get a student's perspective of the project. They reflected well on the design interface and it's ease of use. As most of the user's had limited experience with the problem and algorithms they found the descriptions very helpful. They also enjoyed seeing the 'Import CSV' feature as it allows for third party software to connect to the website and generate custom data with specific distributions. I also explained the ideas behind heuristic algorithms and how they were mostly inspired by natural life processes, which they found very interesting.

4 - Marking against Objectives

Number	Objective Name	Explanation	Link To Evidence	Passed
1.1	Brute Force	(Iterative and Recursive) Algorithms runs as expected providing an exact solution	Test 2 (iterative), Test 3 (recursive)	✓
1.2	Dynamic Programming	(Memoization and Tabulation) Algorithms runs as expected providing an exact solution	Test 7 (memoization), Test 8 (tabulation)	✓
1.3	Branch and Bound	Algorithm runs as expected providing an exact solution	Test 10	✓
2.1	Randomised Search	Algorithm runs as expected	Test 4	✓

Number	Objective Name	Explanation	Link To Evidence	Passed
2.2	Greedy Approximation	Algorithm runs as expected providing a near optimal solution	Test 5	✓
2.3	Genetic Algorithm	Algorithm runs as expected providing a near optimal solution	Test 14	✓
2.4	Ant Colony Optimisation	Algorithm runs as expected providing a near optimal solution	Test 15	✓
2.5	Simulated Annealing	Algorithm runs as expected providing a near optimal solution	Test 16	✓
2.6	Tabu Search	Algorithm runs as expected providing a near optimal solution	Test 17	✓
2.7	Particle Swarm Optimisation	Algorithm runs as expected providing a near optimal solution	Test 20	✓
3.1	Manual Input Error	The website returns a specific error when an invalid input is entered manually	Test 26	✓
3.2	CSV Input Error	A detailed error of the CSV file is shown when not inputted in specified format	Test 26	✓
4.1	Problem Description	A description of the 0/1 optimisation version of the knapsack problem is displayed at the top of the home page	Test 27	✓
4.2	Algorithm Description	For each algorithm there is a description under the select menu	Test 27	✓

Number	Objective Name	Explanation	Link To Evidence	Passed
5.1	Slider Visual	Sliders and textboxes sync together allowing for the user to have full control over their input parameters	Test 29	✓
5.2	Algorithm Parameter	The sliders change values that in turn effect the performance of the algorithm	Any of the other algorithm tests	✓
6.1	Generate Random	Data can be quickly generated randomly and adaptably directly on the website	Test 31	✓
6.2	Input Manually	Exact data values can be passed and loaded directly into the program	Test 32	✓
6.3	Import CSV	A CSV file can be parsed and also loaded directly into the program	Test 33	✓
7.1	Design Interface	The format of the screen changes as the width reduces to better fit the new size	Test 34	✓
8.1	Decision Tree	A binary rooted tree is generated to mimic the pattern of the recursive calls for the brute force recursive approach	Test 24	✓
8.2	Plotted Graphs	Each algorithm has their own visual and data that is displayed on the result page	Test 23	✓
8.3	DP Tabulation Table	A virtual table is used to display the first 250x250 row and columns of the DP table	Test 22	✓
9.1	Comparative Data	The weight, value and execution time is all recorded	Test 30	✓

Number	Objective Name	Explanation	Link To Evidence	Passed
		and displayed to the user		

My product has successfully passed all written objectives as seen and evidenced in table above.

5 - My Findings (what algorithms work best)

During the creation of this project, I tested it many times to ensure the code was functioning as expected. However, this also gave me an insight into what algorithms perform better than others. I concluded that it depends on **multiple key factors**, whether the user wants the **exact value** (rarely required in real world use cases), the **size of data** (number of items 'n') and the computational resources/**time available**. Here are my findings:

- **Best Exact Algorithm (<500 items):** **Branch and Bound**

This algorithm beats the more popular dynamic programming approaches that are commonly taught with respect to this problem. It depends on the specific data given, as this relates to the number of branches that will be pruned, but on average this algorithm performs very well to get an exact solution for 500 items in 15-20s

- **Best Heuristic Algorithm (500+ items):** **Tabu Search with greedy initialisation**

The Tabu search is often able to improve on the greedy solution and get a value extremely close if not the exact value, in under one second. It has often proven better than simulated annealing, which works in a similar way.

- **Best on Extremely Large Data Sets (10,000 - 1,000,000 items):** **Greedy Approximation**

Due to the $O(n \log n)$ time complexity and the inherently greedy nature of the problem, this method which describes quite a simple idea does usually provide a near optimal solution. The absolute error will increase for larger values of n (while the relative error remains constant) however this outweighs the significant time needed to calculate an exact solution.

(Examples of me running these algorithms can be found in the testing video where I ran brute force solutions on a data set of 12 items and other algorithms on a data set of 70 items. Additionally, testing can be done directly on my website - knapsackproblem.com)

6 - Future Improvements

After gaining feedback from my primary user and through my own experience when building the product, here are a few options I may have explored further if I had more time:

- As discussed in the analysis section, one of my acceptable limitations was to only include the 0/1 optimisation version of the knapsack problem. **If I had more time, I would further test both the exact and heuristic algorithms on more complex versions of the knapsack problem** i.e. the multidimensional knapsack problem or the quadratic knapsack problem or even on other NP complete problems such as TSP or graph colouring. I think this would give the user to compare each algorithm in a wider depth of scenarios
 - **Including links to academic papers that further explore each algorithm may make my product more suitable to new users.** When developing and researching the algorithms, I looked at many papers in my documented design section (as evidenced in the bibliography), and therefore linking these directly on the website would allow the user to read about the mathematical foundations of each algorithm before testing them using my product. *(This was suggested by the primary user)*
 - I thought that the next potential step to increase the ease of use for the product was to **automate a part of the process through automatically generating a report when running data on multiple algorithms**. The user could enter specific guidelines and select the algorithms they want comparing, before a full new custom report is created including multiple test cases, accuracy and time complexity graphs overlayed over each other. This also may make it easier for user to explicitly compare between algorithms
 - Changing the random data generator part to be **pseudorandom**. This means that the user could leave the website, and open it from a different website, re-input the seed and be able to continue testing algorithms on the same data set
 - **Adjusting the design interface** on the home page to make it more clear that the user should generate data first. This could be very quickly adjusted in the html and is only an aesthetic change. *(This was suggested by the primary user)*
 - Calculating the **memory usage** would allow the user to also compare the space efficiency of different algorithms. This is often overlooked, however can be equally important to the time complexity of an algorithm.
-

7 - Conclusion

I think that my final product was a success as it passes all the objectives I initially set out to achieve. I like the way the user interface has developed over time and was impressed by the ingenuity of all the algorithms I researched. I also hope that other people interested in the theory of computation and algorithm design will use my product as a learning resource as well as a gateway to a much broader field. Choosing the knapsack problem has served well to capture people's attention through its perceived simplicity, but has proved to be much harder and more complex beyond the surface.